FBLA · Introduction to Programming · 2026

# The Financial Responsibilities of Owning an Alligator

Source Code Reference Book

Built with Processing 4 · Java Mode
16 Source Files · Full Syntax Highlighting

**Competition Year 2025–2026**

# Table of Contents


**§1 Main Entry Point** `A_Main.pde` **4**
setup() and draw() — program initialization and screen-state switcher

**§2 User Input & Interaction** `B_Interaction.pde` **6**
mousePressed, keyPressed, mouseDragged, mouseWheel — all event handlers and input utilities

**§3 Music & Volume Controls** `C_Music.pde` **29**
ControlP5 volume slider and music toggle with event-safe callback handling

**§4 Utilities, Assets & Save System** `D_General_Functions.pde` **31**
fileWork(), Fade class, name validation, saveGame(), loadGame(), JSON helpers

**§5 Home Screen** `E_Home_Screen.pde` **44**
Home screen rendering, fade-in animation, instructions and music overlays

**§6 Cutscene & Pet Naming** `F_Adoption_Cutscene.pde` **46**
Adoption center cutscene, sign reveal animation, alligator customization and naming screen

**§7 Main Gameplay Screen** `G_Main_Screen.pde` **51**
Core HUD, stat bars, tutorial popups, day progression, game over screen

**§8 Inventory System** `G_Inventory.pde` **62**
Item data arrays, 12-slot inventory panel, item detail display helpers

**§9 Store Panel** `G_Store.pde` **70**
Medicine, snacks and meat tabs — selection, purchase, and store fade transitions

**§10 Bank Panel** `G_Bank.pde` **76**
Transaction log, scrollable viewport, personalized financial advice system

**§11 Economy & Jobs** `G_Earn.pde` **80**
Earn panel, job finder (cashier/barista/manager), tasks and upgrade system

**§12 Services Panel** `G_Services.pde` **87**
Vet, walker, cleaner services and prescription course management

**§13 Rest System** `G_Rest.pde` **91**
Gradient rest bar, bouncing timing marker, rest attempt tracking

**§14 Achievement System** `G_Achievements.pde` **94**
30 tiered achievements with progress tracking, priority sorting, and reward collection

**§15 Minigames** `H_Play_Screen.pde` **101**
Swamp Hop (side-scroller), Snack Snatch (food catcher), Fetch Frenzy (top-down)

**§16 Pet Class** `I_Alligator_Class.pde` **117**
Pet class, eat() method with 40+ foods, mood sprites, stat clamping, skin tint system

§ 1

A_Main.pde

# Main Entry Point

setup() and draw() — program initialization and screen-state switcher

```
// ========================
// Imports & Libraries
// ========================


// Sound playback for music and effects
import processing.sound.*;

// ControlP5 for UI elements (buttons, inputs, sliders)
import controlP5.*;

// OpenGL / P2D renderer for performance and graphics
import processing.opengl.*;

import processing.event.MouseEvent;
// ========================
// Global Objects
// ========================

// Background music file
SoundFile music;


// ========================
// Setup Function
// Runs once at program start
// ========================
void setup() {

  // Set window size and use P2D renderer
  size(1100, 700, P2D);

  // Enable smoother edges
  smooth(4);

  // Lock pixel density for consistent UI scaling
  pixelDensity(1);


  // Runs all essential code required in the setup function
  fileWork();


  // Initialize music volume, playback, and settings
  musicAdjusters();

  // Set up pet naming input screen
  nameinput();

  // Allows for layering of ControlP5 elements
  cp5.setAutoDraw(false);

  // Restore previous session if a save file exists
  loadGame();
}


// ========================
```

```
// Draw Function
// Runs every frame
// Handles screen/state switching
// ========================
void draw() {
  // Home screen (main menu)
  if (isHomeScreenVisible == true) {
    homescreen();
    cp5.draw();
  }

  // Intro cutscene
  else if (isCutsceneActive == true) {
    cutscene();
  }

  // Pet naming screen
  if (isNamingActive == true) {
    namingalligatorsegment();
  }

  // Main gameplay screen
  if (isGameStarted == true) {
    mainscreen();
  }
}
```

§ 2

B_Interaction.pde

# User Input & Interaction

mousePressed, keyPressed, mouseDragged, mouseWheel — all event handlers and input utilities

```
// ========================
// B_Interaction.pde
// All user input: mousePressed, keyPressed, keyReleased, mouseDragged,
// mouseWheel, mouseReleased, plus shared input utility helpers.
//
// Convention: all hit-detection uses raw coordinate comparisons.
// Popup guard booleans prevent multiple screens from activating simultaneously.
// ========================

boolean isMouseInSwampHopArea; // true when the mouse is inside the Swamp Hop minigame panel

// Temporary item category flags used during inventory/store sell logic
boolean isMedicine = false;
boolean isMeat = false;
boolean isMoveLeft = false;
boolean isMoveRight = false;


// ========================
// Utility: findItemIndex
// Linear search through an item name array.
// Returns the matching index, or -1 if the item is not in the array.
// Used to categorize inventory items into medicine / snack / meat buckets.
// ========================
int findItemIndex(String item, String[] arr) {
  for (int i = 0; i < arr.length; i++) {
    if (arr[i].equals(item)) return i;
  }
  return -1;
}


// ========================
// Utility: removeInventorySlot
// Marks a slot as EMPTY and left-shifts all remaining items to compact the list.
// Ensures the selected slot index stays within valid bounds after removal.
// ========================
void removeInventorySlot(int slotIndex) {
  inventorySlots[slotIndex] = "EMPTY";
  for (int i = slotIndex; i < inventorySlots.length - 1; i++) {
    inventorySlots[i] = inventorySlots[i + 1];
  }
  inventorySlots[inventorySlots.length - 1] = "EMPTY";
  if (selectedInventorySlot >= inventorySlots.length) selectedInventorySlot = inventorySlots.length - 1;
}


// ========================
// Utility: logSellTransaction
// Canonical way to add sell income — updates money, lifetime earnings,
// transaction count, and the bank log in one call.
// ========================
void logSellTransaction(String itemName, float sellPrice) {
  money += sellPrice;
  totalMoneyEarned += sellPrice;
  bankTransactionsLoggedCount++;
  bankTransactionLog.add("Transaction: Sold " + itemName + " (+$" + nf(sellPrice, 0, 2) + ")");
}
```

```java


// ========================
// Utility: setMinigameEntry
// Ensures exactly one minigame active flag is true at a time.
// Centralizes the logic so callers never accidentally set conflicting flags.
// ========================
void setMinigameEntry(boolean swamp, boolean snatch, boolean fetch) {
  isEnterSwampHop    = swamp;
  isEnterSnackSnatch = snatch;
  isEnterFetchFrenzy = fetch;
  isOnChoiceScreen   = false;
}

void mousePressed() {
  // Capture current screen states at the moment of click to avoid mid-handler transitions
  // causing incorrect branching (e.g., a click that changes isOnChoiceScreen mid-frame)
  boolean wasOnChoiceScreen = isOnChoiceScreen;
  boolean wasEnteringSwamp = isEnterSwampHop;
  boolean wasEnteringSnack = isEnterSnackSnatch;
  boolean wasEnteringFetch = isEnterFetchFrenzy;

  // Pre-compute whether the click is inside the Swamp Hop panel area
  isMouseInSwampHopArea = (mouseX > 88 && mouseX < 373 && mouseY > 258 && mouseY < 658);

  // ========================
  // Home Screen: Button Clicks
  // Handles Instructions, Play, and Music Settings buttons
  // ========================
  if (isHomeScreenVisible) {
    // "Instructions" button (left button) — only when music settings is closed
    if (mouseX > width * 0.13f && mouseX < width * 0.37f &&
        mouseY > height * 0.67f && mouseY < height * 0.8f &&
        isShowingMusicSettings == false) {
      isShowingInstructions = true;
    }

    // "Play" button (center) — only when neither overlay is open
    if (mouseX > width * 0.38f && mouseX < width * 0.61f &&
        mouseY > height * 0.67f && mouseY < height * 0.8f &&
        isShowingMusicSettings == false && isShowingInstructions == false) {
      isHomeScreenVisible = false;
      isCutsceneActive = true;
    }

    // "Music Settings" button (right) — only when instructions overlay is closed
    if (mouseX > width * 0.615f && mouseX < width * 0.85f &&
        mouseY > height * 0.67f && mouseY < height * 0.8f &&
        isShowingInstructions == false) {
      isShowingMusicSettings = true;
    }

    // Close button on the instructions overlay
    if (mouseX > width * 0.785f && mouseX < width * 0.84f &&
        mouseY > height * 0.11f && mouseY < height * 0.185f) {
      isShowingInstructions = false;
    }

    // Close button on the music settings overlay
    if (mouseX > width * 0.68f && mouseX < width * 0.73f &&
        mouseY > height * 0.41f && mouseY < height * 0.48f) {
      isShowingMusicSettings = false;
    }
  }

```

```
124 ■   // ========================
125 ■   // Adoption Center: Pet Selection
126 ■   // Clicking either the sign or the animal portrait chooses that pet type
127 ■   // ========================
128 ■   if (isInsideAdoptionCenter == true) {
129 ■     // Dog sign (button label area) OR dog portrait (larger clickable sprite region)
130 ■     if ((mouseX > 223 && mouseX < 350 &&
131 ■          mouseY > 450 && mouseY < 484) ||
132 ■         (mouseX > 229 && mouseX < 367 &&
133 ■          mouseY > 189 && mouseY < 426)) {
134 ■       isDogSelected = true;
135 ■     }
136 ■
137 ■     // Cat sign OR cat portrait
138 ■     if ((mouseX > 768 && mouseX < 891 &&
139 ■          mouseY > 450 && mouseY < 482) ||
140 ■         (mouseX > 757 && mouseX < 871 &&
141 ■          mouseY > 227 && mouseY < 415)) {
142 ■       isCatSelected = true;
143 ■     }
144 ■   }
145 ■
146 ■   // ========================
147 ■   // Naming Screen: Alligator Color Selection
148 ■   // Three "SELECT" buttons are evenly spaced 300px apart from the base X position.
149 ■   // selectedAlligatorSkin controls which skin tint is applied in I_Alligator_Class.pde
150 ■   // ========================
151 ■   if (isNamingActive) {
152 ■     // Left alligator (default / no tint) — index 0
153 ■     if (mouseX > ((width * 0.27 - 300) + ((width / 2) * 0.9) / 2) - 60 &&
154 ■         mouseX < ((width * 0.27 - 300) + ((width / 2) * 0.9) / 2) + 60 &&
155 ■         mouseY > 659 - 17.5 &&
156 ■         mouseY < 659 + 17.5) {
157 ■       selectedAlligatorSkin = 0;
158 ■     }
159 ■     // Center alligator (green tint) — index 1
160 ■     else if (mouseX > ((width * 0.27) + ((width / 2) * 0.9) / 2) - 60 &&
161 ■              mouseX < ((width * 0.27) + ((width / 2) * 0.9) / 2) + 60 &&
162 ■              mouseY > 659 - 17.5 &&
163 ■              mouseY < 659 + 17.5) {
164 ■       selectedAlligatorSkin = 1;
165 ■     }
166 ■     // Right alligator (blue tint) — index 2
167 ■     else if (mouseX > ((width * 0.27 + 300) + ((width / 2) * 0.9) / 2) - 60 &&
168 ■              mouseX < ((width * 0.27 + 300) + ((width / 2) * 0.9) / 2) + 60 &&
169 ■              mouseY > 659 - 17.5 &&
170 ■              mouseY < 659 + 17.5) {
171 ■       selectedAlligatorSkin = 2;
172 ■     }
173 ■   }
174 ■
175 ■   // ========================
176 ■   // Main Screen: Top-Level Click Routing
177 ■   // Checks the main gameplay screen actions only when isOnMainScreen is active.
178 ■   // Popup guards prevent multiple overlays from opening simultaneously.
179 ■   // ========================
180 ■   if (isOnMainScreen == true) {
181 ■
182 ■     // Game over screen — "QUIT" button (centered, y = height/2 + 155 ± 21)
183 ■     if (isGameOver) {
184 ■       if (mouseX > width/2 - 90 && mouseX < width/2 + 90 &&
185 ■           mouseY > height/2 + 134 && mouseY < height/2 + 176) {
186 ■         exit();
187 ■       }
188 ■       return;  // block all other main-screen clicks while game over is shown
```

```
    }

    // "Confirm Quit" button inside the quit confirmation dialog
    if (mouseX>482 && mouseX<617 && mouseY>307 && mouseY<344 && isShowingQuitDialog) {
      quit();
    }

    // --- Popup Dismiss (the shared X / close button used by all main screen popups) ---
    // A single hit region at the top-right of every popup closes whichever is active.
    if (isWelcomePopupVisible || isShowingCantSell || isShowingPlayPopup || isShowingEarnPopup ||
isShowingJobPopup || isShowingFirstHelpPopup || isShowingTreatmentPopup || isShowingBankPopup ||
isShowingRestPopup || isShowingStoreClosedPopup || isNextDayPopupOpen || isShowingQuitDialog) {
      if (mouseX > 730 && mouseX < 777 &&
          mouseY > 215 && mouseY < 254) {

        if (isWelcomePopupVisible) isWelcomePopupVisible = false;
        if (isShowingCantSell) isShowingCantSell = false;
        if (isShowingEarnPopup) {
          isShowingEarnPopup = false;
          hasShownEarnPopup = true;
          earnPopupDelayTimer = 0;
        }
        if (isShowingPlayPopup) {
          isShowingPlayPopup = false;
          hasShownPlayPopup = true;
          playPopupDelayTimer = 0;
        }
        if (isShowingJobPopup) {
          isShowingJobPopup = false;
          hasShownJobPopup = true;
        }
        if (isShowingFirstHelpPopup) {
          isShowingFirstHelpPopup = false;
          hasShownFirstHelpPopup = true;
          isHelpTaskPending = false;
        }
        if (isShowingTreatmentPopup) {
          isShowingTreatmentPopup = false;
          if (!isShowingVetFailedPopup) {
            hasShownTreatmentPopup = true;
          }
          isShowingVetFailedPopup = false;
          vetTreatmentMessage = "";
        }
        if (isShowingBankPopup) {
          isShowingBankPopup = false;
          hasShownBankPopup = true;
        }
        if (isShowingRestPopup) {
          isShowingRestPopup = false;
          hasShownRestPopup = true;
        }

        if (isShowingStoreClosedPopup) {
          isShowingStoreClosedPopup=false;
        }
        if (isNextDayPopupOpen) {
          isNextDayPopupOpen=false;
        }
        if (isShowingQuitDialog) {
          isShowingQuitDialog=false;
        }
      }
    }

```

```
    // --- Bank Scrollbar: Begin Drag ---
    // Calculates the current thumb position and starts a drag if the user clicks it.
    float bankMaxScroll = max(0, bankScrollContentHeight - bankViewportHeight);
    float bankThumbH;
    float bankThumbY;

    if (bankScrollContentHeight > bankViewportHeight) {
      bankThumbH = max(40, (bankViewportHeight / bankScrollContentHeight) * bankScrollbarHeight);
      bankThumbY = bankScrollbarY + (bankScrollOffset / bankMaxScroll) * (bankScrollbarHeight - bankThumbH);
    } else {
      bankThumbH = bankScrollbarHeight;
      bankThumbY = bankScrollbarY;
    }

    if (mouseX >= bankScrollbarX && mouseX <= bankScrollbarX + bankScrollbarWidth &&
        mouseY >= bankThumbY && mouseY <= bankThumbY + bankThumbH &&
        bankScrollContentHeight > bankViewportHeight) {
      isDraggingBankScrollbar = true;
      bankScrollThumbOffsetY = mouseY - bankThumbY;
    }

    // --- Inventory Button (bottom-left action circle) ---
    if (dist(mouseX, mouseY, 239.5f, 602) <= 49.5f &&
        !isShowingRestPopup && !isRestOpen && isWelcomePopupVisible == false && !isStoreOpen &&
        !isServicesOpen && !isEarnPanelOpen && !isBankOpen && isOnMainScreen &&
        !isShowingCantSell && !isShowingPlayPopup && !isShowingEarnPopup && !isShowingJobPopup &&
        !isShowingFirstHelpPopup && !isShowingTreatmentPopup && !isShowingBankPopup && !isShowingStoreClosedPopup && !isNextDayPopupOpen  && !isShowingQuitDialog) {
      isInventoryVisible = true;
      hasOpenedInventory = true;
    }

    // --- Rest Button — only available after the rest popup has been shown once ---
    if (dist(mouseX, mouseY, 439, 602) <= 50 &&
        !isRestOpen && hasShownRestPopup && !isServicesOpen && !isEarnPanelOpen && !isStoreOpen &&
        !isBankOpen && isOnMainScreen && !isShowingCantSell && !isShowingPlayPopup &&
        !isShowingEarnPopup && !isShowingJobPopup && !isShowingFirstHelpPopup && !isWelcomePopupVisible &&
        !isShowingTreatmentPopup && !isShowingBankPopup && !isShowingStoreClosedPopup && !isNextDayPopupOpen  && !isShowingQuitDialog) {
      isRestOpen = true;
      hasUsedRest = true;
    }

    // --- Bank Button (bottom-right action circle) ---
    if (dist(mouseX, mouseY, 861, 602) <= 50 &&
        !isRestOpen && isWelcomePopupVisible == false && !isStoreOpen &&
        !isServicesOpen && !isEarnPanelOpen && !isBankOpen && isOnMainScreen &&
        !isShowingCantSell && !isShowingPlayPopup && !isShowingEarnPopup && !isShowingJobPopup && !isWelcomePopupVisible &&
        !isShowingFirstHelpPopup && !isShowingTreatmentPopup && !isShowingBankPopup && !isShowingStoreClosedPopup  && !isNextDayPopupOpen  && !isShowingQuitDialog) {
      isBankOpen = true;
      hasViewedBank = true;
    }

    // --- Close Inventory (X button in top-right of inventory panel) ---
    if (isInventoryVisible && mouseX > 933 && mouseY < 171 && mouseX < 977 && mouseY > 132) {
      isInventoryVisible = false;
    }

    // --- Play / Minigame Button — unlocked only after the play popup has been shown ---
    if (hasShownPlayPopup == true && !isRestOpen && !isBankOpen && isShowingPlayPopup == false &&
        dist(mouseX, mouseY, 656, 602) <= 49.5f && !isServicesOpen && !isEarnPanelOpen &&
```

```
        !isInventoryVisible && isOnMainScreen && !isShowingCantSell && !isShowingPlayPopup && !isStoreOpen &&
        !isShowingEarnPopup && !isShowingJobPopup && !isShowingFirstHelpPopup && !isWelcomePopupVisible &&
        !isShowingTreatmentPopup && !isShowingBankPopup && !isShowingStoreClosedPopup  && !isNextDayPopupOpen  && !isShowingQuitDialog) {
      isPlayClicked = true;
      isExitingMinigame = false;
      currentPlaySessionMoneyEarned = 0;  // reset per-session earnings tracker
      minigameFade.setBlack();            // reset minigame fade to black for fresh entry
      isEnterSwampHop = false;
      isEnterSnackSnatch = false;
      isEnterFetchFrenzy = false;
      isOnChoiceScreen = true;
    }
  }

  // ========================
  // Inventory / Store: Slot Selection
  // A 3x4 grid of slots; determines which slot index was clicked.
  // xs[] and ys[] define column and row boundaries respectively.
  // ========================
  boolean medicinegiven;

  if (isInventoryVisible || isViewingMedicineTab || isViewingSnacksTab || isViewingMeatTab) {
    float[] xs = {110, 256.66f, 403.32f, width/2};
    float[] ys = {182, 279.625f, 377.25f, 474.875f, 572.5f};
    int slotIndex = 0;

    for (int r = 0; r < 4; r++) {
      for (int c = 0; c < 3; c++) {
        if (mouseX > xs[c] && mouseX < xs[c+1] &&
            mouseY > ys[r] && mouseY < ys[r+1]) {
          selectedInventorySlot = slotIndex;
        }
        slotIndex++;
      }
    }
  }

  // ========================
  // Inventory: Use (Feed) Selected Item
  // "FEED" button — applies the item's stat effects to the alligator via eat().
  // The first use always consumes the tutorial steak; subsequent uses check stock.
  // ========================
  if (isInventoryVisible == true && selectedInventorySlot != -1) {
    if (mouseX >= 585.125f && mouseX <= 760 &&
        mouseY >= 474.875f && mouseY <= 546) {

      inventoryItemsUsedCount++;

      if (hasFedSteak == false) {
        hasFedSteak = true;
        selectedItemName = "Steak";
        isInventoryVisible = false;
        alligator.eat(selectedItemName);
        inventorySlots[0] = "EMPTY";
        timesFedPet++;

      } else {
        selectedItemName = inventorySlots[selectedInventorySlot];
        isInventoryVisible = false;

        if (selectedItemName.equals("Enrofloxacin")) {
          hasGivenFirstMedicine = true;
        }
```

```

        alligator.eat(selectedItemName);

        int medIndex   = findItemIndex(selectedItemName, medicineItemList);
        int snackIndex = findItemIndex(selectedItemName, snackItemList);
        int meatIndex  = findItemIndex(selectedItemName, meatItemList);

  if (medIndex != -1) {
    if (medicineQuantities[medIndex] == 1) {
      medicineQuantities[medIndex] = 0;
      removeInventorySlot(selectedInventorySlot);
    } else {
      medicineQuantities[medIndex] = medicineQuantities[medIndex] - 1;
    }

    if (isPetSick && medIndex == prescribedMedicineIndex) {
      if (lastDoseTakenDay != day) {
        prescriptionDaysCompleted++;
        lastDoseTakenDay = day;
      }

      if (prescriptionDaysCompleted >= prescriptionDaysRequired) {
        isPetSick = false;
        currentSicknessName = "";
        clearPrescriptionCourse();
      }
    }

    medicineGivenCount++;
    medicinegiven = true;

        } else if (snackIndex != -1) {
          if (snackQuantities[snackIndex] == 1) {
            snackQuantities[snackIndex] = 0;
            removeInventorySlot(selectedInventorySlot);
          } else {
            snackQuantities[snackIndex] = snackQuantities[snackIndex] - 1;
          }
          timesFedPet++;
          medicinegiven = false;

        } else if (meatIndex != -1) {
          if (meatQuantities[meatIndex] == 1) {
            meatQuantities[meatIndex] = 0;
            removeInventorySlot(selectedInventorySlot);
          } else {
            meatQuantities[meatIndex] = meatQuantities[meatIndex] - 1;
          }
          timesFedPet++;
          medicinegiven = false;

        } else {
          inventorySlots[selectedInventorySlot] = "EMPTY";
          timesFedPet++;
          medicinegiven = false;
        }
      }
    }
  }

  // ========================
  // Inventory: Sell Selected Item
  // "SELL" button — returns 75% of the item's purchase price and removes it from inventory.
  // Blocked until the initial tutorial steak has been fed (hasFedSteak guard).
  // ========================
```

```
  if (mouseX >= 780 && mouseX <= 954.875f &&
      mouseY >= 474.875f && mouseY <= 546 && isInventoryVisible) {

    if (!hasFedSteak) {
      cantsell();
      isShowingCantSell = true;

    } else {
      boolean isMedicine = false;
      boolean isSnack = false;
      boolean isMeat = false;

      isMedicine = findItemIndex(inventorySlots[selectedInventorySlot], medicineItemList) != -1;
      isSnack    = findItemIndex(inventorySlots[selectedInventorySlot], snackItemList)    != -1;
      isMeat     = findItemIndex(inventorySlots[selectedInventorySlot], meatItemList)     != -1;

      if (isMedicine) {
        int medIndex = findItemIndex(inventorySlots[selectedInventorySlot], medicineItemList);
        if (medIndex != -1) {
          float sellPrice = (1.0f / medicineDefaultQuantities[medIndex]) * 3.75f;
          logSellTransaction(medicineItemList[medIndex], sellPrice);
          medicineQuantities[medIndex] -= 1;
          if (medicineQuantities[medIndex] <= 0) {
            medicineQuantities[medIndex] = 0;
            removeInventorySlot(selectedInventorySlot);
          }
        }

      } else if (isSnack) {
        int snackIndex = findItemIndex(inventorySlots[selectedInventorySlot], snackItemList);
        if (snackIndex != -1) {
          float sellPrice = snackPrices[snackIndex] * 0.75f;
          logSellTransaction(snackItemList[snackIndex], sellPrice);
          snackQuantities[snackIndex] -= 1;
          if (snackQuantities[snackIndex] <= 0) {
            snackQuantities[snackIndex] = 0;
            removeInventorySlot(selectedInventorySlot);
          }
        }

      } else if (isMeat) {
        int meatIndex = findItemIndex(inventorySlots[selectedInventorySlot], meatItemList);
        if (meatIndex != -1) {
          float sellPrice = meatPrices[meatIndex] * 0.75f;
          logSellTransaction(meatItemList[meatIndex], sellPrice);
          meatQuantities[meatIndex] -= 1;
          if (meatQuantities[meatIndex] <= 0) {
            meatQuantities[meatIndex] = 0;
            removeInventorySlot(selectedInventorySlot);
          }
        }

      } else {
        removeInventorySlot(selectedInventorySlot);
      }
    }
  }

  // ========================
  // Minigame Choice Screen: Select Which Game to Play
  // Three clickable panel regions on the choice screen.
  // setMinigameEntry() ensures only one flag is true at a time.
  // ========================
  if (wasOnChoiceScreen && isFadingOut && !isOnMainScreen) {
    if (mouseX > 88 && mouseX < 373 && mouseY > 258 && mouseY < 658) {
```

```
      setMinigameEntry(true, false, false);   // Swamp Hop
    } else if (mouseX > 403 && mouseX < 695 && mouseY > 257 && mouseY < 664) {
      setMinigameEntry(false, true, false);   // Snack Snatch
    } else if (mouseX > 731 && mouseY > 263 && mouseX < 1013 && mouseY < 661) {
      setMinigameEntry(false, false, true);   // Fetch Frenzy
    }
  }

  // ========================
  // Minigame: "PLAY" / "RETRY" Button (left modal button)
  // Shared hit region used by all three games when the modal is showing
  // ========================
  if (mouseX >= 427 &&
      mouseX <= 533 &&
      mouseY >= 410 &&
      mouseY <= 463) {
    if (isSnatchLost && isEnterSnackSnatch) {
      isSnatchFirstPlay = false;
      resetSnackSnatch();
    }

    if (isSwampHopLost && isEnterSwampHop) {
      isSwampHopFirstPlay = false;
      resetSwampHop();
    }

    if (isFetchLost && isEnterFetchFrenzy) {
      isFetchFirstPlay = false;
      resetFetchFrenzy();
    }
  }

  // ========================
  // Minigame: "EXIT" Button (right modal button)
  // Returns to the main screen and logs any earnings from the session to the bank
  // ========================
  if (mouseX >= width/2 + 70 - 52.5f &&
      mouseX <= width/2 + 70 + 52.5f &&
      mouseY >= 436 - 25 &&
      mouseY <= 436 + 25 &&
      !wasOnChoiceScreen &&
      ((wasEnteringSwamp && isSwampHopLost) ||
       (wasEnteringSnack && isSnatchLost) ||
       (wasEnteringFetch && isFetchLost))) {
    isExitingMinigame = true;
    isOnMainScreen = true;
    isPlayClicked = false;
    isFadingOut = false;
    mainFade.setClear();   // resume on main screen without a fade-in
    isEnterSwampHop = false;
    isEnterFetchFrenzy = false;
    isEnterSnackSnatch = false;
    isSwampHopFirstPlay = true;
    isFetchFirstPlay = true;
    isSnatchFirstPlay = true;
    isExitingMinigame = true;
    isOnMainScreen = true;
    isPlayClicked = false;
    if (currentPlaySessionMoneyEarned > 0) {
      bankTransactionsLoggedCount++;
      bankTransactionLog.add("Transaction: Minigames (+$" + nf(currentPlaySessionMoneyEarned, 0, 2) + ")");
      currentPlaySessionMoneyEarned = 0;
    }
  }

```

```
  // ========================
  // Quit Button (gear icon, top-right corner of main screen)
  // Opens a confirmation dialog before quitting
  // ========================
  if (dist(mouseX, mouseY, 1044, 43) <= 36 &&
      !isBankOpen  && !isInventoryVisible && !isServicesOpen && !isStoreOpen &&
      !isShowingCantSell && !isShowingPlayPopup && !isShowingEarnPopup && !isWelcomePopupVisible && !isShowingJobPopup &&
      !isShowingFirstHelpPopup && !isShowingTreatmentPopup && !isShowingRestPopup &&
      !isShowingBankPopup && !isShowingStoreClosedPopup  && !isNextDayPopupOpen && !isEarnPanelOpen  && !isShowingQuitDialog) {
    isShowingQuitDialog=true;
  }

  // ========================
  // Earn Button (top-right earn icon, unlocked after hasShownEarnPopup)
  // Opens the earn screen (jobs + tasks)
  // ========================
  if (dist(mouseX, mouseY, 1045, 134) <= 43 &&
      !isBankOpen && hasShownEarnPopup && !isInventoryVisible && !isServicesOpen && !isStoreOpen &&
      !isShowingCantSell && !isShowingPlayPopup && !isShowingEarnPopup && !isShowingJobPopup && !isWelcomePopupVisible &&
      !isShowingFirstHelpPopup && !isShowingTreatmentPopup && !isShowingRestPopup &&
      !isShowingBankPopup && !isShowingStoreClosedPopup  && !isNextDayPopupOpen && !isShowingMusicSettings && !isShowingQuitDialog) {
    isEarnPanelOpen = true;
    isJobFinderOpen = false;
    isTasksPanelOpen = false;
    hasOpenedEarnPanel = true;
  }

  // ========================
  // Earn Screen: Navigation and Actions
  // Handles the earn hub, job finder, and tasks/upgrades sub-panels
  // ========================
  if (isEarnPanelOpen == true) {

    // Close earn screen (shared X button)
    if (!isJobFinderOpen && !isTasksPanelOpen &&
        mouseX > 933 && mouseY < 171 && mouseX < 977 && mouseY > 132) {
      isEarnPanelOpen = false;
    }

    // Quit current job — resets salary and upgrade cost
    if (!isJobFinderOpen && !isTasksPanelOpen &&
        mouseX > 766 && mouseX < 866 && mouseY > 156 && mouseY < 256) {
      job = "unemployed";
      salary = 0;
      hasCashierMaxedSalary = false;
      salaryUpgradeCost = 3;
    }

    // Navigate to Job Finder sub-panel
    if (!isJobFinderOpen && !isTasksPanelOpen &&
        mouseX > 205 && mouseX < 455 && mouseY > 410 && mouseY < 500) {
      isJobFinderOpen = true;
      isTasksPanelOpen = false;
      return;
    }

    // Navigate to Tasks & Upgrades sub-panel
    if (!isJobFinderOpen && !isTasksPanelOpen &&
        mouseX > 610 && mouseX < 930 && mouseY > 410 && mouseY < 500) {
      isTasksPanelOpen = true;
      isJobFinderOpen = false;
```

```
      hasClickedTaskTab=true;
      return;
    }

    // Back button inside Job Finder
    if (isJobFinderOpen &&
        mouseX > 863 && mouseX < 906 && mouseY > 155 && mouseY < 194.5f) {
      isJobFinderOpen = false;
    }

    // Back button inside Tasks screen
    if (isTasksPanelOpen &&
        mouseX > 833 && mouseX < 876 && mouseY > 155 && mouseY < 194.5f) {
      isTasksPanelOpen = false;
    }

    // Apply for cashier job (always available, starting salary $15)
    if (isJobFinderOpen && job.equals("unemployed") &&
        mouseX > 260 && mouseX < 339 && mouseY > 478 && mouseY < 498) {
      job = "cashier";
      salary = 15;
    }

    // Apply for barista job (unlocks on day 10, starting salary $35)
    if (isJobFinderOpen && job.equals("unemployed") && day >= 10 &&
        mouseX > 510 && mouseX < 590 && mouseY > 478 && mouseY < 498) {
      job = "barista";
      salary = 35;
    }

    // Apply for manager job (unlocks on day 25, starting salary $75)
    if (isJobFinderOpen && job.equals("unemployed") && day >= 25 &&
        mouseX > 760 && mouseX < 840 && mouseY > 478 && mouseY < 498) {
      job = "manager";
      salary = 75;
    }

    // "Help Around Town" task button (tasks tab)
    // Earns task money, triggers currentSicknessName risk, and logs to bank
if (isTasksPanelOpen && hasShownJobPopup &&
    mouseX > 340 && mouseX < 420 && mouseY > 420 && mouseY < 441) {
  isHelpTaskPending = true;
  hasUsedHelpTask = true;
  helpTaskCount++;
  money += taskRewardAmount;
  moneyEarnedFromTasks += taskRewardAmount;
  totalMoneyEarned += taskRewardAmount;
  bankTransactionsLoggedCount++;
  bankTransactionLog.add("Transaction: Help Around Town (+$" + nf(taskRewardAmount,0,2) + ")");
  isEarnPanelOpen = false;
  isTasksPanelOpen = false;
  isJobFinderOpen = false;

  if (!hasShownFirstHelpPopup) {
    isPetSick = true;
    currentSicknessName = sicknessNames[0];
    helpPopupMessage = "Congrats on earning $" + nf(taskRewardAmount,0,2) + "! While helping around town, "
+ alligator.petName + " was left unattended and developed an infection. Close this window and click the services
button to visit the vet.";
    isShowingFirstHelpPopup = true;
  } else if (random(1) < 0.40f) {

      if (!isPetSick) {
        int randomSicknessIndex = int(random(sicknessNames.length));
        isPetSick = true;
```

```
        currentSicknessName = sicknessNames[randomSicknessIndex];

        helpPopupMessage = "Uh oh! While you were off helping, " + alligator.petName + " was left unattended and I have bad news: " + currentSicknessName + ".";
      }
      else {
        alligator.health -= 10;

        helpPopupMessage = "While you were helping around town, your already sick alligator  became weaker and lost 10 health.";
      }

      isShowingFirstHelpPopup = true;
    }
}

    // Salary upgrade — increases salary by 12% each purchase, capped per job tier
    if (isTasksPanelOpen && !job.equals("unemployed") && money >= salaryUpgradeCost &&
        mouseX > 680 && mouseX < 760 && mouseY > 410 && mouseY < 430 &&
        !(job.equals("cashier") && salary >= 32) &&
        !(job.equals("barista") && salary >= 70)) {

      float cost = salaryUpgradeCost;
      money -= cost;
      totalMoneySpent += cost;
      bankTransactionsLoggedCount++;
      bankTransactionLog.add("Transaction: Salary Upgrade (-$" + nf(cost,0,2) + ")");

      if (job.equals("cashier") && salary * 1.12f >= 32) {
        salary = 32;
        hasCashierMaxedSalary = true;

      } else if (job.equals("barista") && salary * 1.12f >= 70) {
        salary = 70;

      } else {
        salary = salary * 1.12f;
      }

      salaryUpgradeCost = salaryUpgradeCost * 1.4f;
      salaryUpgradeCount++;
    }

    // Task reward upgrade — increases the "Help Around Town" payout by 12% each purchase
    if (isTasksPanelOpen && money >= taskUpgradeCost &&
        mouseX > 775 && mouseX < 855 && mouseY > 410 && mouseY < 430) {
      float cost = taskUpgradeCost;
      money -= cost;
      totalMoneySpent += cost;
      bankTransactionsLoggedCount++;
      bankTransactionLog.add("Transaction: Task Upgrade (-$" + nf(cost,0,2) + ")");
      taskUpgradeCost = taskUpgradeCost * 1.3f;
      taskRewardAmount = taskRewardAmount * 1.12f;
      taskUpgradeCount++;
    }
  }

  // $ Per Point upgrade — increases minigame money earned per score point
  if (isTasksPanelOpen &&
      mouseX > 575 && mouseX < 656 &&
      mouseY > 411 && mouseY < 431 && money>=pointUpgradeCost) {

      money -= pointUpgradeCost;
      totalMoneySpent += pointUpgradeCost;
      bankTransactionsLoggedCount++;
```

```
      bankTransactionLog.add("Transaction: Bought $ Per Point Upgrade (-$" + nf(pointUpgradeCost, 0, 2) + ")");
      playPointUpgradeCount++;

      if (moneyPerMinigamePoint == 0) {
        moneyPerMinigamePoint = 0.10f;
      } else {
        moneyPerMinigamePoint *= 1.2f;
      }

      pointUpgradeCost *= 1.2f;
  }

  if (isEarnPanelOpen == true && mouseX > 933 && mouseY < 171 && mouseX < 977 && mouseY > 132) {
    isEarnPanelOpen = false;
  }

if (isEarnPanelOpen == true && hasShownJobPopup &&
    mouseX > 804 && mouseX < 884 && mouseY > 388 && mouseY < 408) {
  isHelpTaskPending = true;
  helpTaskCount++;
  money += taskRewardAmount;
  moneyEarnedFromTasks += taskRewardAmount;
  totalMoneyEarned += taskRewardAmount;
  bankTransactionsLoggedCount++;
  bankTransactionLog.add("Transaction: Help Around Town (+$" + nf(taskRewardAmount, 0, 2) + ")");
  isEarnPanelOpen = false;

  if (!hasShownFirstHelpPopup) {
    isPetSick = true;
    currentSicknessName = sicknessNames[0];
    alligator.health -= 30;
    helpPopupMessage = "Congrats on earning $" + nf(taskRewardAmount,0,2) + "! While helping around town, " + alligator.petName + " was left unattended and developed an infection. Close this window and click the services button to visit the vet.";
    isShowingFirstHelpPopup = true;
  } else if (!isPetSick && random(1) < 0.15f) {
    int randomSicknessIndex = int(random(sicknessNames.length));
    isPetSick = true;
    currentSicknessName = sicknessNames[randomSicknessIndex];
    alligator.health -= 30;
    helpPopupMessage = "Uh oh! While you were off helping, your " + alligator.petName + " was left unattended and developed " + currentSicknessName + ".";
    isShowingFirstHelpPopup = true;
  }
}

  // ========================
  // Services Button (bottom action circle)
  // Unlocked after the first illness popup — opens vet / walker / cleaner panel
  // ========================
  if (dist(mouseX, mouseY, 760, 602) < 50 &&
      !isBankOpen && hasShownFirstHelpPopup == true && !isInventoryVisible && !isStoreOpen &&
      !isEarnPanelOpen && !isShowingCantSell && !isShowingPlayPopup && !isShowingEarnPopup && !isWelcomePopupVisible &&
      !isShowingJobPopup && !isShowingFirstHelpPopup && !isShowingTreatmentPopup &&
      !isShowingBankPopup && !isShowingStoreClosedPopup  && !isNextDayPopupOpen && !isShowingMusicSettings && !isShowingQuitDialog) {
    isServicesOpen = true;
    hasOpenedServices = true;
  }

  // ========================
  // Vet Button inside Services Panel
  // Transitions from the services overview to the vet quality selection screen
```

```
  // ========================
  if (isServicesOpen &&
      mouseX > 256.67f - 80 && mouseX < 256.67f + 80 &&
      mouseY > 480 && mouseY < 560) {
    isVetOpen = true;
    isServicesOpen = false;
  }

  // ========================
  // Vet Care: 3-Star (Low Quality) — $5
  // 25% chance of failing on sick pets (first use always succeeds).
  // If isPetSick and untreated, prescribes a medicine course the player must fulfill.
  // ========================
if (money >= 5 && isVetOpen &&
    mouseX > 380 && mouseX < 500 &&
    mouseY > 405 && mouseY < 465) {

  hasUsedLowQualityVet = true;
  money -= 5;
  totalMoneySpent += 5;
  bankTransactionsLoggedCount++;
  bankTransactionLog.add("Transaction: 3 Star Vet (-$5.00)");
  isVetOpen = false;
  timesUsedVetCare++;
  lowQualityCareCount++;

  boolean alreadyPrescribed = false;
  for (int _pi = 0; _pi < medicineIsPrescribed.length; _pi++) { if (medicineIsPrescribed[_pi]) { alreadyPrescribed = true; break; } }

  if (!isPetSick || alreadyPrescribed) {
    alligator.health = min(100, alligator.health + 30);
    vetTreatmentMessage = "The vet treated " + alligator.petName + "'s health and restored 30 health.";
    isShowingTreatmentPopup = true;
  } else {
    boolean lowQualityCareFails = false;

    if (isFirstLowQualityVetAttempt) {
      isFirstLowQualityVetAttempt = false;
      lowQualityCareFails = false;
    } else if (random(1) < 0.25f) {
      lowQualityCareFails = true;
    }

    if (lowQualityCareFails) {
      isShowingVetFailedPopup = true;
      vetTreatmentMessage = "Sorry, the vet wasn't able to help. A high quality doctor will provide care 100% of the time.";
      isShowingTreatmentPopup = true;
    } else {
      isShowingVetFailedPopup = false;

      for (int i = 0; i < sicknessNames.length; i++) {
        if (currentSicknessName.equals(sicknessNames[i])) {
          startPrescriptionCourse(i);

          vetTreatmentMessage = "The vet has prescribed " + alligator.petName + " " + medicineItemList[i] + ". Head to the store after closing this window to buy it free of charge!";
          isShowingTreatmentPopup = true;
          break;
        }
      }
    }
  }
}
```

```
878 ■
879 ■   if (isVetOpen && mouseX > 711 && mouseY > 231 && mouseX < 758 && mouseY < 275) {
880 ■     isVetOpen = false;
881 ■   }
882 ■
883 ■   // ========================
884 ■   // Store Button (bottom action circle)
885 ■   // Unlocked after the treatment popup is shown. Blocked on day % 7 == 0 (store closed).
886 ■   // ========================
887 ■   if (dist(mouseX, mouseY, 337, 602) < 50 &&
888 ■       !isBankOpen && hasShownTreatmentPopup && !isStoreOpen && !isStoreMainScreenFading &&
889 ■       !isShowingCantSell && !isShowingPlayPopup && !isShowingEarnPopup && !isShowingJobPopup && !
isWelcomePopupVisible &&
890 ■       !isShowingFirstHelpPopup && !isShowingTreatmentPopup && !isShowingBankPopup && (day%7!=0) && !
isShowingStoreClosedPopup  && !isNextDayPopupOpen  && !isShowingQuitDialog) {
891 ■     isStoreMainScreenFading = true;  // begin fade-to-black before entering the store
892 ■     return;
893 ■   } else if (dist(mouseX, mouseY, 337, 602) < 50 &&
894 ■       !isBankOpen && hasShownTreatmentPopup && !isStoreOpen &&
895 ■       !isShowingCantSell && !isShowingPlayPopup && !isShowingEarnPopup && !isShowingJobPopup && !
isWelcomePopupVisible &&
896 ■       !isShowingFirstHelpPopup && !isShowingTreatmentPopup && !isShowingBankPopup && !
isShowingStoreClosedPopup  && !isNextDayPopupOpen  && !isShowingQuitDialog) {
897 ■     // Store is closed today — show a message instead of opening the store
898 ■     isShowingStoreClosedPopup=true;
899 ■   }
900 ■
901 ■   // ========================
902 ■   // Store Tab Navigation
903 ■   // Three tabs at the top of the store: Snacks, Medicine, Meat
904 ■   // ========================
905 ■   if (mouseX > 627 && mouseX < 1070 && mouseY > 39 && mouseY < 88 && isStoreOpen && !isViewingSnacksTab && !
isViewingMeatTab) {
906 ■     isViewingMedicineTab = true;
907 ■   }
908 ■
909 ■   if (mouseX > 28 && mouseX < 485 && mouseY > 33 && mouseY < 88 && isStoreOpen && !isViewingMeatTab && !
isViewingMedicineTab) {
910 ■     isViewingSnacksTab = true;
911 ■   }
912 ■
913 ■   // ========================
914 ■   // Store: Purchase Medicine
915 ■   // Prescribed medicines are free (medicineIsPrescribed[] flag set by vet); non-prescribed cost $5 each.
916 ■   // Adds to inventory slot matching the item name (or any EMPTY slot).
917 ■   // ========================
918 ■   if (isViewingMedicineTab == true &&
919 ■       mouseX > 682.5625f && mouseX < 857.4375f &&
920 ■       mouseY > 474.875f && mouseY < 546 &&
921 ■       selectedInventorySlot != -1 &&
922 ■       inventoryHasRoomFor(medicineItemList[selectedInventorySlot]) &&
923 ■       ((!medicineIsPrescribed[selectedInventorySlot] && money >= 5) ||
medicineIsPrescribed[selectedInventorySlot]))) {
924 ■
925 ■     int medIndex = selectedInventorySlot;
926 ■     boolean boughtMed = false;
927 ■
928 ■     for (int slot = 0; slot < 12; slot++) {
929 ■       if (inventorySlots[slot].equals("EMPTY") || inventorySlots[slot].equals(medicineItemList[medIndex])) {
930 ■
931 ■         inventorySlots[slot] = medicineItemList[medIndex];
932 ■
933 ■         if (medicineItemList[medIndex].equals("Enrofloxacin")) {
934 ■           hasFirstBoughtMedicine = true;
935 ■         }
```

```
936 ■
937 ■         itemsBoughtCount++;
938 ■         timesBoughtMedicine++;
939 ■
940 ■         medicineQuantities[medIndex] += medicineDefaultQuantities[medIndex];
941 ■         boughtMed = true;
942 ■         break;
943 ■       }
944 ■     }
945 ■
946 ■     if (boughtMed && medicineIsPrescribed[medIndex] == false) {
947 ■       money -= 5;
948 ■       totalMoneySpent += 5;
949 ■       bankTransactionsLoggedCount++;
950 ■       bankTransactionLog.add("Transaction: Bought " + medicineItemList[medIndex] + " (-$5.00)");
951 ■     } else if (boughtMed && medicineIsPrescribed[medIndex] == true) {
952 ■       bankTransactionsLoggedCount++;
953 ■       bankTransactionLog.add("Transaction: Bought " + medicineItemList[medIndex] + " (-$0.00)");
954 ■     }
955 ■
956 ■   if (boughtMed) {
957 ■     isViewingMedicineTab = false;
958 ■     medicineIsPrescribed[medIndex] = false;
959 ■   }
960 ■   }
961 ■
962 ■   // ========================
963 ■   // Store: Purchase Snack
964 ■   // ========================
965 ■   if (isViewingSnacksTab == true &&
966 ■       mouseX > 682.5625f && mouseX < 857.4375f &&
967 ■       mouseY > 474.875f && mouseY < 546 &&
968 ■       selectedInventorySlot != -1 &&
969 ■       inventoryHasRoomFor(snackItemList[selectedInventorySlot]) &&
970 ■       money >= snackPrices[selectedInventorySlot]) {
971 ■
972 ■     int snackIndex = selectedInventorySlot;
973 ■     boolean boughtSnack = false;
974 ■
975 ■     for (int slot = 0; slot < 12; slot++) {
976 ■       if (inventorySlots[slot].equals("EMPTY") || inventorySlots[slot].equals(snackItemList[snackIndex])) {
977 ■
978 ■         inventorySlots[slot] = snackItemList[snackIndex];
979 ■
980 ■         itemsBoughtCount++;
981 ■         snackQuantities[snackIndex] += 1;
982 ■
983 ■         money -= snackPrices[snackIndex];
984 ■         totalMoneySpent += snackPrices[snackIndex];
985 ■         bankTransactionsLoggedCount++;
986 ■         bankTransactionLog.add("Transaction: Bought " + snackItemList[snackIndex] + " (-$" +
nf(snackPrices[snackIndex], 0, 2) + ")");
987 ■         boughtSnack = true;
988 ■         break;
989 ■       }
990 ■     }
991 ■
992 ■     if (boughtSnack) {
993 ■       isViewingSnacksTab = false;
994 ■     }
995 ■   }
996 ■
997 ■   // ========================
998 ■   // Store: Purchase Meat
999 ■   // ========================
```

```
  if (isViewingMeatTab == true &&
      mouseX > 682.5625f && mouseX < 857.4375f &&
      mouseY > 474.875f && mouseY < 546 &&
      selectedInventorySlot != -1 &&
      inventoryHasRoomFor(meatItemList[selectedInventorySlot]) &&
      money >= meatPrices[selectedInventorySlot]) {

    int meatIndex = selectedInventorySlot;
    boolean boughtMeat = false;

    for (int slot = 0; slot < 12; slot++) {
      if (inventorySlots[slot].equals("EMPTY") || inventorySlots[slot].equals(meatItemList[meatIndex])) {

        inventorySlots[slot] = meatItemList[meatIndex];

        itemsBoughtCount++;
        meatQuantities[meatIndex] += 1;

        money -= meatPrices[meatIndex];
        totalMoneySpent += meatPrices[meatIndex];
        bankTransactionsLoggedCount++;
        bankTransactionLog.add("Transaction: Bought " + meatItemList[meatIndex] + " (-$" +
nf(meatPrices[meatIndex], 0, 2) + ")");
        boughtMeat = true;
        break;
      }
    }

    if (boughtMeat) {
      isViewingMeatTab = false;
    }
  }

  if (isViewingMedicineTab && mouseX > 933 && mouseY < 171 && mouseX < 977 && mouseY > 132) {
    isViewingMedicineTab = false;
  }

  if (isViewingSnacksTab && mouseX > 933 && mouseY < 171 && mouseX < 977 && mouseY > 132) {
    isViewingSnacksTab = false;
  }

  if (isViewingMeatTab && mouseX > 933 && mouseY < 171 && mouseX < 977 && mouseY > 132) {
    isViewingMeatTab = false;
  }

  // Trigger fade-out instead of closing immediately; store() closes itself when black
  if (mouseX > 1000 && mouseX < 1100 && mouseY > 631 && mouseY < 684 && isStoreOpen && !
isViewingMedicineTab && !isViewingSnacksTab && !isViewingMeatTab && !isStoreFadingOut) {
    isStoreFadingOut = true;
    hasClickedStoreExit = true;
  }

  // =========================
  // Vet Care: 5-Star (High Quality) — $20
  // Always succeeds; prescribes medicine or restores 30 health if already treated
  // =========================
  if (money >= 20 && mouseX > 599 && mouseY > 418 && mouseX < 720 && mouseY < 452 && isVetOpen) {
    hasNeverBoughtHighQualityCare = false;
    money -= 20;
    totalMoneySpent += 20;
    bankTransactionsLoggedCount++;
    bankTransactionLog.add("Transaction: 5 Star Vet (-$20.00)");
    timesUsedVetCare++;
    highQualityCareCount++;
    isVetOpen = false;
```

```
1063 ■
1064 ■     boolean alreadyPrescribed = false;
1065 ■     for (int _pi = 0; _pi < medicineIsPrescribed.length; _pi++) { if (medicineIsPrescribed[_pi]) {
alreadyPrescribed = true; break; } }
1066 ■
1067 ■     if (!isPetSick || alreadyPrescribed) {
1068 ■       alligator.health = min(100, alligator.health + 30);
1069 ■       vetTreatmentMessage = "The vet treated " + alligator.petName + "'s health and restored 30 health.";
1070 ■       isShowingTreatmentPopup = true;
1071 ■     } else {
1072 ■       for (int i = 0; i < sicknessNames.length; i++) {
1073 ■         if (currentSicknessName.equals(sicknessNames[i])) {
1074 ■           startPrescriptionCourse(i);
1075 ■
1076 ■           vetTreatmentMessage = "The vet has prescribed " + alligator.petName + " " + medicineItemList[i] + "
. Head to the store after closing this window to buy it free of charge!";
1077 ■           isShowingTreatmentPopup = true;
1078 ■           break;
1079 ■         }
1080 ■       }
1081 ■     }
1082 ■   }
1083 ■
1084 ■   // =========================
1085 ■   // Bank Screen: Scrollbar Drag Start
1086 ■   // Separate from the main-screen scrollbar check above; handles when isBankOpen is active
1087 ■   // =========================
1088 ■   if (isBankOpen) {
1089 ■     float bankMaxScroll2 = max(0, bankScrollContentHeight - bankViewportHeight);
1090 ■
1091 ■     if (bankScrollContentHeight > bankViewportHeight) {
1092 ■       float bankThumbH2 = max(40, (bankViewportHeight / bankScrollContentHeight) * bankScrollbarHeight);
1093 ■       float bankThumbY2 = map(bankScrollOffset, 0, bankMaxScroll2, bankScrollbarY, bankScrollbarY +
bankScrollbarHeight - bankThumbH2);
1094 ■
1095 ■       if (mouseX >= bankScrollbarX && mouseX <= bankScrollbarX + bankScrollbarWidth &&
1096 ■           mouseY >= bankThumbY2 && mouseY <= bankThumbY2 + bankThumbH2) {
1097 ■         isDraggingBankScrollbar = true;
1098 ■         bankScrollThumbOffsetY = mouseY - bankThumbY2;
1099 ■       }
1100 ■     }
1101 ■   }
1102 ■
1103 ■   if (mouseX > 733 && mouseY > 132 && mouseX < 776 && mouseY < 171.5f && isBankOpen) {
1104 ■     isBankOpen = false;
1105 ■     if (!hasViewedBankFirstTime) {
1106 ■       alligator.energy = 15;
1107 ■       hasViewedBankFirstTime = true;
1108 ■     }
1109 ■   }
1110 ■
1111 ■   // =========================
1112 ■   // Rest Screen: Stop Bar Action
1113 ■   // Player stops an oscillating marker; position determines energy gain or loss.
1114 ■   // Perfect zone: +30 energy. Near zone: +10. Miss: -10.
1115 ■   // =========================
1116 ■   if (isRestOpen && restAttemptsRemaining > 0 &&
1117 ■       mouseX > 398 && mouseX < 479 && mouseY > 511 && mouseY < 530) {
1118 ■     restAttemptsRemaining--;
1119 ■     isRestOpen = false;
1120 ■     hasAlligatorRestedOnce = true;
1121 ■     if (restMarkerX > 425 && restMarkerX < 456) {
1122 ■       alligator.energy += 30;
1123 ■       totalEnergyRestoredFromResting += 30;
1124 ■       timesRestedSuccessfully++;
```

```
    } else if ((restMarkerX > 383 && restMarkerX <= 425) || (restMarkerX < 513 && restMarkerX >= 456)) {
      alligator.energy += 10;
      totalEnergyRestoredFromResting += 10;
    } else {
      alligator.energy -= 10;
    }
    restAttempts++;
  }

  if (isRestOpen && mouseX > 567 && mouseX < 583 && mouseY > 454 && mouseY < 470) {
    isRestOpen = false;
  }

  // ========================
  // Achievements Button (top-right, star icon)
  // Opens the scrollable achievement progress and reward claim panel
  // ========================
  if (dist(mouseX, mouseY, 1045, 232) <= 43 &&
      !isShowingRestPopup && !isRestOpen && isWelcomePopupVisible == false &&
      !isServicesOpen && !isEarnPanelOpen && !isBankOpen && isOnMainScreen &&
      !isShowingCantSell && !isShowingPlayPopup && !isShowingEarnPopup && !isShowingJobPopup &&
      !isShowingFirstHelpPopup && !isShowingTreatmentPopup && !isShowingBankPopup && !isWelcomePopupVisible &&
      !isInventoryVisible && !isStoreOpen && !isShowingStoreClosedPopup  && !isNextDayPopupOpen  && !isShowingQuitDialog) {
    isAchievementsOpen = true;
    hasOpenedAchievements = true;
  }

  // ========================
  // Next Day Button (center bottom circle)
  // Advances the in-game day, applies salary, and decays pet stats
  // ========================
  if (dist(mouseX, mouseY, width/2, 602) <= 50  &&
      !isShowingRestPopup && !isRestOpen && isWelcomePopupVisible == false &&
      !isServicesOpen && !isEarnPanelOpen && !isAchievementsOpen && !isBankOpen && isOnMainScreen &&
      !isShowingCantSell && !isShowingPlayPopup && !isShowingEarnPopup && !isShowingJobPopup &&
      !isShowingFirstHelpPopup && !isShowingTreatmentPopup && !isShowingBankPopup &&
      !isInventoryVisible && !isStoreOpen && !isShowingStoreClosedPopup  && !isNextDayPopupOpen  && !isShowingQuitDialog) {
    isNextDayPopupOpen=true;
    hasAdvancedDay=true;
    isDayEdited=false;
  }

  if (isAchievementsOpen) {
    updateAchievementProgress();

    float achvMaxScroll = max(0, achievementScrollContentHeight - achievementViewportHeight);

    if (mouseX > 733 && mouseY > 134 && mouseX < 776 && mouseY < 172) {
      isAchievementsOpen = false;
      hasClosedAchievements = true;
    }

    if (achievementScrollContentHeight > achievementViewportHeight) {
      float achvThumbH = max(40, (achievementViewportHeight / achievementScrollContentHeight) * achievementScrollbarHeight);
      float achvThumbY = map(achievementScrollOffset, 0, achvMaxScroll, achievementScrollbarY, achievementScrollbarY + achievementScrollbarHeight - achvThumbH);

      if (mouseX >= achievementScrollbarX && mouseX <= achievementScrollbarX + achievementScrollbarWidth &&
          mouseY >= achvThumbY && mouseY <= achvThumbY + achvThumbH) {
        isDraggingAchievementScrollbar = true;
        achievementScrollThumbOffsetY = mouseY - achvThumbY;
```

```
      }
    }

    float boxX = achievementViewportX + 10;
    float boxW = achievementViewportWidth - 30;

    for (int row = 0; row < 30; row++) {
      int i = achievementDrawOrder[row];
      float y = achievementViewportY + 20 + row * achievementItemLineHeight - achievementScrollOffset;

      float buttonCX = boxX + boxW - 52;
      float buttonCY = y + 38;
      float buttonW = 80;
      float buttonH = 20;

      if (mouseX >= buttonCX - buttonW/2 && mouseX <= buttonCX + buttonW/2 &&
          mouseY >= buttonCY - buttonH/2 && mouseY <= buttonCY + buttonH/2 &&
          mouseX >= achievementViewportX && mouseX <= achievementViewportX + achievementViewportWidth &&
          mouseY >= achievementViewportY && mouseY <= achievementViewportY + achievementViewportHeight) {
        if (i >= 0 && i < 30 && isAchievementCollectable[i]) {
          money += achievementRewards[i];
          totalMoneyEarned += achievementRewards[i];
          bankTransactionsLoggedCount++;
          bankTransactionLog.add("Transaction: Achievement Reward (+$" + nf(achievementRewards[i], 0, 2) + "
)");
          achievementTiers[i]++;
          refreshAchievementData();
        }
        break;
      }
    }
  }

  // Store: Meat tab button (at the bottom of the store screen)
  if (isStoreOpen && !isViewingMedicineTab && !isViewingSnacksTab && mouseX>304 && mouseY>591 && mouseX<777
&& mouseY<654) {
    isViewingMeatTab=true;
  }

  // ========================
  // Services: Walker and Cleaner
  // Walker costs $10: drains energy, boosts happiness/health (takes alligator on a walk)
  // Cleaner costs $10: reduces sick risk, boosts happiness/health (cleans the habitat)
  // ========================
  if (isServicesOpen && money>=10 && mouseX>468 && mouseY>499 && mouseX<631 && mouseY<543) {
    isServicesOpen=false;
    alligator.energy-=25;
    alligator.happiness+=15;
    alligator.health+=5;
    walkersHiredCount++;
    money-=10;
    totalMoneySpent+=10;
    bankTransactionsLoggedCount++;
    bankTransactionLog.add("Transaction: Hired Walker (-$10.00)");
    alligator.energy    = clampStat(alligator.energy,    0, 100);
    alligator.happiness = clampStat(alligator.happiness, 0, 100);
    alligator.health    = clampStat(alligator.health,    0, 100);
  }

  if (isServicesOpen && money>=10 && mouseX>762 && mouseY>499 && mouseX<925 && mouseY<544) {
    isServicesOpen=false;
    alligator.sickrisk-=15;
    alligator.happiness+=10;
    alligator.health+=5;
    cleanersHiredCount++;
```

```
    money-=10;
    totalMoneySpent+=10;
    bankTransactionsLoggedCount++;
    bankTransactionLog.add("Transaction: Hired Cleaner (-$10.00)");
    alligator.sickrisk  = clampStat(alligator.sickrisk,  20, 100);
    alligator.happiness = clampStat(alligator.happiness, 0, 100);
    alligator.health    = clampStat(alligator.health,    0, 100);
  }

  // ========================
  // Job Finder: Apply for Locked Jobs
  // Barista unlocks on day 10, Manager on day 25 (progression gates)
  // ========================
  if (job.equals("unemployed") && day >= 10 && isJobFinderOpen &&
      mouseX>510 && mouseX<590 && mouseY>477 && mouseY<497) {
    job = "barista";
    salary = 35;
  }

  if (job.equals("unemployed") && day >= 25 && isJobFinderOpen &&
      mouseX>760 && mouseX<840 && mouseY>477 && mouseY<497) {
    job = "manager";
    salary = 75;
  }

  // Close services panel (X button)
  if (mouseX>931 && mouseX<979 && mouseY>132 && mouseY<174 && isServicesOpen) {
    isServicesOpen=false;
  }
}



// ========================
// Key Press Handler
// Handles minigame controls and developer test keys
// ========================
void keyPressed() {
  // Swamp Hop: SPACE to jump (only when alligator is on the ground)
  if (key == ' ' && isSwampHopOnGround) {
    hopVelocityY = -hopJumpStrength;
    isSwampHopOnGround = false;
  }

  // Snack Snatch: A/D or arrow keys to move left/right
  if ((key == 'a' || key == 'A' || keyCode == LEFT) && isEnterSnackSnatch) {
    isMoveLeft = true;
  }

  if ((key == 'd' || key == 'D' || keyCode == RIGHT) && isEnterSnackSnatch) {
    isMoveRight = true;
  }

  // Fetch Frenzy: WASD or arrow keys to set movement direction
  if ((key == 'a' || key == 'A' || keyCode == LEFT) && isEnterFetchFrenzy) {
    fetchFacingDirection = "LEFT";
  }

  if ((key == 'd' || key == 'D' || keyCode == RIGHT) && isEnterFetchFrenzy) {
    fetchFacingDirection = "RIGHT";
  }

  if ((key == 'w' || key == 'W' || keyCode == UP) && isEnterFetchFrenzy) {
    fetchFacingDirection = "UP";
  }
```

```
  if ((key == 's' || key == 'S' || keyCode == DOWN) && isEnterFetchFrenzy) {
    fetchFacingDirection = "DOWN";
  }

  // --- Developer Test Keys ---
  // These shortcuts exist for rapid testing during development.
  if (key == 'y') money += 1000;           // Add $1000 for testing economy features
  if (key == 'z') alligator.energy -= 50;  // Drain energy to test rest/minigame logic
}

// ========================
// Key Release Handler
// Stops continuous movement in Snack Snatch when A/D or arrow keys are released
// ========================
void keyReleased() {
  if ((key == 'a' || key == 'A' || keyCode == LEFT) && isEnterSnackSnatch) {
    isMoveLeft = false;
  }

  if ((key == 'd' || key == 'D' || keyCode == RIGHT) && isEnterSnackSnatch) {
    isMoveRight = false;
  }
}

// ========================
// Mouse Drag Handler
// Handles dragging the bank and achievements scrollbar thumbs
// ========================
void mouseDragged() {
  if (cp5.isMouseOver()) return;

  if (isDraggingBankScrollbar) {
    float bankmaxScroll = max(0, bankScrollContentHeight - bankViewportHeight);
    if (bankmaxScroll > 0) {
      float bankthumbH = max(40, (bankViewportHeight / bankScrollContentHeight) * bankScrollbarHeight);

      float banknewThumbY = mouseY - bankScrollThumbOffsetY;
      banknewThumbY = constrain(banknewThumbY, bankScrollbarY, bankScrollbarY + bankScrollbarHeight - bankthumbH);

      bankScrollOffset = map(banknewThumbY, bankScrollbarY, bankScrollbarY + bankScrollbarHeight - bankthumbH, 0, bankmaxScroll);
    }
  }

  if (isDraggingAchievementScrollbar) {
    float achvmaxScroll = max(0, achievementScrollContentHeight - achievementViewportHeight);
    if (achvmaxScroll > 0) {
      float achvthumbH = max(40, (achievementViewportHeight / achievementScrollContentHeight) * achievementScrollbarHeight);

      float achvnewThumbY = mouseY - achievementScrollThumbOffsetY;
      achvnewThumbY = constrain(achvnewThumbY, achievementScrollbarY, achievementScrollbarY + achievementScrollbarHeight - achvthumbH);

      achievementScrollOffset = map(achvnewThumbY, achievementScrollbarY, achievementScrollbarY + achievementScrollbarHeight - achvthumbH, 0, achvmaxScroll);
    }
  }
}

// ========================
// Mouse Wheel Handler
// Scrolls the bank transaction list or achievements panel when the cursor is over them
```

```
// ========================
void mouseWheel(MouseEvent event) {
  float e = event.getCount();

  if (mouseX >= bankViewportX && mouseX <= bankViewportX + bankViewportWidth &&
      mouseY >= bankViewportY && mouseY <= bankViewportY + bankViewportHeight && isBankOpen) {
    bankScrollContentHeight = 140 + bankTransactionLog.size() * bankItemLineHeight;
    float bankMaxScroll = max(0, bankScrollContentHeight - bankViewportHeight);

    bankScrollOffset += e * 20;
    bankScrollOffset = constrain(bankScrollOffset, 0, bankMaxScroll);
  }

  if (mouseX >= achievementViewportX && mouseX <= achievementViewportX + achievementViewportWidth &&
      mouseY >= achievementViewportY && mouseY <= achievementViewportY + achievementViewportHeight &&
isAchievementsOpen) {
    achievementScrollContentHeight = 20 + 30 * achievementItemLineHeight;
    float achvMaxScroll = max(0, achievementScrollContentHeight - achievementViewportHeight);

    achievementScrollOffset += e * 20;
    achievementScrollOffset = constrain(achievementScrollOffset, 0, achvMaxScroll);
  }
}

// ========================
// Mouse Release Handler
// Ends any active scrollbar drag when the mouse button is released
// ========================
void mouseReleased() {
  isDraggingBankScrollbar = false;
  isDraggingAchievementScrollbar = false;
}
```

§ 3

C_Music.pde

# Music & Volume Controls

ControlP5 volume slider and music toggle with event-safe callback handling

```
// ========================
// Music / Volume Controls (ControlP5)
// Keeps slider + toggle in sync without triggering recursive events
// ========================

float lastNonZeroVolume = 50;             // Remembers the last non-zero slider value (used when turning music back on)
boolean isSuppressingVolumeCallback = false;    // Flag for suppressing event loops (declared here; use when needed)
ControlP5 cp5;                     // Main ControlP5 instance


// ========================
// Slider Callback: volume(float v)
// Automatically called by ControlP5 when the "volume" slider changes
// ========================
void volume(float v) {

  // Apply audio change immediately
  applyVolume(v);

  // If volume is basically 0, force toggle OFF (but without triggering its callback)
  if (v <= 0.001) {
    setToggleValueNoEvent(0);
  }
  // Otherwise, remember volume and force toggle ON
  else {
    lastNonZeroVolume = v;
    setToggleValueNoEvent(1);
  }
}


// ========================
// Audio Application Helper
// Converts slider percent (0–100) to amp value and updates the music volume
// ========================
void applyVolume(float v) {

  // Map 0–100 slider range to a safe amp range
  float ampValue = map(v, 0, 100, 0.0, 0.25);
  music.amp(ampValue);
}


// ========================
// UI Helper: Set Slider Without Firing Events
// Temporarily disables broadcasting so ControlP5 doesn't call volume() again
// ========================
void setSliderValueNoEvent(float v) {

  Controller c = cp5.getController("volume");
  c.setBroadcast(false);
  c.setValue(v);
  c.setBroadcast(true);
}


```

```
// ========================
// UI Helper: Set Toggle Without Firing Events
// Temporarily disables broadcasting so toggle callbacks don't chain
// ========================
void setToggleValueNoEvent(float v) {

  Controller c = cp5.getController("musicOn");
  c.setBroadcast(false);
  c.setValue(v);
  c.setBroadcast(true);
}
```

§ 4

D_General_Functions.pde

# Utilities, Assets & Save System

fileWork(), Fade class, name validation, saveGame(), loadGame(), JSON helpers

```
// ========================
// Fade
// A reusable screen-transition overlay that fades the display to/from black.
// Each distinct transition owns one Fade instance with its own opacity.
//
// opacity: 0 = fully transparent (screen visible), 255 = fully black
// outComplete: one-way latch — stays true once opacity first reaches 255.
//   Use this to detect "done fading to black" in multi-phase sequences
//   without re-checking opacity (which would flip back as we fade in again).
//
// Usage:
//   Fade myFade = new Fade(255);          // start black (e.g. screen entry)
//   myFade.stepIn(speed);                 // move toward clear each frame
//   myFade.stepOut(speed);                // move toward black each frame
//   myFade.draw();                        // draw the overlay on top of everything
//   myFade.isBlack() / myFade.isClear()  // check completion states
// ========================
class Fade {
  float opacity;
  boolean outComplete;   // latches true the first time opacity reaches 255

  Fade(float startOpacity) {
    opacity = startOpacity;
    outComplete = (startOpacity >= 255);
  }

  // Step opacity toward 255 (to black); sets outComplete when fully black
  boolean stepOut(float speed) {
    opacity = min(255, opacity + speed);
    if (opacity >= 255) outComplete = true;
    return outComplete;
  }

  // Step opacity toward 0 (to clear); returns true when fully transparent
  boolean stepIn(float speed) {
    opacity = max(0, opacity - speed);
    return opacity <= 0;
  }

  // Draw a full-screen black rectangle at the current opacity level.
  // Call after all other draw calls so the overlay sits on top.
  void draw() {
    noStroke();
    fill(0, opacity);
    rectMode(CORNER);
    rect(0, 0, width, height);
  }

  boolean isBlack() { return opacity >= 255; }
  boolean isClear()  { return opacity <= 0;  }
  void setBlack() { opacity = 255; outComplete = true; }
  void setClear()  { opacity = 0; }
}


// ========================
// Naming UI + File Loading / Initialization
// Loads assets (images/fonts/audio) and builds ControlP5 UI (music + naming)
```

```
// ========================

Textfield nameField;         // Text input for naming the pet
Button    confirmBtn;        // Button to confirm name entry
boolean   isNameInputVisible = false; // Tracks whether naming UI should currently be visible


// ========================
// Alligator placeholder name
// ========================

String defaultPetName = "Moss";

// ========================
// Naming / Input-Related Variables
// ========================

String typedInput = "";  // Placeholder string for typed input

PFont fontTimesReference;        // Font reference

// Holds the current validation error message shown on the naming screen
// Empty string means no error (input is valid or not yet submitted)
String petNameValidationError = "";

boolean isNameChosen = false;


// ========================
// Asset Loading + Audio + Fonts
// Called once during setup() to load everything the game needs
// ========================
void fileWork() {
  //Creating the alligator pet
    alligator = new Pet(defaultPetName);

  // --- Images / Screens ---
  homescreen                = loadImage("gametitlescreen.png");
  instructions              = loadImage("instructions.png");
  musicscreen               = loadImage("musicscreen.png");
  outsideofadoptioncenter   = loadImage("outsideofadoptioncenter.png");
  adoptioncenterinterior    = loadImage("adoptioncenterinterior.png");
  pickingcat                = loadImage("pickingcat.png");
  pickingdog                = loadImage("pickingdog.png");
  namingalligatorbackground = loadImage("namingalligatorbackground.png");
  alligator.neutralalligator= loadImage("neutralalligator.png");
  alligator.hungryalligator = loadImage("hungryalligator.png");
  alligator.energeticalligator = loadImage("energeticalligator.png");
  alligator.sickalligator   = loadImage("sickalligator.png");
  mainscreen                = loadImage("mainscreen.png");
  mainscreenbuttons         = loadImage("mainscreenbuttons.png");
  achievementsbutton        = loadImage("achievementsbutton.png");
  settingsbutton            = loadImage("settingsbutton.png");
  earnbutton                = loadImage("earnbutton.png");
  popupbackground           = loadImage("popupbackground.png");
  cloudframe1               = loadImage("cloudframe1.png");
  cloudframe2               = loadImage("cloudframe2.png");
  cloudframe3               = loadImage("cloudframe3.png");
  steak                     = loadImage("steak.png");
  redarrow                  = loadImage("redarrow.png");
  minigamechoice            = loadImage("minigamechoice.png");
  swamphopbackground        = loadImage("swamphopbackground.png");
  alligatorf1               = loadImage("alligatorf1.png");
  alligatorf2               = loadImage("alligatorf2.png");
  alligatorf3               = loadImage("alligatorf3.png");
```

```
alligatorf4               = loadImage("alligatorf4.png");
log                       = loadImage("log.png");
vine                      = loadImage("vine.png");
rock                      = loadImage("rocks.png");
mud                       = loadImage("mud.png");
bluegill                  = loadImage("bluegill.png");
bass                      = loadImage("bass.png");
perch                     = loadImage("perch.png");
goldfish                  = loadImage("goldfish.png");
crab                      = loadImage("crab.png");
lambchop                  = loadImage("lambchop.png");
porkchop                  = loadImage("porkchop.png");
broccoli                  = loadImage("broccoli.png");
carrot                    = loadImage("carrot.png");
tomato                    = loadImage("tomato.png");
pepper                    = loadImage("pepper.png");
fetchfrenzybackground     = loadImage("fetchfrenzybackground.png");
ball                      = loadImage("ball.png");
topalligator1             = loadImage("topalligator1.png");
topalligator2             = loadImage("topalligator2.png");
unemployed                = loadImage("unemployed.png");
cashier                   = loadImage("cashier.png");
barista                   = loadImage("barista.png");
manager                   = loadImage("manager.png");
town                      = loadImage("town.png");
cash                      = loadImage("cash.png");
house                     = loadImage("house.png");
lock                      = loadImage("lock.png");
vet                       = loadImage("vet.png");
walker                    = loadImage("walker.png");
cleaner                   = loadImage("cleaner.png");
storebackground           = loadImage("storebackground.png");
medicine                  = loadImage("medicine.png");
nachos                    = loadImage("nachos.png");
cheesepuffs   = loadImage("cheesepuffs.png");
chips         = loadImage("chips.png");
chocolatebar  = loadImage("chocolatebar.png");
cookies       = loadImage("cookies.png");
crackers      = loadImage("crackers.png");
energydrink   = loadImage("energydrink.png");
granolabar    = loadImage("granolabar.png");
popcorn       = loadImage("popcorn.png");
pretzels      = loadImage("pretzels.png");
soda          = loadImage("soda.png");
trailmix      = loadImage("trailmix.png");
chicken = loadImage("chicken.png");
catfish = loadImage("catfish.png");
frog = loadImage("frog.png");
shrimp = loadImage("shrimp.png");

// Initialize image arrays for store/inventory grid drawing
snackImages = new PImage[]{nachos, cheesepuffs, chips, chocolatebar, cookies, crackers,
                           energydrink, granolabar, popcorn, pretzels, soda, trailmix};
meatImages  = new PImage[]{bluegill, bass, perch, goldfish, crab, lambchop,
                           porkchop, steak, chicken, catfish, frog, shrimp};

// --- Background Music ---
music = new SoundFile(this, "music.mp3");
music.loop();
music.amp(0.2);

// --- Fonts ---
times15 = createFont("timesnewroman.ttf", 15);
times50 = createFont("timesnewroman.ttf", 50);
times30 = createFont("timesnewroman.ttf", 30);
```

```
  arcade  = createFont("arcade.otf", 40);
}


// ========================
// Music Settings UI
// Builds a volume slider + on/off toggle (hidden by default)
// Keeps UI + audio state synchronized via callbacks
// ========================
void musicAdjusters() {

  // Create ControlP5 UI manager (needed for sliders, toggles, textfields, buttons)
  cp5 = new ControlP5(this);

  // Temporarily stop ControlP5 from broadcasting events while building components
  // (prevents callbacks firing during setup)
  cp5.setBroadcast(false);

  // -------------------------
  // Volume Slider ("volume")
  // -------------------------
  cp5.addSlider("volume")
    .setFont(times15)
    .setPosition(width * 0.26, height * 0.53)
    .setSize(300, 28)
    .setRange(0, 100)
    .setValue(50)
    .setLabel("VOLUME")
    .setColorCaptionLabel(color(0))
    .setColorValueLabel(color(0))
    .setColorBackground(color(40))
    .setColorForeground(color(227, 139, 7))
    .setColorActive(color(227, 139, 7))
    .setDecimalPrecision(0)
    .setVisible(false); // hidden until user opens music settings screen

  // Ensure the slider's numeric value label uses the correct font
  ((controlP5.Controller) cp5.getController("volume"))
    .getValueLabel()
    .setFont(times15);

  // -------------------------
  // Music Toggle ("musicOn")
  // -------------------------
  Toggle t = cp5.addToggle("musicOn")
    .setPosition(width * 0.63, height * 0.53)
    .setSize(50, 25)
    .setValue(true)
    .setFont(times15)
    .setLabel("ON / OFF")
    .setColorCaptionLabel(color(0))
    .setColorBackground(color(40))
    .setColorActive(color(227, 139, 7))
    .setColorForeground(color(227, 139, 7))
    .setVisible(false); // hidden until user opens music settings screen

  // Turn broadcasting back on now that UI elements exist
  cp5.setBroadcast(true);

  // -------------------------
  // Toggle Change Callback
  // When user flips ON/OFF, update slider + actual audio safely
  // -------------------------
  t.onChange(new CallbackListener() {
    public void controlEvent(CallbackEvent e) {
```

```java

      // Toggle controller gives 1 (on) or 0 (off)
      float v = e.getController().getValue();
      boolean on = (v == 1);

      if (on) {
        // Restore last non-zero volume when turning back on
        float target = (lastNonZeroVolume > 0 ? lastNonZeroVolume : 0);
        setSliderValueNoEvent(target);
        applyVolume(target);
      } else {
        // Mute when off
        setSliderValueNoEvent(0);
        applyVolume(0);
      }
    }
  });
}


// ========================
// Name Input UI Builder
// Creates a hidden Textfield + Confirm button for naming the pet
// ========================
void nameinput() {

  // ControlP5 font wrapper for text rendering in UI components
  ControlFont cf = new ControlFont(times15, 20);

  // -------------------------
  // Name Textfield ("nameInput")
  // -------------------------
  nameField = cp5.addTextfield("nameInput")
    .setPosition(width * 0.35, 300)
    .setSize(220, 40)
    .setAutoClear(false)  // keep text after pressing enter/clicking away
    .hide()               // hidden until naming screen is shown
    .setLabel("");        // no label text (clean UI)

  // Hide the caption label and style the displayed text
  nameField.getCaptionLabel().setVisible(false);
  nameField.getValueLabel().setFont(cf);

  // Textfield color theme
  nameField.setColorBackground(color(20, 80, 50));
  nameField.setColorForeground(color(140, 255, 170));
  nameField.setColorActive(color(140, 255, 170));
  nameField.setColorValueLabel(color(255));

  // Light padding so the caret/text doesn't start at the border
  nameField.setText("  ");

  // -------------------------
  // Confirm Button ("confirm")
  // ControlP5 will automatically call void confirm() when pressed
  // -------------------------
  confirmBtn = cp5.addButton("confirm")
    .setPosition(width * 0.35 + 230, 300)
    .setSize(110, 40)
    .hide()               // hidden until naming screen is shown
    .setLabel("CONFIRM");

  // Button label styling + colors
  confirmBtn.getCaptionLabel().setFont(cf).setColor(color(255));
  confirmBtn.setColorBackground(color(10, 50, 35));
```

```
  confirmBtn.setColorForeground(color(140, 255, 170));
  confirmBtn.setColorActive(color(30, 100, 70));
}


// ========================
// Input Validation
// Validates the pet name on two levels before accepting it:
//   - Syntactical: checks format rules (non-empty, max length, allowed characters)
//   - Semantic:    checks that the value makes sense as a name (at least one letter)
// Sets petNameValidationError to a human-readable message on failure.
// Returns true if the name is acceptable, false otherwise.
// ========================

boolean isValidName(String raw) {

  // Syntactical check: cannot be null or entirely whitespace
  if (raw == null || raw.trim().length() == 0) {
    petNameValidationError = "Name cannot be empty. Please enter a name.";
    return false;
  }

  String trimmed = raw.trim();

  // Syntactical check: enforce a maximum of 20 characters for display fit
  if (trimmed.length() > 20) {
    petNameValidationError = "Name is too long (max 20 characters).";
    return false;
  }

  // Semantic check: name must include at least one actual letter — not just symbols/spaces
  boolean hasLetter = false;
  for (int i = 0; i < trimmed.length(); i++) {
    if (Character.isLetter(trimmed.charAt(i))) {
      hasLetter = true;
      break;
    }
  }
  if (!hasLetter) {
    petNameValidationError = "Name must contain at least one letter.";
    return false;
  }

  // Syntactical check: only letters, spaces, hyphens, and apostrophes are valid
  // (covers names like "Al" or "Snap-jaw" or "O'Scales" but blocks digits/symbols)
  for (int i = 0; i < trimmed.length(); i++) {
    char c = trimmed.charAt(i);
    if (!Character.isLetter(c) && c != ' ' && c != '-' && c != '\'') {
      petNameValidationError = "Only letters, spaces, hyphens, and apostrophes are allowed.";
      return false;
    }
  }

  // All checks passed — clear any leftover error message
  petNameValidationError = "";
  return true;
}


// ========================
// Naming Confirmation
// Triggered by the ControlP5 button named "confirm"
// Validates the name first; only proceeds if it passes both syntactical and semantic checks.
// ========================

```

```
void confirm() {

  String rawInput = nameField.getText();

  // Validate before accepting — if invalid, show error and block the transition
  if (!isValidName(rawInput)) return;

  // Mark name selection complete and assign the formatted name to the pet
  isNameChosen = true;
  alligator.petName = formatName(rawInput);
}


// ========================
// Name Formatting Helper
// Normalizes user input for display consistency:
// - trims ends
// - collapses multiple spaces
// - converts to lowercase
// - capitalizes first character
// ========================
String formatName(String raw) {

  raw = raw.trim();                   // remove leading/trailing spaces
  raw = raw.replaceAll("\\s+", " ");  // collapse multiple spaces into single spaces
  raw = raw.toLowerCase();            // normalize to lowercase

  // If user entered nothing meaningful, return empty string
  if (raw.length() == 0) return raw;

  // Capitalize first letter only
  return raw.substring(0, 1).toUpperCase() + raw.substring(1);
}


// ========================
// saveGame / loadGame
// Persist and restore the entire game state to/from "save.json" in the sketch's data folder.
// Every global variable that affects gameplay is saved so the player can quit and resume.
// Arrays (inventory, prescriptions, achievements) are serialized via helper functions below.
// Transient UI flags (popup visibility, selected slot, etc.) are reset on load so the
// resume experience starts from a clean visual state even if the player quit mid-popup.
// ========================

void saveGame() {
  JSONObject save = new JSONObject();

  // cutscene / setup progress
  save.setBoolean("homescreenvisible", isHomeScreenVisible);
  save.setBoolean("cutscenestart", isCutsceneActive);
  save.setBoolean("inNaming", isNamingActive);
  save.setBoolean("startrealgame", isGameStarted);
  save.setBoolean("dogadopted", isDogSelected);
  save.setBoolean("catadopted", isCatSelected);
  save.setInt("selectedAlligator", selectedAlligatorSkin);
  save.setBoolean("namechosen", isNameChosen);
  save.setString("alligatorname", defaultPetName == null ? "" : defaultPetName);
  save.setString("userInput", typedInput == null ? "" : typedInput);

  // main progression
  save.setBoolean("onmainscreen", isOnMainScreen);
  save.setInt("day", day);
  save.setFloat("money", money);
  save.setString("job", job == null ? "" : job);
  save.setFloat("salary", salary);
```

```
449    save.setFloat("totalJobEarnings", totalJobEarnings);
450    save.setFloat("taskmoney", taskRewardAmount);
451    save.setFloat("moneyperpt", moneyPerMinigamePoint);
452    save.setFloat("ptupgcost", pointUpgradeCost);
453    save.setFloat("salupgcost", salaryUpgradeCost);
454    save.setFloat("taskupgcost", taskUpgradeCost);
455
456    // pet state
457    save.setString("petName", alligator.petName == null ? "" : alligator.petName);
458    save.setFloat("health", alligator.health);
459    save.setFloat("happiness", alligator.happiness);
460    save.setFloat("energy", alligator.energy);
461    save.setFloat("hunger", alligator.hunger);
462    save.setFloat("sickrisk", alligator.sickrisk);
463
464    // sickness / treatment
465    save.setBoolean("sick", isPetSick);
466    save.setString("sickness", currentSicknessName == null ? "" : currentSicknessName);
467    save.setBoolean("cleanerVisited", hasCleanerVisited);
468    save.setInt("activePrescriptionIndex", prescribedMedicineIndex);
469    save.setInt("treatmentDaysNeeded", prescriptionDaysRequired);
470    save.setInt("treatmentDaysCompleted", prescriptionDaysCompleted);
471    save.setInt("lastTreatmentDay", lastDoseTakenDay);
472
473    // tutorial / important progression flags
474    save.setBoolean("welcomepopupvisible", isWelcomePopupVisible);
475    save.setBoolean("firstinventoryclick", hasOpenedInventory);
476    save.setBoolean("fedsteak", hasFedSteak);
477    save.setBoolean("firstearnclick", hasOpenedEarnPanel);
478    save.setBoolean("firsthelpclick", hasUsedHelpTask);
479    save.setBoolean("firsttasktabclick", hasClickedTaskTab);
480    save.setBoolean("firstservicesclick", hasOpenedServices);
481    save.setBoolean("firstbuymedicine", hasFirstBoughtMedicine);
482    save.setBoolean("firstmedicinegiven", hasGivenFirstMedicine);
483    save.setBoolean("firstbankclick", hasViewedBank);
484    save.setBoolean("firstrestclick", hasUsedRest);
485    save.setBoolean("firstalligatorrest", hasAlligatorRestedOnce);
486    save.setBoolean("firstachievementsclick", hasOpenedAchievements);
487    save.setBoolean("firstachievementsclosed", hasClosedAchievements);
488    save.setBoolean("firstnextdayclick", hasAdvancedDay);
489    save.setBoolean("firsthelppopupshown", hasShownFirstHelpPopup);
490    save.setBoolean("firstLowQualityCareAlwaysSucceeds", isFirstLowQualityVetAttempt);
491    save.setBoolean("neverboughthighqualitycare", hasNeverBoughtHighQualityCare);
492    save.setBoolean("lowqualitycaregiven", hasUsedLowQualityVet);
493    save.setBoolean("bankpopupshown", hasShownBankPopup);
494    save.setBoolean("showplayarrow", isShowingPlayArrow);
495
496    // tutorial popup "already shown" flags — saved so they don't replay on resume
497    save.setBoolean("earnpopupshown", hasShownEarnPopup);
498    save.setBoolean("playpopupShown", hasShownPlayPopup);
499    save.setBoolean("jobpopupshown", hasShownJobPopup);
500    save.setBoolean("treatmentpopupshown", hasShownTreatmentPopup);
501    save.setBoolean("restpopupshown", hasShownRestPopup);
502    save.setBoolean("firstbankview", hasViewedBankFirstTime);
503    save.setBoolean("firstenergystabalized", hasEnergyStabilized);
504
505    // prescriptions — save both named fields (for backward compat) and array
506    String[] prescNames = {"enrofloxacinPresc","doxycyclinePresc","oseltamivirPresc","vitaminBComplexPresc",
507      "cyproheptadinePresc","potassiumChloridePresc","coenzymeQ10Presc","fluoxetinePresc",
508      "trazodonePresc","meloxicamPresc","calciumCarbonatePresc","activatedCharcoalPresc"};
509    for (int i = 0; i < prescNames.length; i++) save.setBoolean(prescNames[i], medicineIsPrescribed[i]);
510    save.setJSONArray("presc", booleanArrayToJson(medicineIsPrescribed));
511
512    // inventory
513    save.setJSONArray("inventoryslots", stringArrayToJson(inventorySlots));
```

```
  save.setJSONArray("medQtys", intArrayToJson(medicineQuantities));
  save.setJSONArray("snackQtys", intArrayToJson(snackQuantities));
  save.setJSONArray("meatQtys", intArrayToJson(meatQuantities));

  // bank
  JSONArray bankArr = new JSONArray();
  for (int i = 0; i < bankTransactionLog.size(); i++) {
    bankArr.append(bankTransactionLog.get(i));
  }
  save.setJSONArray("bankTransactions", bankArr);

  // counters / stats / achievements
  save.setInt("timesRestedSuccessfully", timesRestedSuccessfully);
  save.setInt("timesFedPet", timesFedPet);
  save.setInt("timesUsedVetCare", timesUsedVetCare);
  save.setInt("timesBoughtMedicine", timesBoughtMedicine);
  save.setInt("salaryUpgradeCount", salaryUpgradeCount);
  save.setInt("taskUpgradeCount", taskUpgradeCount);
  save.setInt("playPointUpgradeCount", playPointUpgradeCount);
  save.setInt("currentDay", currentDay);
  save.setInt("inventoryFullCount", inventoryFullCount);
  save.setInt("helpTaskCount", helpTaskCount);
  save.setInt("medicineGivenCount", medicineGivenCount);
  save.setInt("lowQualityCareCount", lowQualityCareCount);
  save.setInt("highQualityCareCount", highQualityCareCount);
  save.setInt("itemsBoughtCount", itemsBoughtCount);
  save.setInt("inventoryItemsUsedCount", inventoryItemsUsedCount);
  save.setFloat("totalMoneyEarned", totalMoneyEarned);
  save.setFloat("totalMoneySpent", totalMoneySpent);
  save.setFloat("highestMoneyBalance", highestMoneyBalance);
  save.setFloat("moneyEarnedFromSalary", moneyEarnedFromSalary);
  save.setFloat("moneyEarnedFromTasks", moneyEarnedFromTasks);
  save.setFloat("moneyEarnedFromMinigames", moneyEarnedFromMinigames);
  save.setInt("restAttempts", restAttempts);
  save.setFloat("totalEnergyRestoredFromResting", totalEnergyRestoredFromResting);
  save.setFloat("totalHealthRestored", totalHealthRestored);
  save.setFloat("totalHappinessRestored", totalHappinessRestored);
  save.setInt("cleanersHiredCount", cleanersHiredCount);
  save.setInt("bankTransactionsLoggedCount", bankTransactionsLoggedCount);
  save.setInt("walkersHiredCount", walkersHiredCount);
  save.setFloat("currentPlaySessionMoneyEarned", currentPlaySessionMoneyEarned);

  save.setInt("besthopscore", swampHopBestScore);
  save.setInt("bestsnatchscore", snatchBestScore);
  save.setInt("bestfetchscore", fetchBestScore);

  save.setJSONArray("achievementtiers", intArrayToJson(achievementTiers));

  saveJSONObject(save, "save.json");
}


void loadGame() {
  File saveFile = new File(sketchPath("data/save.json"));
  if (!saveFile.exists()) return;
  JSONObject save = loadJSONObject("save.json");
  if (save == null) return;

  // cutscene / setup progress
  isHomeScreenVisible = save.getBoolean("homescreenvisible", isHomeScreenVisible);
  isCutsceneActive = save.getBoolean("cutscenestart", isCutsceneActive);
  isNamingActive = save.getBoolean("inNaming", isNamingActive);
  isGameStarted = save.getBoolean("startrealgame", isGameStarted);
  isDogSelected = save.getBoolean("dogadopted", isDogSelected);
  isCatSelected = save.getBoolean("catadopted", isCatSelected);
```

```
    selectedAlligatorSkin = save.getInt("selectedAlligator", selectedAlligatorSkin);
    isNameChosen = save.getBoolean("namechosen", isNameChosen);
    defaultPetName = save.getString("alligatorname", defaultPetName);
    typedInput = save.getString("userInput", typedInput);

    // main progression
    isOnMainScreen = save.getBoolean("onmainscreen", isOnMainScreen);
    day = save.getInt("day", day);
    money = save.getFloat("money", money);
    job = save.getString("job", job);
    salary = save.getFloat("salary", salary);
    totalJobEarnings = save.getFloat("totalJobEarnings", totalJobEarnings);
    taskRewardAmount = save.getFloat("taskmoney", taskRewardAmount);
    moneyPerMinigamePoint = save.getFloat("moneyperpt", moneyPerMinigamePoint);
    pointUpgradeCost = save.getFloat("ptupgcost", pointUpgradeCost);
    salaryUpgradeCost = save.getFloat("salupgcost", salaryUpgradeCost);
    taskUpgradeCost = save.getFloat("taskupgcost", taskUpgradeCost);

    // pet state
    alligator.petName = save.getString("petName", alligator.petName);
    alligator.health = save.getFloat("health", alligator.health);
    alligator.happiness = save.getFloat("happiness", alligator.happiness);
    alligator.energy = save.getFloat("energy", alligator.energy);
    alligator.hunger = save.getFloat("hunger", alligator.hunger);
    alligator.sickrisk = save.getFloat("sickrisk", alligator.sickrisk);

    // sickness / treatment
    isPetSick = save.getBoolean("sick", isPetSick);
    currentSicknessName = save.getString("sickness", currentSicknessName);
    hasCleanerVisited = save.getBoolean("cleanerVisited", hasCleanerVisited);
    prescribedMedicineIndex = save.getInt("activePrescriptionIndex", prescribedMedicineIndex);
    prescriptionDaysRequired = save.getInt("treatmentDaysNeeded", prescriptionDaysRequired);
    prescriptionDaysCompleted = save.getInt("treatmentDaysCompleted", prescriptionDaysCompleted);
    lastDoseTakenDay = save.getInt("lastTreatmentDay", lastDoseTakenDay);

    // tutorial / progression flags
    isWelcomePopupVisible = save.getBoolean("welcomepopupvisible", isWelcomePopupVisible);
    hasOpenedInventory = save.getBoolean("firstinventoryclick", hasOpenedInventory);
    hasFedSteak = save.getBoolean("fedsteak", hasFedSteak);
    hasOpenedEarnPanel = save.getBoolean("firstearnclick", hasOpenedEarnPanel);
    hasUsedHelpTask = save.getBoolean("firsthelpclick", hasUsedHelpTask);
    hasClickedTaskTab = save.getBoolean("firsttasktabclick", hasClickedTaskTab);
    hasOpenedServices = save.getBoolean("firstservicesclick", hasOpenedServices);
    hasFirstBoughtMedicine = save.getBoolean("firstbuymedicine", hasFirstBoughtMedicine);
    hasGivenFirstMedicine = save.getBoolean("firstmedicinegiven", hasGivenFirstMedicine);
    hasViewedBank = save.getBoolean("firstbankclick", hasViewedBank);
    hasUsedRest = save.getBoolean("firstrestclick", hasUsedRest);
    hasAlligatorRestedOnce = save.getBoolean("firstalligatorrest", hasAlligatorRestedOnce);
    hasOpenedAchievements = save.getBoolean("firstachievementsclick", hasOpenedAchievements);
    hasClosedAchievements = save.getBoolean("firstachievementsclosed", hasClosedAchievements);
    hasAdvancedDay = save.getBoolean("firstnextdayclick", hasAdvancedDay);
    hasShownFirstHelpPopup = save.getBoolean("firsthelppopupshown", hasShownFirstHelpPopup);
    isFirstLowQualityVetAttempt = save.getBoolean("firstLowQualityCareAlwaysSucceeds",
isFirstLowQualityVetAttempt);
    hasNeverBoughtHighQualityCare = save.getBoolean("neverboughthighqualitycare",
hasNeverBoughtHighQualityCare);
    hasUsedLowQualityVet = save.getBoolean("lowqualitycaregiven", hasUsedLowQualityVet);
    hasShownBankPopup = save.getBoolean("bankpopupshown", hasShownBankPopup);
    isShowingPlayArrow = save.getBoolean("showplayarrow", isShowingPlayArrow);

    // tutorial popup "already shown" flags
    hasShownEarnPopup = save.getBoolean("earnpopupshown", hasShownEarnPopup);
    hasShownPlayPopup = save.getBoolean("playpopupShown", hasShownPlayPopup);
    hasShownJobPopup = save.getBoolean("jobpopupshown", hasShownJobPopup);
    hasShownTreatmentPopup = save.getBoolean("treatmentpopupshown", hasShownTreatmentPopup);
```

```
642 ■   hasShownRestPopup = save.getBoolean("restpopupshown", hasShownRestPopup);
643 ■   hasViewedBankFirstTime = save.getBoolean("firstbankview", hasViewedBankFirstTime);
644 ■   hasEnergyStabilized = save.getBoolean("firstenergystabalized", hasEnergyStabilized);
645 ■
646 ■   // prescriptions — load from named fields first, then override with medicineIsPrescribed[] array if present
647 ■   String[] prescNames = {"enrofloxacinPresc","doxycyclinePresc","oseltamivirPresc","vitaminBComplexPresc",
648 ■     "cyproheptadinePresc","potassiumChloridePresc","coenzymeQ10Presc","fluoxetinePresc",
649 ■     "trazodonePresc","meloxicamPresc","calciumCarbonatePresc","activatedCharcoalPresc"};
650 ■   for (int i = 0; i < prescNames.length; i++) medicineIsPrescribed[i] = save.getBoolean(prescNames[i],
medicineIsPrescribed[i]);
651 ■   JSONArray prescArr = save.getJSONArray("presc");
652 ■   if (prescArr != null) {
653 ■     for (int i = 0; i < min(medicineIsPrescribed.length, prescArr.size()); i++) {
654 ■       medicineIsPrescribed[i] = prescArr.getBoolean(i);
655 ■     }
656 ■   }
657 ■
658 ■   // inventory
659 ■   JSONArray invArr = save.getJSONArray("inventoryslots");
660 ■   if (invArr != null) {
661 ■     for (int i = 0; i < min(inventorySlots.length, invArr.size()); i++) {
662 ■       inventorySlots[i] = invArr.getString(i);
663 ■     }
664 ■   }
665 ■
666 ■   JSONArray medQtyArr = save.getJSONArray("medQtys");
667 ■   if (medQtyArr != null) {
668 ■     for (int i = 0; i < min(medicineQuantities.length, medQtyArr.size()); i++) {
669 ■       medicineQuantities[i] = medQtyArr.getInt(i);
670 ■     }
671 ■   }
672 ■
673 ■   JSONArray snackQtyArr = save.getJSONArray("snackQtys");
674 ■   if (snackQtyArr != null) {
675 ■     for (int i = 0; i < min(snackQuantities.length, snackQtyArr.size()); i++) {
676 ■       snackQuantities[i] = snackQtyArr.getInt(i);
677 ■     }
678 ■   }
679 ■
680 ■   JSONArray meatQtyArr = save.getJSONArray("meatQtys");
681 ■   if (meatQtyArr != null) {
682 ■     for (int i = 0; i < min(meatQuantities.length, meatQtyArr.size()); i++) {
683 ■       meatQuantities[i] = meatQtyArr.getInt(i);
684 ■     }
685 ■   }
686 ■
687 ■   // bank
688 ■   bankTransactionLog.clear();
689 ■   JSONArray bankArr = save.getJSONArray("bankTransactions");
690 ■   if (bankArr != null) {
691 ■     for (int i = 0; i < bankArr.size(); i++) {
692 ■       bankTransactionLog.add(bankArr.getString(i));
693 ■     }
694 ■   }
695 ■
696 ■   // counters / stats / achievements
697 ■   timesRestedSuccessfully = save.getInt("timesRestedSuccessfully", timesRestedSuccessfully);
698 ■   timesFedPet = save.getInt("timesFedPet", timesFedPet);
699 ■   timesUsedVetCare = save.getInt("timesUsedVetCare", timesUsedVetCare);
700 ■   timesBoughtMedicine = save.getInt("timesBoughtMedicine", timesBoughtMedicine);
701 ■   salaryUpgradeCount = save.getInt("salaryUpgradeCount", salaryUpgradeCount);
702 ■   taskUpgradeCount = save.getInt("taskUpgradeCount", taskUpgradeCount);
703 ■   playPointUpgradeCount = save.getInt("playPointUpgradeCount", playPointUpgradeCount);
704 ■   currentDay = save.getInt("currentDay", currentDay);
705 ■   inventoryFullCount = save.getInt("inventoryFullCount", inventoryFullCount);
```

```
    helpTaskCount = save.getInt("helpTaskCount", helpTaskCount);
    medicineGivenCount = save.getInt("medicineGivenCount", medicineGivenCount);
    lowQualityCareCount = save.getInt("lowQualityCareCount", lowQualityCareCount);
    highQualityCareCount = save.getInt("highQualityCareCount", highQualityCareCount);
    itemsBoughtCount = save.getInt("itemsBoughtCount", itemsBoughtCount);
    inventoryItemsUsedCount = save.getInt("inventoryItemsUsedCount", inventoryItemsUsedCount);
    totalMoneyEarned = save.getFloat("totalMoneyEarned", totalMoneyEarned);
    totalMoneySpent = save.getFloat("totalMoneySpent", totalMoneySpent);
    highestMoneyBalance = save.getFloat("highestMoneyBalance", highestMoneyBalance);
    moneyEarnedFromSalary = save.getFloat("moneyEarnedFromSalary", moneyEarnedFromSalary);
    moneyEarnedFromTasks = save.getFloat("moneyEarnedFromTasks", moneyEarnedFromTasks);
    moneyEarnedFromMinigames = save.getFloat("moneyEarnedFromMinigames", moneyEarnedFromMinigames);
    restAttempts = save.getInt("restAttempts", restAttempts);
    totalEnergyRestoredFromResting = save.getFloat("totalEnergyRestoredFromResting",
totalEnergyRestoredFromResting);
    totalHealthRestored = save.getFloat("totalHealthRestored", totalHealthRestored);
    totalHappinessRestored = save.getFloat("totalHappinessRestored", totalHappinessRestored);
    cleanersHiredCount = save.getInt("cleanersHiredCount", cleanersHiredCount);
    bankTransactionsLoggedCount = save.getInt("bankTransactionsLoggedCount", bankTransactionsLoggedCount);
    walkersHiredCount = save.getInt("walkersHiredCount", walkersHiredCount);
    currentPlaySessionMoneyEarned = save.getFloat("currentPlaySessionMoneyEarned",
currentPlaySessionMoneyEarned);

    swampHopBestScore = save.getInt("besthopscore", swampHopBestScore);
    snatchBestScore = save.getInt("bestsnatchscore", snatchBestScore);
    fetchBestScore = save.getInt("bestfetchscore", fetchBestScore);

    JSONArray achievementtiersArr = save.getJSONArray("achievementtiers");
    if (achievementtiersArr != null) {
      for (int i = 0; i < min(achievementTiers.length, achievementtiersArr.size()); i++) {
        achievementTiers[i] = achievementtiersArr.getInt(i);
      }
    }

    // reset temporary UI/runtime stuff so resume is clean
    isShowingInstructions = false;
    isShowingMusicSettings = false;
    isInventoryVisible = false;
    isShowingCantSell = false;
    isShowingPlayPopup = false;
    isShowingPlayArrow = false;
    isShowingEarnPopup = false;
    isEarnPanelOpen = false;
    isHelpTaskPending = false;
    isShowingFirstHelpPopup = false;
    isServicesOpen = false;
    isVetOpen = false;
    isShowingTreatmentPopup = false;
    isShowingVetFailedPopup = false;
    isStoreOpen = false;
    isViewingMedicineTab = false;
    isViewingSnacksTab = false;
    isViewingMeatTab = false;
    isBankOpen = false;
    isShowingBankPopup = false;
    isShowingRestPopup = false;
    isRestOpen = false;
    isAchievementsOpen = false;
    isShowingStoreClosedPopup = false;
    isNextDayPopupOpen = false;
    isDayEdited = false;
    isDayChangeConfirmed = false;
    isPlayClicked = false;
    isFadingOut = false;
    isOnChoiceScreen = false;
```

```
  isEnterSwampHop = false;
  isEnterSnackSnatch = false;
  isEnterFetchFrenzy = false;
  isExitingMinigame = false;
  selectedInventorySlot = -1;

  refreshAchievementData();
}

// ========================
// JSON Serialization Helpers
// Convert native arrays to JSONArray objects for use in saveGame().
// Processing's saveJSONObject() only accepts JSONArray/JSONObject children,
// so primitive arrays must be converted before they can be written to disk.
// ========================

// Converts a boolean[] to a JSONArray of boolean values
JSONArray booleanArrayToJson(boolean[] arr) {
  JSONArray j = new JSONArray();
  for (int i = 0; i < arr.length; i++) {
    j.setBoolean(i, arr[i]);
  }
  return j;
}

// Converts a String[] to a JSONArray; null entries become empty strings to avoid JSON errors
JSONArray stringArrayToJson(String[] arr) {
  JSONArray j = new JSONArray();
  for (int i = 0; i < arr.length; i++) {
    j.setString(i, arr[i] == null ? "" : arr[i]);
  }
  return j;
}

// Converts an int[] to a JSONArray of integer values
JSONArray intArrayToJson(int[] arr) {
  JSONArray j = new JSONArray();
  for (int i = 0; i < arr.length; i++) {
    j.setInt(i, arr[i]);
  }
  return j;
}
```

§ 5

E_Home_Screen.pde

# Home Screen

Home screen rendering, fade-in animation, instructions and music overlays

```
// ========================
// Home Screen State & Assets
// ========================

Fade homeFade = new Fade(255);  // home screen entry: starts black, fades to clear

PImage homescreen;
PImage instructions;
PImage musicscreen;

PFont times15;


// ========================
// Home Screen Flags
// ========================

boolean isHomeScreenVisible   = true;
boolean isShowingInstructions   = false;
boolean isShowingMusicSettings  = false;


// ========================
// Home Screen Rendering
// Handles fade-in, overlays, and UI visibility
// ========================
void homescreen() {

  imageMode(CORNER);
  rectMode(CORNER);

  // Fade in from black on initial load
  homeFade.stepIn(2);

  // Draw background slightly oversized to avoid edge gaps
  image(homescreen, -25, 0, width + 25, height + 100);

  // Draw fade overlay on top of the background
  homeFade.draw();

  // ------------------------
  // Instructions Overlay
  // ------------------------
  if (isShowingInstructions == true) {
    image(instructions, 0, -50, 1100, 700);
    instructionstext();
  }

  // ------------------------
  // Music Settings Overlay
  // ------------------------
  if (isShowingMusicSettings == true) {

    image(musicscreen, width * 0.08, height * 0.05, 900, 650);

    // Show ControlP5 audio controls
    cp5.getController("volume").setVisible(true);
    cp5.getController("musicOn").setVisible(true);
```

```
  } else {

    // Hide audio controls when music menu is closed
    cp5.getController("volume").setVisible(false);
    cp5.getController("musicOn").setVisible(false);
  }
}


// ========================
// Instructions Text Rendering
// Displays gameplay explanation and player guidance
// ========================
void instructionstext() {

  textMode(SHAPE);
  textAlign(LEFT, TOP);
  textFont(times15);
  textSize(20);
  fill(0);
  smooth(8);

  text("You have adopted a baby alligator.", 200, 150);
  text("Your goal is to keep it healthy, safe, and financially supported over time.", 200, 175);
  text("Each day, you will make decisions about:", 200, 200);

  text("• feeding", 200, 230);
  text("• resting", 200, 270);
  text("• playing", 200, 310);
  text("• medical care", 200, 350);
  text("• spending and earning money", 200, 390);

  text("Press + to move forward one day in time", 200, 430);
  text("Your choices affect your alligator's health, mood, and future expenses.", 200, 470);
  text("Watch your money and stats carefully.", 200, 495);
  text("Cheap choices may save money now but can cause problems later.", 200, 520);
  text("Features and responsibilities will be taught as you play.", 200, 545);
  text("There is no single \u201cright\u201d way to play. Learn from the consequences of your decisions.",
200, 570);
}
```

§ 6

F_Adoption_Cutscene.pde

# Cutscene & Pet Naming

Adoption center cutscene, sign reveal animation, alligator customization and naming screen

```
// ========================
// Cutscene / Story Flow State
// Controls fades, scene transitions, and adoption/naming progression
// ========================

boolean isNamingActive            = false;
boolean isCutsceneActive          = false;
boolean isEnteringAdoptionCenter  = false;
boolean isInsideAdoptionCenter    = false;
boolean isDogSelected             = false;
boolean isCatSelected             = false;
boolean isGameStarted             = false;

// ========================
// Cutscene Fade Instances
// Each covers one transition in the cutscene/naming flow.
// ========================
// cutFade1: phases 1+2 — fades to black, then from black to reveal city street
Fade cutFade1  = new Fade(0);
// cutFade2: phase 3 — fades to black entering adoption center, then from black to reveal interior
Fade cutFade2  = new Fade(0);
// cutFade3: phase 4 — fades to black for naming transition; reused as the naming screen entry fade
Fade cutFade3  = new Fade(0);
// namingFade: naming screen exit — fades to black before starting the main game
Fade namingFade = new Fade(0);


// ========================
// Timing / Animation State
// ========================

int   petSignDisplayTimer = 0;

float cityPanOffset  = 0;
float cityPanPauseTimer    = 0;


// ========================
// Cutscene Assets
// ========================

PImage outsideofadoptioncenter;
PImage adoptioncenterinterior;
PImage pickingcat;
PImage pickingdog;
PImage namingalligatorbackground;

PFont  times50;

// ========================
// Cutscene Main Function
// Handles: fade-in, pan, entering adoption center, choosing pet, transitioning to naming
// ========================
void cutscene() {

  rectMode(CORNER);
  imageMode(CORNER);
  noStroke();
```

```

  // ----------------------------------------------------
  // Phase 1: Fade to black (0→255) — screen goes dark before city is revealed
  // ----------------------------------------------------
  if (!cutFade1.outComplete) {
    cutFade1.stepOut(3);
    cutFade1.draw();
  }

  // ----------------------------------------------------
  // Phase 2: City pan — fade from black (255→0) to reveal the street scene
  // ----------------------------------------------------
  else {

    // Pan the background until the adoption center is reached
    if (cityPanOffset <= 765) {
      cityPanOffset += 3;
    } else {

      // Pause briefly before transitioning inside
      cityPanPauseTimer++;

      if (cityPanPauseTimer >= 40) {
        isEnteringAdoptionCenter = true;
      }
    }

    image(outsideofadoptioncenter, 0 - cityPanOffset, -190, width * 1.75, height * 1.75);

    // Overlay fades away to reveal the street
    cutFade1.stepIn(2);
    cutFade1.draw();
  }

  // ----------------------------------------------------
  // Phase 3: Transition inside the adoption center
  //   — fade to black (cutFade2 outComplete), then reveal the interior
  // ----------------------------------------------------
  if (isEnteringAdoptionCenter) {

    // Fade to black before showing the interior
    if (!cutFade2.outComplete) {
      cutFade2.stepOut(8);
      cutFade2.draw();
    }

    // Interior scene
    else {

      isInsideAdoptionCenter = true;

      image(adoptioncenterinterior, 0, 0, width, height);

      // Fade from black into the interior
      cutFade2.stepIn(3);
      cutFade2.draw();

      // Show the pet sign animation based on selection
      if (isDogSelected) signanimation(pickingdog);
      if (isCatSelected) signanimation(pickingcat);

      // After a pet is chosen, wait briefly, then transition to naming
      if (isDogSelected == true || isCatSelected == true) {

        petSignDisplayTimer++;
```

```
124 ■
125 ■         if (petSignDisplayTimer > 220) {
126 ■
127 ■           // Fade to black (cutFade3) before switching to naming screen
128 ■           if (!cutFade3.outComplete) {
129 ■             cutFade3.stepOut(4);
130 ■             cutFade3.draw();
131 ■           }
132 ■
133 ■           // Naming segment begins once cutFade3 is fully black
134 ■           else {
135 ■             isNamingActive = true;
136 ■           }
137 ■
138 ■           // Extra fade step to accelerate cutFade2 completion (preserves original timing)
139 ■           cutFade2.stepIn(3);
140 ■         }
141 ■       }
142 ■     }
143 ■   }
144 ■ }
145 ■
146 ■
147 ■ // ========================
148 ■ // Sign Reveal Animation State
149 ■ // Shared by dog/cat reveal animations
150 ■ // ========================
151 ■
152 ■ float signRevealProgress = 0;
153 ■ float signRevealSpeed    = 0.015;
154 ■
155 ■ float signBaseX          = width / 2 - 240;
156 ■
157 ■ float signMaskLineY      = 191;
158 ■ float signFinalY         = height / 2 - 200;
159 ■
160 ■
161 ■ // ========================
162 ■ // Sign Animation
163 ■ // Reveals the sign upward by slicing the image
164 ■ // Used for both dog and cat signs — pass the appropriate PImage
165 ■ // ========================
166 ■ void signanimation(PImage img) {
167 ■
168 ■   float x      = width / 2.0 - 230;
169 ■   float finalY = height / 2.0 - 200;
170 ■
171 ■   float targetW = 480;
172 ■   float targetH = 350;
173 ■
174 ■   float startY = signMaskLineY - targetH;
175 ■
176 ■   // Reveal increases until full height is shown
177 ■   signRevealProgress = min(signRevealProgress + signRevealSpeed, 1);
178 ■
179 ■   float y = lerp(startY, finalY, signRevealProgress);
180 ■
181 ■   int hSrc = constrain(int(img.height * signRevealProgress), 0, img.height);
182 ■
183 ■   int sy = img.height - hSrc;
184 ■
185 ■   float scaleY  = targetH / (float)img.height;
186 ■   float hDraw   = hSrc * scaleY;
187 ■   float drawTop = y + (targetH - hDraw);
188 ■
```

```
  // Clip to the hide line (prevents drawing above the mask line)
  if (drawTop < signMaskLineY) {

    float cutDraw = signMaskLineY - drawTop;
    int cutSrc    = int(cutDraw / scaleY);

    sy   += cutSrc;
    hSrc -= cutSrc;

    if (hSrc <= 0) return;

    hDraw   = hSrc * scaleY;
    drawTop = signMaskLineY;
  }

  // Draw only the visible slice
  PImage slice = img.get(0, sy, img.width, hSrc);

  imageMode(CORNER);
  image(slice, x, drawTop, targetW, hDraw);
}



// ========================
// Naming Screen
// Shows naming background, pet sprite, and input UI
// ========================

boolean isNamingScreenShown = false;
int selectedAlligatorSkin = 0;

void namingalligatorsegment() {
  isNamingScreenShown = true;
  isInsideAdoptionCenter = false;

  imageMode(CORNER);
  image(namingalligatorbackground, 0, 0, width, height);

  image(alligator.neutralalligator, width * 0.27 - 300, height * 0.5, (width / 2) * 0.9, height / 2);

  tint(0,255,0);
  image(alligator.neutralalligator, width * 0.27, height * 0.5, (width / 2) * 0.9, height / 2);

  tint(70,130,255);
  image(alligator.neutralalligator, width * 0.27 + 300, height * 0.5, (width / 2) * 0.9, height / 2);
  noTint();

  rectMode(CENTER);
  stroke(255);
  strokeWeight(2);
  textAlign(CENTER, CENTER);
  textSize(16);

  float[] btnXOffsets = {-300, 0, 300};
  for (int i = 0; i < 3; i++) {
    float btnX = (width * 0.27 + btnXOffsets[i]) + ((width / 2) * 0.9) / 2;
    if (selectedAlligatorSkin == i) fill(0, 255, 0, 120);
    else fill(80, 220);
    rect(btnX, 659, 120, 35);
    fill(255);
    if (selectedAlligatorSkin == i) text("SELECTED", btnX, 659);
    else text("SELECT", btnX, 659);
  }

```

```
  nameField.show();
  confirmBtn.show();

  fill(255);
  textFont(times50);
  text("Customize Your Alligator:", int(width/2), int(height * 0.35));
  textSize(30);
  text("Name: ", 320, 320);
  cp5.draw();

  // Display validation error message in red if the last confirm attempt was invalid.
  // petNameValidationError is set by isValidName() in D_General_Functions.pde and cleared
  // as soon as the user submits a valid name.
  if (petNameValidationError != null && petNameValidationError.length() > 0) {
    fill(255, 80, 80);
    textFont(times30);
    textSize(15);
    textAlign(LEFT, CENTER);
    text(petNameValidationError, width * 0.35, 355);
  }

  noStroke();
  rectMode(CORNER);

  // Entry fade: cutFade3 arrives at 255 from the cutscene transition; step to clear
  cutFade3.stepIn(4);
  cutFade3.draw();

  // Exit fade: once name is confirmed, fade to black then switch to the main game
  if (isNameChosen) {
    if (namingFade.stepOut(4)) {
      isGameStarted = true;
      isOnMainScreen = true;
      isCutsceneActive = false;
      isNamingActive = false;
    }
    namingFade.draw();
  }
}
```

§ 7

G_Main_Screen.pde

# Main Gameplay Screen

Core HUD, stat bars, tutorial popups, day progression, game over screen

```
// ========================
// G_Main_Screen.pde
// Core main gameplay screen: state flags, HUD rendering, mainscreen() orchestrator,
// stat bars, day progression, tutorial popups, quit/game-over overlays.
//
// Sub-systems live in their own files:
//   G_Inventory.pde   — inventory panel + item data
//   G_Store.pde       — store panel + fade logic
//   G_Bank.pde        — bank panel + transaction log
//   G_Earn.pde        — earn panel, job finder, tasks/upgrades
//   G_Services.pde    — vet, walker, cleaner, prescriptions
//   G_Rest.pde        — rest bar mini-game
//   G_Achievements.pde — achievement system + panel
// ========================


// ========================
// UI Image Assets
// ========================
PImage mainscreen;
PImage mainscreenbuttons;
PImage achievementsbutton;
PImage settingsbutton;
PImage earnbutton;
PImage popupbackground;
PImage steak;
PImage redarrow;

PFont arcade;
PFont times30;


// ========================
// Screen State Flags
// ========================
boolean isOnMainScreen = false;

// Tutorial progression flags
boolean isWelcomePopupVisible  = true;
boolean isInventoryVisible     = false;
boolean hasOpenedInventory     = false;
boolean hasFedSteak            = false;
boolean isShowingCantSell      = false;
boolean isShowingPlayPopup     = false;
boolean hasShownPlayPopup      = false;
boolean isShowingPlayArrow     = false;
boolean isPlayClicked          = false;
boolean isFadingOut            = false;
boolean hasEnergyStabilized    = false;
boolean isNextDayPopupOpen     = false;
boolean hasAdvancedDay         = false;
boolean isShowingQuitDialog    = false;
boolean isGameOver             = false;
boolean hasFirstBoughtMedicine = false;
boolean hasGivenFirstMedicine  = false;


// ========================
```

```
// UI Runtime State
// ========================
int selectedInventorySlot = -1;
int playPopupDelayTimer   = 0;

String selectedItemName;


// ========================
// HUD Stat Bars (minigame overlay — main screen creates them locally each frame)
// ========================
StatBar healthbar       = new StatBar(99,      66.15f,   180, 14);
StatBar happinessbar    = new StatBar(129.25f, 101.5f,   180, 14);
StatBar energybar       = new StatBar(99,      136.15f,  180, 14);
StatBar sicknessriskbar = new StatBar(189.2f,  170.555f, 180, 14);
StatBar hungerbar       = new StatBar(99,      206.15f,  180, 14);


// ========================
// Main Screen Entry/Exit Fades
// ========================
Fade mainFade = new Fade(255);   // starts black; fades to clear on entry


// ========================
// Core Economy & Progression
// ========================
int   day   = 1;
float money = 0;


// ========================
// Food Thought Bubble Animation
// ========================
PImage cloudframe1, cloudframe2, cloudframe3;
int cloudFrameTimer = 0;


// ========================
// Global Pet Reference
// Instantiated in D_General_Functions.pde :: fileWork()
// ========================
Pet alligator;


// ========================
// Guide Arrow Animation
// ========================
float guideArrowOffset    = 0;
boolean isGuideArrowMovingUp = true;


// ========================
// Day / Sickness State
// ========================
int     sickRoll           = 0;
boolean isDayEdited        = false;
boolean isDayChangeConfirmed = false;
String  sicknessStatusText = "";
String  salaryInfoText     = "";


// ========================
// mainscreen()
// Central render function — draws background, pet, HUD, and delegates to sub-panels.
```

```
// ========================
void mainscreen() {
  imageMode(CORNER);
  isNamingActive = false;
  if (!isOnChoiceScreen) isOnMainScreen = true;

  if (alligator.health <= 0 && !isGameOver) {
    isGameOver = true;
  }

  image(mainscreen, 0, 0, width, height);
  if (isPetSick) {
    alligator.sickmood();
  } else if (alligator.hunger > 80) {
    alligator.hungrymood();
  } else if (alligator.energy > 80) {
    alligator.energeticmood();
  } else {
    alligator.neutralmood();
  }

  image(mainscreenbuttons, 0, height * 0.52f, 1000, 600);
  image(achievementsbutton, width * 0.806f, height * 0.21f, 320, 190);
  image(settingsbutton, width * 0.83655f, -35, 248, 168);
  image(earnbutton, width * 0.845f, height * 0.075f, 231, 177);

  textFont(arcade);
  textAlign(RIGHT, CENTER);
  fill(255);
  text("$" + String.format("%03.2f", money) + "   " + "Day " + day, width * 0.9f, height * 0.061f);

  stroke(0);
  statbars();
  imageMode(CORNER);

  // Food thought bubble (Day 1 until fed)
  if (!hasFedSteak) {
    cloudFrameTimer++;
    if (cloudFrameTimer >= 30) cloudFrameTimer = 0;
    if      (cloudFrameTimer < 10) image(cloudframe1, width*0.48f, height*0.35f);
    else if (cloudFrameTimer < 20) image(cloudframe2, width*0.48f, height*0.35f);
    else                           image(cloudframe3, width*0.48f, height*0.35f);
    imageMode(CENTER);
    image(steak, 632, 326, steak.width/8, steak.height/8);
  }

  if (isWelcomePopupVisible) {
    image(popupbackground, width/2, height*0.42f, popupbackground.width*0.6f, popupbackground.height*0.6f);
    fill(0);
    textFont(times50);
    textSize(20);
    drawWrappedTextInBox("Welcome to your first day of alligator pet care! It seems like " +
alligator.petName + " is already hungry! Head to your inventory after closing this window to feed him the food you
were given at the adoption center!", 338, 271, 761, 400, 6);
  }

  // Tutorial guide arrows
  if (hasOpenedInventory == false || (isStoreOpen == false && hasFirstBoughtMedicine && !isInventoryVisible
&& !hasGivenFirstMedicine)) {
    redarrow(239.5f, 500, "down");
  }
  if (isShowingPlayArrow) redarrow(656, 500, "down");
  if ((hasShownEarnPopup || isShowingEarnPopup) && isEarnPanelOpen == false && !hasOpenedEarnPanel)
redarrow(940, 134, "right");
  if (hasShownBankPopup && !hasViewedBank) redarrow(861, 500, "down");
```

```
  if (hasShownTreatmentPopup && !isStoreOpen && !hasFirstBoughtMedicine) redarrow(337, 500, "down");

  if (isInventoryVisible) inventory();

  // Play popup logic
  if (hasFedSteak && !hasShownPlayPopup && !isShowingPlayPopup) {
    playPopupDelayTimer++;
    if (playPopupDelayTimer > 60 && alligator.energy > 80) {
      isShowingPlayPopup = true;
      isShowingPlayArrow = true;
    }
  }
  if (alligator.energy < 80 && hasShownPlayPopup) hasEnergyStabilized = true;
  if (alligator.energy < 80 && hasEnergyStabilized) isShowingPlayArrow = false;
  if (isShowingPlayPopup) playpopup();

  // Earn popup logic
  if (hasEnergyStabilized && isExitingMinigame && !hasShownEarnPopup) {
    earnPopupDelayTimer++;
    if (earnPopupDelayTimer > 60) isShowingEarnPopup = true;
  }
  if (isShowingEarnPopup && !isInventoryVisible) earnpopup();

  if (isEarnPanelOpen) earn();

  if (job.equals("cashier") && !hasShownJobPopup && !isShowingJobPopup && !isJobFinderOpen && !isTasksPanelOpen) {
    isShowingJobPopup = true;
  }
  if (isEarnPanelOpen && !isJobFinderOpen && !isTasksPanelOpen && job.equals("unemployed")) redarrow(155, 455, "right");
  if (isShowingFirstHelpPopup) help();
  if (hasShownFirstHelpPopup && !isServicesOpen && !hasOpenedServices) redarrow(760, 502, "down");
  if (isServicesOpen && !isVetOpen) services();
  if (!isJobFinderOpen && job.equals("cashier") && !hasShownJobPopup) jobpopup();
  if (hasShownJobPopup && !hasClickedTaskTab && !isJobFinderOpen && !job.equals("unemployed")) redarrow(529, 455, "right");

  if (isVetOpen) vet();
  if (isShowingTreatmentPopup) treatmentpopup();
  if (isStoreOpen) store();

  if (hasGivenFirstMedicine && !hasShownBankPopup) {
    bankpopup();
    isShowingBankPopup = true;
  }
  if (isBankOpen) bank();

  if (hasViewedBankFirstTime && !hasShownRestPopup) {
    isShowingRestPopup = true;
    restpopup();
  }
  if (hasShownRestPopup && !isRestOpen && !hasAlligatorRestedOnce) redarrow(439, 500, "down");
  if (isRestOpen) rest();

  if (hasAlligatorRestedOnce && !hasOpenedAchievements) redarrow(940, 231, "right");
  if (isAchievementsOpen) achievements();

  if (money >= highestMoneyBalance) highestMoneyBalance = money;

  if (isShowingStoreClosedPopup) storeclosedpopup();
  if (hasClosedAchievements && !hasAdvancedDay) redarrow(width/2, 500, "down");
  if (isNextDayPopupOpen) nextday();
  if (isShowingQuitDialog) quitpopup();

```

```
  // Fade overlays
  noStroke();
  if (isPlayClicked) {
    isFadingOut = true;
    if (mainFade.isBlack()) {
      if (!isExitingMinigame) play();
      isOnMainScreen = false;
    } else {
      mainFade.stepOut(2);
      mainFade.draw();
    }
  } else if (isStoreMainScreenFading) {
    if (mainFade.isBlack()) {
      isStoreOpen = true;
      isStoreMainScreenFading = false;
      isStoreEntryFading = true;
      storeFade.setClear();
      isStoreFadingOut = false;
    } else {
      mainFade.stepOut(2);
    }
    mainFade.draw();
  } else if (isStoreEntryFading) {
    if (mainFade.stepIn(2)) isStoreEntryFading = false;
    mainFade.draw();
  } else if (!isStoreOpen) {
    mainFade.stepIn(1);
    mainFade.draw();
  }

  if (isGameOver) gameOverScreen();
}


// ========================
// Stat Bar Display (HUD)
// ========================
void statbars() {
  textAlign(LEFT, CENTER);
  strokeWeight(2);
  textSize(27);
  text("Pet Statistics:", width * 0.015f, height * 0.05f);

  textSize(20);
  text("Health:", width * 0.015f, height * 0.1f);
  StatBar healthbar = new StatBar(width * 0.09f, height * 0.0945f, 180, 14);
  healthbar.setValue(alligator.health);
  healthbar.drawpositive();

  fill(255);
  text("Happiness:", width * 0.015f, height * 0.15f);
  StatBar happinessbar = new StatBar(width * 0.1175f, height * 0.145f, 180, 14);
  happinessbar.setValue(alligator.happiness);
  happinessbar.drawpositive();

  fill(255);
  text("Energy:", width * 0.015f, height * 0.2f);
  StatBar energybar = new StatBar(width * 0.09f, height * 0.1945f, 180, 14);
  energybar.setValue(alligator.energy);
  energybar.drawenergyscale();

  fill(255);
  text("Risk of Sickness:", width * 0.015f, height * 0.25f);
  StatBar sicknessriskbar = new StatBar(width * 0.172f, height * 0.24365f, 180, 14);
  sicknessriskbar.setValue(alligator.sickrisk);
```

```
  sicknessriskbar.drawnegative();

  fill(255);
  text("Hunger:", width * 0.015f, height * 0.3f);
  StatBar hungerbar = new StatBar(width * 0.09f, height * 0.2945f, 180, 14);
  hungerbar.setValue(alligator.hunger);
  hungerbar.drawnegative();
}


// ========================
// StatBar Class
// ========================
class StatBar {
  float x, y, w, h;
  float value = 100;

  StatBar(float x, float y, float w, float h) {
    this.x = x; this.y = y; this.w = w; this.h = h;
  }

  void setValue(float v) { value = constrain(v, 0, 100); }

  void drawpositive() {
    fill(50); rect(x, y, w, h, 4);
    if      (value >= 90) fill(0, 200, 100);
    else if (value >= 70) fill(182, 232, 35);
    else if (value >= 50) fill(227, 220, 0);
    else if (value >= 40) fill(227, 155, 0);
    else                  fill(201, 8, 8);
    rect(x, y, map(value, 0, 100, 0, w), h, 4);
  }

  void drawnegative() {
    fill(50); rect(x, y, w, h, 4);
    if      (value >= 90) fill(201, 8, 8);
    else if (value >= 70) fill(227, 155, 0);
    else if (value >= 50) fill(227, 220, 0);
    else if (value >= 40) fill(182, 232, 35);
    else                  fill(0, 200, 100);
    rect(x, y, map(value, 0, 100, 0, w), h, 4);
  }

  void drawenergyscale() {
    fill(50); rect(x, y, w, h, 4);
    if      (value >= 80) fill(201, 8, 8);
    else if (value >= 70) fill(182, 232, 35);
    else if (value >= 40) fill(0, 200, 100);
    else if (value >= 20) fill(182, 232, 35);
    else                  fill(201, 8, 8);
    rect(x, y, map(value, 0, 100, 0, w), h, 4);
  }
}


// ========================
// drawWrappedTextInBox
// Word-wraps a string inside a bounding rectangle.
// ========================
void drawWrappedTextInBox(String sentence, float leftX, float topY, float rightX, float bottomY, float
extraSpacing) {
  float maxW  = rightX - leftX;
  float lineH = textAscent() + textDescent() + extraSpacing;
  float y     = topY;
  String[] words = splitTokens(sentence, " ");
```

```
  String line = "";

  textAlign(LEFT, TOP);

  for (int i = 0; i < words.length; i++) {
    String testLine = (line.length() == 0) ? words[i] : line + " " + words[i];
    if (textWidth(testLine) <= maxW) {
      line = testLine;
    } else {
      if (y + (textAscent() + textDescent()) > bottomY) return;
      text(line, leftX, y);
      y += lineH;
      line = words[i];
    }
  }
  if (line.length() > 0) {
    if (y + (textAscent() + textDescent()) > bottomY) return;
    text(line, leftX, y);
  }
}


// ========================
// redarrow — animated bouncing guide arrow
// ========================
void redarrow(float arrowx, float arrowy, String direction) {
  imageMode(CENTER);

  float drawX = arrowx;
  float drawY = arrowy;

  if (direction.equals("right")) drawX = arrowx + guideArrowOffset;
  else                           drawY = arrowy + guideArrowOffset;

  pushMatrix();
  translate(drawX, drawY);
  if (direction.equals("right")) rotate(-HALF_PI);
  image(redarrow, 0, 0, redarrow.width/3, redarrow.height/3);
  popMatrix();

  if (isGuideArrowMovingUp) {
    guideArrowOffset--;
    if (guideArrowOffset <= -25) isGuideArrowMovingUp = false;
  } else {
    guideArrowOffset++;
    if (guideArrowOffset >= 0) isGuideArrowMovingUp = true;
  }
}


// ========================
// Tutorial Popups (main-screen level)
// ========================
void playpopup() {
  imageMode(CENTER);
  image(popupbackground, width/2, height*0.42f, popupbackground.width*0.6f, popupbackground.height*0.6f);
  fill(0);
  textFont(times50);
  textSize(20);
  drawWrappedTextInBox("Uh oh! Feeding the steak to " + alligator.petName + " gave them too much energy. An alligator with too much energy is very dangerous! Let " + alligator.petName + " play after closing this window to stabilize their energy!", 338, 271, 761, 400, 6);
}


```

```
// ========================
// Next Day Logic
// ========================
void nextday() {
  imageMode(CENTER);
  image(popupbackground, width/2, height*0.42f, popupbackground.width*0.6f, popupbackground.height*0.6f);

  fill(0);
  textFont(times50);
  textSize(20);

  if (!isDayEdited) {
    decideSicknessForNewDay();
    daychanges();
    day++;
    currentDay++;
    isDayEdited = true;
  }

  drawWrappedTextInBox(
    "Welcome to day " + day + "! " + sicknessStatusText + salaryInfoText + " Check " + alligator.petName + "
's stats to guide your care!",
    338, 271, 761, 400, 6
  );
}

void decideSicknessForNewDay() {
  sickRoll = int(random(0, 101));

  if (sickRoll <= alligator.sickrisk && !isPetSick) {
    isPetSick = true;

    int sicknessIndex = 0;
    if      (alligator.hunger >= 85)                                    sicknessIndex = 4;
    else if (alligator.energy <= 10)                                    sicknessIndex = 6;
    else if (alligator.happiness <= 15 || alligator.energy >= 90)       sicknessIndex = 8;
    else if (alligator.hunger >= 70)                                    sicknessIndex = 3;
    else if (alligator.energy <= 20)                                    sicknessIndex = 5;
    else if (alligator.happiness <= 25)                                 sicknessIndex = 7;
    else if (random(1) < 0.20)                                          sicknessIndex = 9;
    else if (random(1) < 0.20)                                          sicknessIndex = 11;
    else if (random(1) < 0.20)                                          sicknessIndex = 10;
    else if (random(1) < 0.35)                                          sicknessIndex = 1;
    else if (random(1) < 0.20)                                          sicknessIndex = 2;

    currentSicknessName = sicknessNames[sicknessIndex];
  } else if (!isPetSick) {
    currentSicknessName = "";
  }
}

void daychanges() {
  restAttemptsRemaining = 2;
  alligator.happiness -= 10;
  alligator.hunger    += 40;
  alligator.energy    += 40;

  if (!hasCleanerVisited) alligator.sickrisk += 15;
  hasCleanerVisited = false;

  if (isPetSick && currentSicknessName != null && !currentSicknessName.equals("")) {
    alligator.health -= 20;

    if (currentSicknessName.equals(sicknessNames[2])) {
      sicknessStatusText = alligator.petName + " currently has the flu.";
```

```
    } else if (
      currentSicknessName.equals(sicknessNames[6]) ||
      currentSicknessName.equals(sicknessNames[7]) ||
      currentSicknessName.equals(sicknessNames[8]) ||
      currentSicknessName.equals(sicknessNames[11]) ||
      currentSicknessName.equals(sicknessNames[4])
    ) {
      sicknessStatusText = alligator.petName + " has been diagnosed with " + currentSicknessName + ".";
    } else {
      String article = "a";
      char first = currentSicknessName.toLowerCase().charAt(0);
      if (first == 'a' || first == 'e' || first == 'i' || first == 'o' || first == 'u') article = "an";
      sicknessStatusText = alligator.petName + " currently has " + article + " " + currentSicknessName + ".";
    }
  } else {
    sicknessStatusText = alligator.petName + " is not currently sick.";
  }

  if (!job.equals("unemployed")) {
    salaryInfoText = " You have earned $" + nf(salary, 0, 2) + " (salary).";
    money                 += salary;
    totalJobEarnings      += salary;
    totalMoneyEarned      += salary;
    moneyEarnedFromSalary += salary;
    bankTransactionsLoggedCount++;
    bankTransactionLog.add("Transaction: Salary " + "(+$" + nf(salary, 0, 2) + ")");
  } else {
    salaryInfoText = "";
  }
}


// ========================
// Quit Dialog
// ========================
void quitpopup() {
  imageMode(CENTER);
  image(popupbackground, width/2, height*0.42f, popupbackground.width*0.6f, popupbackground.height*0.6f);
  rectMode(CENTER);
  strokeWeight(3);
  fill(255, 0, 0);
  rect(width/2, 325, 135, 35);
  textAlign(CENTER, CENTER);
  fill(255);
  text("SAVE & QUIT", width/2, 325);
  rectMode(CORNER);
  noStroke();
}

void quit() {
  saveGame();
  exit();
}


// ========================
// Game Over Screen
// ========================
void gameOverScreen() {
  float cx     = width  / 2.0;
  float panelW = 700;
  float panelH = 430;
  float panelTop = height / 2.0 - panelH / 2.0;

  noStroke();
```

```
    fill(0, 200);
    rectMode(CORNER);
    rect(0, 0, width, height);

    fill(30, 15, 10);
    stroke(180, 50, 50);
    strokeWeight(4);
    rectMode(CENTER);
    rect(cx, height / 2.0, panelW, panelH, 18);
    noStroke();

    textFont(arcade);
    textAlign(CENTER, CENTER);
    String petName = (alligator != null) ? alligator.petName : "Your alligator";

    float y = panelTop + 60;
    textSize(52);
    fill(200, 50, 50);
    text("GAME OVER", cx, y);

    y += 70;
    textSize(20);
    fill(230, 210, 200);
    text(petName + " has passed away.", cx, y);

    y += 38;
    textSize(15);
    fill(210, 190, 180);
    float innerLeft  = cx - 290;
    float innerRight = cx + 290;
    drawWrappedTextInBox(
      "Without enough food, medicine, or care, " + petName +
      "'s health slowly fell to zero. Alligators need consistent " +
      "nutrition and regular checkups to stay strong. " + petName +
      " lived for " + (day - 1) + " day" + (day - 1 == 1 ? "" : "s") +
      " under your care.",
      innerLeft, y, innerRight, y + 115, 5
    );

    y += 120;
    stroke(180, 50, 50, 120);
    strokeWeight(1);
    line(cx - 300, y, cx + 300, y);
    noStroke();

    y += 28;
    textFont(arcade);
    textSize(13);
    fill(160, 140, 130);
    textAlign(CENTER, CENTER);
    text("Tip: Visit the vet and feed " + petName + " nutritious meat to keep his health up.", cx, y);

    y += 52;
    fill(180, 40, 40);
    stroke(220, 80, 80);
    strokeWeight(2);
    rectMode(CENTER);
    rect(cx, y, 200, 50, 8);
    noStroke();
    textSize(26);
    fill(255);
    textAlign(CENTER, CENTER);
    text("QUIT", cx, y);
    rectMode(CORNER);
}
```

§ 8

G_Inventory.pde

# Inventory System

Item data arrays, 12-slot inventory panel, item detail display helpers

```
// ========================
// G_Inventory.pde
// Inventory panel rendering, item detail display helpers, and inventory utility functions.
// ========================


// ========================
// Inventory Data
// ========================
String[] inventorySlots = {
  "Steak", "EMPTY", "EMPTY", "EMPTY",
  "EMPTY", "EMPTY", "EMPTY", "EMPTY",
  "EMPTY", "EMPTY", "EMPTY", "EMPTY"
};
int filledSlotCount = 0;

int[] medicineDefaultQuantities = {
  2, // Enrofloxacin
  3, // Doxycycline
  3, // Oseltamivir
  3, // Vitamin B-Complex
  2, // Cyproheptadine
  2, // Potassium Chloride
  2, // Coenzyme Q10
  4, // Fluoxetine
  3, // Trazodone
  2, // Meloxicam
  3, // Calcium Carbonate
  1  // Activated Charcoal
};
int[] medicineQuantities = { 0,0,0,0,0,0,0,0,0,0,0,0 };
int[] snackQuantities    = { 0,0,0,0,0,0,0,0,0,0,0,0 };
int[] meatQuantities     = { 0,0,0,0,0,0,0,0,0,0,0,0 };


// ========================
// Item Name / Description Arrays
// ========================
String[] medicineItemList = {
  "Enrofloxacin","Doxycycline","Oseltamivir","Vitamin B-Complex",
  "Cyproheptadine","Potassium Chloride","Coenzyme Q10","Fluoxetine",
  "Trazodone","Meloxicam","Calcium Carbonate","Activated Charcoal"
};
boolean[] medicineIsPrescribed = new boolean[12];

String[] medicineNames = {
  "Enrofloxacin","Doxycycline","Oseltamivir","Vitamin B-Complex",
  "Cyproheptadine","Potassium Chloride","Coenzyme Q10","Fluoxetine",
  "Trazodone","Meloxicam","Calcium Carbonate","Activated Charcoal"
};
String[] medicineDesc1 = {
  "Pet with an infection? Give 1 pill daily",
  "Pet with a cold? Give 1 pill daily",
  "Pet with a flu? Give 1 pill daily",
  "Pet malnourished? Give 1 pill daily",
  "Pet is starving? Give 1 pill daily",
  "Pet has low energy? Give 1 pill daily",
  "Pet exhausted? Give 1 pill daily",
```

```
  "Pet depressed? Give 1 pill daily",
  "Pet stressed? Give 1 pill daily",
  "Pet injured? Give 1 pill daily",
  "Pet has weak bones? Give 1 pill daily",
  "Pet is food poisoned? Give 1 pill"
};
String[] medicineDesc2 = {
  "for 2 days. Or your pet dies.",
  "for 3 days. Or your pet dies.",
  "for 3 days.",
  "for 3 days.",
  "for 2 days.",
  "for 2 days.",
  "for 2 days.",
  "for 4 days.",
  "for 3 days.",
  "for 2 days.",
  "for 3 days.",
  "immediately to remove toxins."
};

String[] snackItemList = {
  "Nachos","Cheesepuffs","Chips","Chocolate Bar","Cookies","Crackers",
  "Energy Drink","Granola Bar","Popcorn","Pretzels","Soda","Trail Mix"
};
String[] snackNames = {
  "Nachos","Cheesepuffs","Chips","Chocolate Bar","Cookies","Crackers",
  "Energy Drink","Granola Bar","Popcorn","Pretzels","Soda","Trail Mix"
};
String[] snackDesc1 = {
  "Messy cheesy chips with heavy seasoning.",
  "Crunchy cheese puffs packed with salt",
  "Crispy potato chips loaded with oil",
  "Sugary chocolate bar with lots of",
  "Sweet baked cookies full of sugar",
  "Light crunchy crackers that are",
  "Very high caffeine and sugar.",
  "A packaged granola bar that seems",
  "Buttery popcorn with lots of salt",
  "Twisty salty pretzels with lots of",
  "Pure sugar and fizz.",
  "A snack mix with salty and sweet"
};
String[] snackDesc2 = {
  "Tasty, but definitely not healthy.",
  "and artificial flavoring.",
  "and sodium.",
  "processed sweeteners.",
  "and refined carbs.",
  "salty and not very filling.",
  "A terrible choice for a pet.",
  "healthy, but still processed.",
  "and flavor dust.",
  "sodium.",
  "Not suitable for an alligator.",
  "bits all combined together."
};
String[] snackStats = {
  "-40 Hunger   -15 Health   +5 Happiness   +30 Energy",
  "-25 Hunger   -5 Health   +20 Happiness   +10 Energy",
  "-30 Hunger   -10 Health   +10 Happiness   +10 Energy",
  "-7 Hunger   -5 Health   +20 Happiness   +5 Energy",
  "-30 Hunger   -5 Health   +10 Happiness   +5 Energy",
  "-5 Hunger   +20 Energy",
  "+20 Hunger   -35 Health   -10 Happiness   +70 Energy",
```

```
124 ■   "-15 Hunger   -5 Health   +5 Happiness   +20 Energy",
125 ■   "-10 Hunger   -10 Health   +30 Happiness   +10 Energy",
126 ■   "-10 Hunger   -5 Health   +5 Happiness   +15 Energy",
127 ■   "+30 Hunger   -30 Health   +5 Happiness   +40 Energy",
128 ■   "-10 Hunger   -5 Health   +5 Happiness   +20 Energy"
129 ■ };
130 ■
131 ■ String[] meatItemList = {
132 ■   "Bluegill","Bass","Perch","Goldfish","Crab","Lamb Chop",
133 ■   "Pork Chop","Steak","Chicken","Catfish","Frog","Shrimp"
134 ■ };
135 ■ String[] meatNames = {
136 ■   "Bluegill","Bass","Perch","Goldfish","Crab","Lamb Chop",
137 ■   "Pork Chop","Steak","Chicken","Catfish","Frog","Shrimp"
138 ■ };
139 ■ String[] meatDesc1 = {
140 ■   "A nutritious freshwater fish with",
141 ■   "A filling fish meal packed with",
142 ■   "A lean fish choice that's healthy",
143 ■   "A small prey-sized bite that gives",
144 ■   "Rich shellfish meat with minerals",
145 ■   "A heavy red meat meal that is very",
146 ■   "A hearty cut of pork with lots of",
147 ■   "A premium meat cut that gives huge",
148 ■   "Lean poultry meat that's balanced,",
149 ■   "A dense fish meal with strong",
150 ■   "A natural prey option that feels",
151 ■   "A light seafood bite that boosts"
152 ■ };
153 ■ String[] meatDesc2 = {
154 ■   "solid protein and moderate fat.",
155 ■   "protein and natural oils.",
156 ■   "and easy to digest.",
157 ■   "a quick little meal.",
158 ■   "and lots of flavor.",
159 ■   "filling and energy dense.",
160 ■   "protein and fat.",
161 ■   "nutrition and energy.",
162 ■   "healthy, and reliable.",
163 ■   "protein and rich flavor.",
164 ■   "especially satisfying to eat.",
165 ■   "health more than fullness."
166 ■ };
167 ■ String[] meatStats = {
168 ■   "-55 Hunger   +15 Health   +10 Happiness   +20 Energy",
169 ■   "-60 Hunger   +20 Health   +10 Happiness   +25 Energy",
170 ■   "-50 Hunger   +20 Health   +5 Happiness   +15 Energy",
171 ■   "-30 Hunger   +5 Health   +15 Happiness   +10 Energy",
172 ■   "-50 Hunger   +25 Health   +20 Happiness   +15 Energy",
173 ■   "-70 Hunger   +10 Health   +15 Happiness   +35 Energy",
174 ■   "-65 Hunger   +10 Health   +10 Happiness   +30 Energy",
175 ■   "-70 Hunger   +20 Health   +20 Happiness   +40 Energy",
176 ■   "-60 Hunger   +20 Health   +10 Happiness   +20 Energy",
177 ■   "-60 Hunger   +15 Health   +10 Happiness   +25 Energy",
178 ■   "-45 Hunger   +15 Health   +25 Happiness   +20 Energy",
179 ■   "-35 Hunger   +25 Health   +15 Happiness   +10 Energy"
180 ■ };
181 ■
182 ■ float[] snackPrices = {
183 ■   4.50f, // Nachos
184 ■   3.25f, // Cheesepuffs
185 ■   2.75f, // Chips
186 ■   2.50f, // Chocolatebar
187 ■   3.00f, // Cookies
188 ■   2.25f, // Crackers
```

```
  3.75f, // Energydrink
  2.50f, // Granolabar
  2.75f, // Popcorn
  2.25f, // Pretzels
  2.00f, // Soda
  3.50f  // Trailmix
};
float[] meatPrices = {
  6.00f, // Bluegill
  7.00f, // Bass
  6.50f, // Perch
  4.50f, // Goldfish
  8.00f, // Crab
  10.00f, // Lamb Chop
  9.00f, // Pork Chop
  11.00f, // Steak
  7.50f, // Chicken
  8.50f, // Catfish
  7.00f, // Frog
  6.50f  // Shrimp
};

// Image arrays for snack/meat grid display (initialized in fileWork())
PImage[] snackImages;
PImage[] meatImages;

float[] snackDisplayScales  = {9, 8, 8, 12, 6.5f, 7, 11.5f, 7, 8, 6, 14, 8};
float[] snackDisplayOffsetY = {0, 5, 10, 0, -2, 2, 0, 10, 1.5f, 3, 0, 6};


// ========================
// indexOf — finds index of val in arr, returns -1 if not found
// ========================
int indexOf(String[] arr, String val) {
  for (int i = 0; i < arr.length; i++) if (arr[i].equals(val)) return i;
  return -1;
}


// ========================
// inventoryHasRoomFor
// Returns true if inventory has an EMPTY slot or already holds the same item.
// ========================
boolean inventoryHasRoomFor(String item) {
  for (int i = 0; i < inventorySlots.length; i++) {
    if (inventorySlots[i].equals("EMPTY") || inventorySlots[i].equals(item)) {
      return true;
    }
  }
  return false;
}


// ========================
// Item Detail Display Helpers
// ========================
void drawItemDetail(String name, String desc1, String desc2, String statLine, PImage img, float scale) {
  textAlign(CENTER);
  textSize(30);
  text(name, 770, 233);
  textSize(20);
  text(desc1, 770, 406);
  text(desc2, 770, 436);
  textSize(15);
  text(statLine, 770, 258);
```

```
  imageMode(CENTER);
  image(img, 770, 333, img.width/scale, img.height/scale);
  imageMode(CORNER);
}

void drawMedicineDetail(int idx) {
  textAlign(CENTER);
  textSize(30);
  text(medicineNames[idx], 770, 233);
  textSize(20);
  text(medicineDesc1[idx], 770, 406);
  text(medicineDesc2[idx], 770, 436);
  if (medicineIsPrescribed[idx]) text("Prescribed: Yes", 770, 258);
  else text("Prescribed: No", 770, 258);
  imageMode(CENTER);
  image(medicine, 770, 333, medicine.width/9, medicine.height/9);
  imageMode(CORNER);
}

void drawSnackDetail(int idx) {
  drawItemDetail(snackNames[idx], snackDesc1[idx], snackDesc2[idx], snackStats[idx], snackImages[idx],
snackDisplayScales[idx]);
}

void drawMeatDetail(int idx) {
  drawItemDetail(meatNames[idx], meatDesc1[idx], meatDesc2[idx], meatStats[idx], meatImages[idx], 7);
}


// =========================
// Inventory Panel
// =========================
void inventory() {
  rectMode(CORNERS);
  stroke(169);
  strokeWeight(5);
  fill(80, 220);
  rect(110, 122.5f, 990, 572.5f);
  noFill();
  rect(933, 132, 976, 171.5f);
  textAlign(LEFT, CENTER);
  textSize(40);
  fill(255);
  text("X", 942, 151.75f);
  fill(169);
  line(width/2, 182, width/2, 572.5f);
  line(110, 182, 990, 182);

  float[] xs = {110, 256.66f, 403.32f, width/2};
  float[] ys = {182, 279.625f, 377.25f, 474.875f, 572.5f};

  noFill();
  stroke(169);
  strokeWeight(5);
  strokeCap(SQUARE);

  for (int i = 0; i < xs.length; i++) {
    line(xs[i], ys[0], xs[i], ys[ys.length-1]);
  }
  for (int j = 0; j < ys.length; j++) {
    line(xs[0], ys[j], xs[xs.length-1], ys[j]);
  }

  filledSlotCount = 0;
  for (String slots : inventorySlots) {
```

```
    if (!slots.equals("EMPTY")) filledSlotCount++;
  }

  if (selectedInventorySlot >= 0 && selectedInventorySlot < inventorySlots.length) {
    if (!inventorySlots[selectedInventorySlot].equals("EMPTY")) {
      fill(0, 255, 0, 65);
      rect(585.125f, 474.875f, 760, 546);
      textAlign(CENTER);
      fill(255);
      textSize(40);
      text("FEED", 672.5625f, 522.5f);

      fill(255, 0, 0, 65);
      rect(780, 474.875f, 954.875f, 546);
      fill(255);
      text("SELL", 867.4375f, 522.5f);
    } else {
      textAlign(CENTER);
      fill(255);
      textSize(30);
      text("No Item Selected", 770, 335);
      textSize(20);
      text("Select an item or head to ", 770, 370);
      text("the store to buy one!", 770, 390);
    }
  } else {
    textAlign(CENTER);
    fill(255);
    textSize(30);
    text("No Item Selected", 770, 335);
    textSize(20);
    text("Select an item or head to ", 770, 370);
    text("the store to buy one!", 770, 390);
  }

  if (selectedInventorySlot >= 0 && selectedInventorySlot < inventorySlots.length) {
    int r = selectedInventorySlot / 3;
    int c = selectedInventorySlot % 3;

    float x1 = xs[c], x2 = xs[c+1];
    float y1 = ys[r], y2 = ys[r+1];

    float o = 1;
    stroke(255);

    line(x1 - o, y1,     x2 + o, y1);
    line(x1 - o, y2,     x2 + o, y2);
    line(x1,     y1 - o, x1,     y2 + o);
    line(x2,     y1 - o, x2,     y2 + o);
  }

  rectMode(CORNER);
  strokeWeight(1);
  textAlign(CENTER);
  fill(255);
  textSize(35);
  text("INVENTORY:", width/2, height*0.235f);

  imageMode(CENTER);

  for (int i = 0; i < inventorySlots.length; i++) {
    if (!inventorySlots[i].equals("EMPTY")) {
      int row = i / 3;
      int col = i % 3;

```

```
      float slotX1 = xs[col];
      float slotX2 = xs[col + 1];
      float slotY1 = ys[row];
      float slotY2 = ys[row + 1];

      float steakX = slotX1 + 70;
      float steakY = slotY1 + 51;
      float qtyX   = slotX1 + 110;
      float qtyY   = slotY1 + 86;

      float medX = (slotX1 + slotX2) / 2;
      float medY = (slotY1 + slotY2) / 2;

      String slotItem = inventorySlots[i];
      int medIdx   = indexOf(medicineItemList, slotItem);
      int snackIdx = indexOf(snackItemList, slotItem);
      int meatIdx  = indexOf(meatItemList, slotItem);

      if (slotItem.equals("Steak") && !hasFedSteak) {
        image(steak, steakX, steakY, steak.width/7, steak.height/7);
        textSize(30);
        fill(255);
        text("x1", qtyX, qtyY);

      } else if (medIdx >= 0) {
        image(medicine, medX, medY, medicine.width/14, medicine.height/14);
        fill(255);
        textSize(30);
        text("x" + medicineQuantities[medIdx], qtyX, qtyY);

      } else if (snackIdx >= 0) {
        image(snackImages[snackIdx], medX, medY + snackDisplayOffsetY[snackIdx],
              snackImages[snackIdx].width/snackDisplayScales[snackIdx],
              snackImages[snackIdx].height/snackDisplayScales[snackIdx]);
        fill(255);
        textSize(30);
        text("x" + snackQuantities[snackIdx], qtyX, qtyY);

      } else if (meatIdx >= 0) {
        image(meatImages[meatIdx], medX, medY,
              meatImages[meatIdx].width/8, meatImages[meatIdx].height/8);
        fill(255);
        textSize(30);
        text("x" + meatQuantities[meatIdx], qtyX, qtyY);
      }
    }
  }

  if (selectedInventorySlot != -1 && selectedInventorySlot < inventorySlots.length) {
    String selectedItem = inventorySlots[selectedInventorySlot];

    textAlign(CENTER);
    fill(255);

    if (selectedItem.equals("Steak") && !hasFedSteak) {
      textSize(30);
      text("Steak", 770, 233);
      textSize(20);
      text("The dream of any pet, having", 770, 406);
      text("no potential negative effects.", 770, 436);
      textSize(14);
      text("-70 Hunger   +40 Energy   +20 Health   +20 Happiness", 770, 258);
      textSize(20);
      imageMode(CENTER);
      image(steak, 770, 333, steak.width/4, steak.height/4);
```

```
      imageMode(CORNER);

    } else {
      int selMedIdx   = indexOf(medicineItemList, selectedItem);
      int selSnackIdx = indexOf(snackItemList, selectedItem);
      int selMeatIdx  = indexOf(meatItemList, selectedItem);

      if (selMedIdx >= 0) {
        drawMedicineDetail(selMedIdx);
      } else if (selSnackIdx >= 0) {
        drawSnackDetail(selSnackIdx);
      } else if (selMeatIdx >= 0) {
        drawMeatDetail(selMeatIdx);
      }
    }
  }

  if (isShowingCantSell == true) cantsell();
  stroke(0);
}

void cantsell() {
  imageMode(CENTER);
  image(popupbackground, width/2, height*0.42f, popupbackground.width*0.6f, popupbackground.height*0.6f);
  fill(0);
  textFont(times50);
  textSize(20);
  drawWrappedTextInBox("Careful! Selling your steak could leave " + alligator.petName + " starving, and that can be deadly. From here on out, financial decisions are yours. Choose wisely!", 338, 271, 761, 400, 6);
}
```

§ 9

G_Store.pde

# Store Panel

Medicine, snacks and meat tabs — selection, purchase, and store fade transitions

```
// ========================
// G_Store.pde
// Store panel rendering, tab display (medicine / snacks / meat), and store fade logic.
// ========================


// ========================
// Store State
// ========================
boolean isStoreOpen = false;
boolean isStoreMainScreenFading = false;  // Phase 1: main screen fades to black before store opens
boolean isStoreEntryFading = false;       // Phase 2: fade from black to reveal store
boolean isStoreFadingOut = false;         // True while store fades to black on exit
boolean isViewingMedicineTab = false;
boolean isViewingSnacksTab = false;
boolean isViewingMeatTab = false;
boolean isMedicineBought = false;
boolean hasClickedStoreExit = false;
boolean canAffordMedicine;
boolean isShowingStoreClosedPopup = false;

PImage storebackground;
PImage medicine;
PImage nachos, cheesepuffs, chips, chocolatebar, cookies, crackers;
PImage energydrink, granolabar, popcorn, pretzels, soda, trailmix;
PImage chicken, catfish, frog, shrimp;

Fade storeFade = new Fade(255);


// ========================
// Store Panel Shared Helpers
// ========================
void drawStorePanelFrame(String title) {
  rectMode(CORNERS);
  stroke(169);
  strokeWeight(5);
  fill(80, 220);
  rect(110, 122.5f, 990, 572.5f);
  noFill();
  rect(933, 132, 976, 171.5f);
  textAlign(LEFT, CENTER);
  textSize(40);
  fill(255);
  text("X", 942, 151.75f);
  fill(169);
  line(width/2, 182, width/2, 572.5f);
  line(110, 182, 990, 182);
  rectMode(CORNER);
  strokeWeight(1);
  textAlign(CENTER);
  fill(255);
  textSize(35);
  text(title, width/2, height*0.235f);
}

void drawStoreGrid(float[] xs, float[] ys) {
  noFill();
```

```
  stroke(169);
  strokeWeight(5);
  strokeCap(SQUARE);
  for (int i = 0; i < xs.length; i++) line(xs[i], ys[0], xs[i], ys[ys.length-1]);
  for (int j = 0; j < ys.length; j++) line(xs[0], ys[j], xs[xs.length-1], ys[j]);
}


// ========================
// Store Panel
// ========================
void store() {
  textSize(30);

  String moneyText = "$" + String.format("%03.2f", money);
  float padding = 20;
  float boxW = textWidth(moneyText) + padding * 2;
  float boxH = 53;
  float boxLeft = 0;
  float boxTop = 631;
  float boxRight = boxLeft + boxW;
  float boxBottom = boxTop + boxH;

  float exitPadding = 20;
  float exitW = textWidth("EXIT") + exitPadding * 2;
  float exitRight = width;
  float exitLeft = exitRight - exitW;
  float exitTop = 631;
  float exitBottom = exitTop + boxH;

  imageMode(CENTER);
  image(storebackground, width/2, height/2, width, height);

  rectMode(CORNERS);
  fill(129, 86, 0);
  stroke(201, 178, 2);
  strokeWeight(5);
  rect(boxLeft, boxTop, boxRight, boxBottom);
  rect(exitLeft, exitTop, exitRight, exitBottom);

  fill(255);
  textAlign(LEFT, CENTER);
  text(moneyText, boxLeft + padding, height * 0.939f);

  textAlign(RIGHT, CENTER);
  text("EXIT", exitRight - exitPadding, height * 0.939f);

  stroke(0);
  strokeWeight(2);
  if (!hasFirstBoughtMedicine && !isViewingMedicineTab && !isViewingSnacksTab && !isViewingMeatTab)
    redarrow(545, 59, "right");

  if (isViewingMedicineTab) {
    drawStorePanelFrame("MEDICINE:");
    float[] xs = {110, 256.66f, 403.32f, width/2};
    float[] ys = {182, 279.625f, 377.25f, 474.875f, 572.5f};
    drawStoreGrid(xs, ys);
    fill(169);
    line(width/2, 182, width/2, 572.5f);
    line(110, 182, 990, 182);
    canAffordMedicine = inventoryHasRoomFor(medicineItemList[selectedInventorySlot]) &&
      (((!medicineIsPrescribed[selectedInventorySlot] && money >= 5) ||
medicineIsPrescribed[selectedInventorySlot]));
    rectMode(CORNERS);

```

```
    if (selectedInventorySlot != -1) {
      if ((!medicineIsPrescribed[selectedInventorySlot] && money >= 5) ||
medicineIsPrescribed[selectedInventorySlot]) && canAffordMedicine) {
        fill(0, 255, 0, 65);
      } else {
        fill(169, 80);
      }
      rect(682.5625f, 474.875f, 857.4375f, 546);
      textAlign(CENTER);
      fill(255);
      textSize(25);

      if (selectedInventorySlot >= 0 && selectedInventorySlot < medicineIsPrescribed.length &&
medicineIsPrescribed[selectedInventorySlot]) {
        text("$5.00   $0.00", 770, 522.5f);
        stroke(255, 0, 0);
        line(692, 514, 769, 514);
      } else {
        text("$5.00", 770, 522.5f);
      }

      int r = selectedInventorySlot / 3;
      int c = selectedInventorySlot % 3;
      float x1 = xs[c], x2 = xs[c+1];
      float y1 = ys[r], y2 = ys[r+1];
      float o = 1;
      stroke(255);
      line(x1 - o, y1, x2 + o, y1);
      line(x1 - o, y2, x2 + o, y2);
      line(x1, y1 - o, x1, y2 + o);
      line(x2, y1 - o, x2, y2 + o);

    } else {
      textAlign(CENTER);
      fill(255);
      textSize(30);
      text("No Item Selected", 770, 335);
      textSize(20);
      text("Select an item", 770, 370);
      text("and purchase it!", 770, 390);
    }

    imageMode(CENTER);
    for (int row = 0; row < 4; row++) {
      for (int col = 0; col < 3; col++) {
        float x1 = xs[col], x2 = xs[col + 1];
        float y1 = ys[row], y2 = ys[row + 1];
        image(medicine, (x1+x2)/2, (y1+y2)/2, medicine.width/14, medicine.height/14);
      }
    }

    if (selectedInventorySlot >= 0 && selectedInventorySlot < medicineItemList.length) {
      drawMedicineDetail(selectedInventorySlot);
    }
  }

  if (isViewingSnacksTab) {
    drawStorePanelFrame("SNACKS:");
    float[] xs = {110, 256.66f, 403.32f, width/2};
    float[] ys = {182, 279.625f, 377.25f, 474.875f, 572.5f};
    drawStoreGrid(xs, ys);
    fill(169);
    line(width/2, 182, width/2, 572.5f);
    line(110, 182, 990, 182);
    rectMode(CORNERS);
```

```
186 ■
187 ■     if (selectedInventorySlot != -1) {
188 ■       if (inventoryHasRoomFor(snackItemList[selectedInventorySlot]) && money >=
snackPrices[selectedInventorySlot])
189 ■         fill(0, 255, 0, 65);
190 ■       else fill(169, 80);
191 ■       rect(682.5625f, 474.875f, 857.4375f, 546);
192 ■       textAlign(CENTER);
193 ■       fill(255);
194 ■       textSize(25);
195 ■       text("$" + String.format("%03.2f", snackPrices[selectedInventorySlot]), 770, 522.5f);
196 ■
197 ■       int r = selectedInventorySlot / 3;
198 ■       int c = selectedInventorySlot % 3;
199 ■       float x1 = xs[c], x2 = xs[c+1];
200 ■       float y1 = ys[r], y2 = ys[r+1];
201 ■       float o = 1;
202 ■       stroke(255);
203 ■       line(x1 - o, y1, x2 + o, y1);
204 ■       line(x1 - o, y2, x2 + o, y2);
205 ■       line(x1, y1 - o, x1, y2 + o);
206 ■       line(x2, y1 - o, x2, y2 + o);
207 ■
208 ■     } else {
209 ■       textAlign(CENTER);
210 ■       fill(255);
211 ■       textSize(30);
212 ■       text("No Item Selected", 770, 335);
213 ■       textSize(20);
214 ■       text("Select an item", 770, 370);
215 ■       text("and purchase it!", 770, 390);
216 ■     }
217 ■
218 ■     imageMode(CENTER);
219 ■     for (int row = 0; row < 4; row++) {
220 ■       for (int col = 0; col < 3; col++) {
221 ■         int snackIndex = row * 3 + col;
222 ■         float x1 = xs[col], x2 = xs[col + 1];
223 ■         float y1 = ys[row], y2 = ys[row + 1];
224 ■         image(snackImages[snackIndex], (x1+x2)/2, (y1+y2)/2 + snackDisplayOffsetY[snackIndex],
225 ■               snackImages[snackIndex].width/snackDisplayScales[snackIndex],
226 ■               snackImages[snackIndex].height/snackDisplayScales[snackIndex]);
227 ■       }
228 ■     }
229 ■
230 ■     if (selectedInventorySlot >= 0 && selectedInventorySlot < snackItemList.length) {
231 ■       drawSnackDetail(selectedInventorySlot);
232 ■     }
233 ■   }
234 ■
235 ■   if (isViewingMeatTab) {
236 ■     drawStorePanelFrame("MEAT:");
237 ■     float[] xs = {110, 256.66f, 403.32f, width/2};
238 ■     float[] ys = {182, 279.625f, 377.25f, 474.875f, 572.5f};
239 ■     drawStoreGrid(xs, ys);
240 ■     fill(169);
241 ■     line(width/2, 182, width/2, 572.5f);
242 ■     line(110, 182, 990, 182);
243 ■     rectMode(CORNERS);
244 ■
245 ■     if (selectedInventorySlot != -1) {
246 ■       if (inventoryHasRoomFor(meatItemList[selectedInventorySlot]) && money >=
meatPrices[selectedInventorySlot])
247 ■         fill(0, 255, 0, 65);
248 ■       else fill(169, 80);
```

```
      rect(682.5625f, 474.875f, 857.4375f, 546);
      textAlign(CENTER);
      fill(255);
      textSize(25);
      text("$" + String.format("%03.2f", meatPrices[selectedInventorySlot]), 770, 522.5f);

      int r = selectedInventorySlot / 3;
      int c = selectedInventorySlot % 3;
      float x1 = xs[c], x2 = xs[c+1];
      float y1 = ys[r], y2 = ys[r+1];
      float o = 1;
      stroke(255);
      line(x1 - o, y1, x2 + o, y1);
      line(x1 - o, y2, x2 + o, y2);
      line(x1, y1 - o, x1, y2 + o);
      line(x2, y1 - o, x2, y2 + o);

    } else {
      textAlign(CENTER);
      fill(255);
      textSize(30);
      text("No Item Selected", 770, 335);
      textSize(20);
      text("Select an item", 770, 370);
      text("and purchase it!", 770, 390);
    }

    imageMode(CENTER);
    for (int row = 0; row < 4; row++) {
      for (int col = 0; col < 3; col++) {
        int meatIndex = row * 3 + col;
        float x1 = xs[col], x2 = xs[col + 1];
        float y1 = ys[row], y2 = ys[row + 1];
        image(meatImages[meatIndex], (x1+x2)/2, (y1+y2)/2,
              meatImages[meatIndex].width/8, meatImages[meatIndex].height/8);
      }
    }

    if (selectedInventorySlot >= 0 && selectedInventorySlot < meatItemList.length) {
      drawMeatDetail(selectedInventorySlot);
    }
  }

  if (hasFirstBoughtMedicine && !hasClickedStoreExit) redarrow(919, 657, "right");
  rectMode(CORNER);

  // Store fade overlay
  if (isStoreFadingOut) {
    if (storeFade.stepOut(8)) {
      isStoreOpen = false;
      isStoreFadingOut = false;
      isOnMainScreen = true;
      storeFade.setClear();
    }
  } else {
    storeFade.stepIn(8);
  }
  storeFade.draw();
}

void storeclosedpopup() {
  imageMode(CENTER);
  image(popupbackground, width/2, height*0.42f, popupbackground.width*0.6f, popupbackground.height*0.6f);
  fill(0);
  textFont(times50);
```

```
  textSize(20);
  drawWrappedTextInBox("The store is closed on Sundays (days that are multiples of 7). Come back tomorrow!", 338, 271, 761, 400, 6);
}
```

§ 10

G_Bank.pde

# Bank Panel

Transaction log, scrollable viewport, personalized financial advice system

```
// ========================
// G_Bank.pde
// Bank panel rendering, transaction log display, and financial advice.
// ========================


// ========================
// Bank State
// ========================
boolean isBankOpen = false;
boolean hasViewedBank = false;
boolean hasShownBankPopup = false;
boolean isShowingBankPopup = false;
boolean hasViewedBankFirstTime = false;

ArrayList<String> bankTransactionLog = new ArrayList<String>();
int bankTransactionsLoggedCount = 0;

float bankScrollOffset = 0;
float bankScrollContentHeight = 0;
float bankViewportX = 320;
float bankViewportY = 190;
float bankViewportWidth = 450;
float bankViewportHeight = 370;
float bankItemLineHeight = 30;
float bankScrollbarX = 770;
float bankScrollbarY = 190;
float bankScrollbarWidth = 12;
float bankScrollbarHeight = 370;
boolean isDraggingBankScrollbar = false;
float bankScrollThumbOffsetY = 0;


// ========================
// Bank Popup
// ========================
void bankpopup() {
  imageMode(CENTER);
  image(popupbackground, width/2, height*0.42f, popupbackground.width*0.6f, popupbackground.height*0.6f);
  fill(0);
  textFont(times50);
  textSize(20);
  drawWrappedTextInBox("Don't forget to give " + alligator.petName + " another dose tomorrow (see prescription description). Click Bank after closing this window to view your transactions and get personalized financial advice.", 338, 271, 761, 400, 6);
}


// ========================
// Bank Panel
// ========================
void bank() {
  rectMode(CORNERS);
  stroke(169);
  strokeWeight(5);
```

```
  fill(80, 220);
  rect(310, 122.5f, 790, 572.5f);
  line(310, 182, 790, 182);

  textAlign(CENTER);
  fill(255);
  textSize(35);
  text("BANK:", width/2, height*0.235f);

  bankScrollContentHeight = 170 + bankTransactionLog.size() * bankItemLineHeight;
  float maxScroll = max(0, bankScrollContentHeight - bankViewportHeight);
  bankScrollOffset = constrain(bankScrollOffset, 0, maxScroll);

  pushMatrix();
  clip((int)bankViewportX, (int)bankViewportY, (int)bankViewportWidth, (int)bankViewportHeight);

  fill(255);
  textAlign(CENTER, TOP);
  textSize(25);
  text("Advice:", bankViewportX + bankViewportWidth/2, bankViewportY + 10 - bankScrollOffset);

  line(bankViewportX, bankViewportY + 92 - bankScrollOffset, bankViewportX + bankViewportWidth, bankViewportY + 92 - bankScrollOffset);

  textSize(13.75f);
  fill(255);
  if (hasNeverBoughtHighQualityCare) {
    drawWrappedTextInBox(
      "You have only purchased low quality care. While it's cheaper, the vet can prescribe an incorrect medication or refuse to help even after you pay.",
      bankViewportX + 10, bankViewportY + 38 - bankScrollOffset,
      bankViewportX + bankViewportWidth - 10, bankViewportY + 86 - bankScrollOffset, 2);
  } else if (!hasCleanerVisited) {
    drawWrappedTextInBox(
      "Hiring a cleaner daily will ensure that " + alligator.petName + "'s risk of sickness won't increase daily.",
      bankViewportX + 10, bankViewportY + 38 - bankScrollOffset,
      bankViewportX + bankViewportWidth - 10, bankViewportY + 86 - bankScrollOffset, 2);
  } else if (timesFedPet < 5) {
    drawWrappedTextInBox(
      "Keep an eye on hunger and energy. Feeding " + alligator.petName + " regularly will help keep hunger from getting to high and energy from getting too low.",
      bankViewportX + 10, bankViewportY + 38 - bankScrollOffset,
      bankViewportX + bankViewportWidth - 10, bankViewportY + 86 - bankScrollOffset, 2);
  } else if (timesRestedSuccessfully < 3) {
    drawWrappedTextInBox(
      "Resting is a good way to restore energy, but make sure these rests are successful and the line lands in the COLOR_GREEN zone.",
      bankViewportX + 10, bankViewportY + 38 - bankScrollOffset,
      bankViewportX + bankViewportWidth - 10, bankViewportY + 86 - bankScrollOffset, 2);
  } else if (helpTaskCount < 3) {
    drawWrappedTextInBox(
      "Helping around town is a good way to earn money, but leaving " + alligator.petName + " alone can have consequences.",
      bankViewportX + 10, bankViewportY + 38 - bankScrollOffset,
      bankViewportX + bankViewportWidth - 10, bankViewportY + 86 - bankScrollOffset, 2);
  } else if (money < 15) {
    drawWrappedTextInBox(
      "Your money is getting low. Consider working, helping around town, or playing minigames to earn more.",
      bankViewportX + 10, bankViewportY + 38 - bankScrollOffset,
      bankViewportX + bankViewportWidth - 10, bankViewportY + 86 - bankScrollOffset, 2);
  } else if (alligator.sickrisk >= 60) {
    drawWrappedTextInBox(
      alligator.petName + "'s sickness risk is pretty high. Cleaning and careful care can help prevent future problems.",
```

```
      bankViewportX + 10, bankViewportY + 38 - bankScrollOffset,
      bankViewportX + bankViewportWidth - 10, bankViewportY + 86 - bankScrollOffset, 2);
  } else if (alligator.health <= 40) {
    drawWrappedTextInBox(
      alligator.petName + "'s health is getting low. You should focus on recovery before it becomes dangerous.",
      bankViewportX + 10, bankViewportY + 38 - bankScrollOffset,
      bankViewportX + bankViewportWidth - 10, bankViewportY + 86 - bankScrollOffset, 2);
  } else if (alligator.hunger >= 50) {
    drawWrappedTextInBox(
      alligator.petName + " is getting hungry. Make sure to keep food stocked in your inventory.",
      bankViewportX + 10, bankViewportY + 38 - bankScrollOffset,
      bankViewportX + bankViewportWidth - 10, bankViewportY + 86 - bankScrollOffset, 2);
  } else {
    drawWrappedTextInBox(
      "Keep balancing money and care to keep " + alligator.petName + " healthy, happy, and alive.",
      bankViewportX + 10, bankViewportY + 38 - bankScrollOffset,
      bankViewportX + bankViewportWidth - 10, bankViewportY + 86 - bankScrollOffset, 2);
  }

  textAlign(LEFT, TOP);
  textSize(20);
  for (int i = 0; i < bankTransactionLog.size(); i++) {
    float y = bankViewportY + 110 + i * bankItemLineHeight - bankScrollOffset;
    text(bankTransactionLog.get(i), bankViewportX + 10, y);
  }

  noClip();
  popMatrix();

  noFill();
  stroke(200);
  strokeWeight(2);
  rectMode(CORNER);
  rect(bankViewportX, bankViewportY, bankViewportWidth, bankViewportHeight);

  fill(120);
  stroke(80);
  rect(bankScrollbarX, bankScrollbarY, bankScrollbarWidth, bankScrollbarHeight);

  if (bankScrollContentHeight > bankViewportHeight) {
    float thumbH = max(40, (bankViewportHeight / bankScrollContentHeight) * bankScrollbarHeight);
    float thumbY = map(bankScrollOffset, 0, maxScroll, bankScrollbarY, bankScrollbarY + bankScrollbarHeight - thumbH);
    fill(220);
    stroke(100);
    rect(bankScrollbarX, thumbY, bankScrollbarWidth, thumbH);
  } else {
    fill(220);
    stroke(100);
    rect(bankScrollbarX, bankScrollbarY, bankScrollbarWidth, bankScrollbarHeight);
  }

  noFill();
  stroke(169);
  rectMode(CORNERS);
  rect(733, 132, 776, 171.5f);
  textAlign(LEFT, CENTER);
  textSize(40);
  fill(255);
  text("X", 742, 151.75f);

  strokeWeight(2);
  stroke(0);
  rectMode(CORNER);
```

```
175 ■ }
```

§ 1 1

G_Earn.pde

# Economy & Jobs

Earn panel, job finder (cashier/barista/manager), tasks and upgrade system

```
// ========================
// G_Earn.pde
// Earn panel: job display, Job Finder, Tasks & Upgrades, job/help popups.
// ========================


// ========================
// Earn / Job State
// ========================
String job = "unemployed";
float salary = 0;
float totalJobEarnings = 0;
float taskRewardAmount = 5;
float pointUpgradeCost = 3;
float salaryUpgradeCost = 3;
float taskUpgradeCost = 3;
boolean hasCashierMaxedSalary = false;

boolean isEarnPanelOpen = false;
boolean hasOpenedEarnPanel = false;
boolean isShowingEarnPopup = false;
boolean hasShownEarnPopup = false;
int earnPopupDelayTimer = 0;

boolean isJobFinderOpen = false;
boolean isTasksPanelOpen = false;
boolean hasShownJobPopup = false;
boolean isShowingJobPopup = false;
boolean isHelpTaskPending = false;
boolean hasUsedHelpTask = false;
boolean hasClickedTaskTab = false;
boolean hasShownFirstHelpPopup = false;
boolean isShowingFirstHelpPopup = false;
String helpPopupMessage = "";

float moneyEarnedFromSalary = 0;
float moneyEarnedFromTasks = 0;
int salaryUpgradeCount = 0;
int taskUpgradeCount = 0;
int helpTaskCount = 0;

PImage unemployed, cashier, barista, manager;
PImage town, cash, house, lock;


// ========================
// Earn Popup
// ========================
void earnpopup() {
  imageMode(CENTER);
  image(popupbackground, width/2, height*0.42f, popupbackground.width*0.6f, popupbackground.height*0.6f);
  fill(0);
  textFont(times50);
  textSize(20);
```

```
  drawWrappedTextInBox("Nice job stabilizing " + alligator.petName + "'s energy. As an owner, playing can get annoying, so you can use services to help with their needs! But services cost money. Click the earn button after closing this window!", 338, 271, 761, 400, 6);
}

void jobpopup() {
  imageMode(CENTER);
  image(popupbackground, width/2, height*0.42f, popupbackground.width*0.6f, popupbackground.height*0.6f);
  fill(0);
  textFont(times50);
  textSize(20);
  drawWrappedTextInBox("Congrats on becoming a cashier! You now earn a daily salary. In this tab you can upgrade it, earn money when " + alligator.petName + " plays, or help around town for extra cash. Close this window and try helping out!", 338, 271, 761, 400, 6);
}

void help() {
  imageMode(CENTER);
  image(popupbackground, width/2, height*0.42f, popupbackground.width*0.6f, popupbackground.height*0.6f);
  fill(0);
  textFont(times50);
  textSize(20);
  drawWrappedTextInBox(helpPopupMessage, 338, 271, 761, 400, 6);
}


// ========================
// Earn Panel (main view)
// ========================
void earn() {
  if (isJobFinderOpen) {
    earnJobFinder();
    return;
  }
  if (isTasksPanelOpen) {
    earnTasksUpgrades();
    return;
  }

  rectMode(CORNERS);
  stroke(169);
  strokeWeight(5);
  fill(80, 220);
  rect(110, 122.5f, 990, 572.5f);

  if (!job.equals("unemployed")) fill(255, 0, 0, 65);

  rectMode(CENTER);
  rect(816, 206, 100, 100);

  textAlign(CENTER);
  fill(255);
  line(110, 275, 990, 275);

  textSize(40);
  text("QUIT", 816, 200);
  text("JOB", 816, 240);

  textSize(25);
  text("CURRENT JOB:", width/2, 164.5f);

  imageMode(CENTER);

  if (job.equals("unemployed")) {
    image(unemployed, 466, 230, unemployed.width/7, unemployed.height/7);
```

```
    textSize(25);
    text("Unemployed", 580, 193);
    textAlign(CORNER);
    text("Job Info:", 129, 165);
    textSize(15);
    text("Current Salary: $0/day", 129, 187);
    text("Next Paycheck: N/A", 129, 209);
    text("Total Job Earnings: N/A", 129, 231);
    if (day >= 25)      text("Next Job: Manager is available!", 129, 253);
    else if (day >= 10) text("Next Job: Barista is available!", 129, 253);
    else                text("Next Job: Cashier is available!", 129, 253);
    textAlign(CENTER);
    text("There's not much to", 580, 223);
    text("this job's description.", 580, 238);
    text("Get a job below.", 580, 253);

  } else if (job.equals("cashier")) {
    image(cashier, 466, 230, unemployed.width/7, unemployed.height/7);
    textSize(25);
    text("Cashier", 580, 193);
    textAlign(CORNER);
    text("Job Info:", 129, 165);
    textSize(15);
    text("Current Salary: $" + nf(salary, 0, 2) + "/day", 129, 187);
    text("Next Paycheck: Day " + (day+1), 129, 209);
    text("Total Job Earnings: $" + nf(totalJobEarnings, 0, 2), 129, 231);
    if (day < 10)      text("Next Job: Barista on Day 10", 129, 253);
    else if (day < 25) text("Next Job: Barista is available!", 129, 253);
    else               text("Next Job: Cafe Manager is available!", 129, 253);
    textAlign(CENTER);
    text("Work the", 580, 223);
    text("register for", 580, 238);
    text(alligator.petName + "!", 580, 253);

  } else if (job.equals("barista")) {
    image(barista, 466, 230, barista.width/3, barista.height/3);
    textSize(25);
    text("Barista", 580, 193);
    textAlign(CORNER);
    text("Job Info:", 129, 165);
    textSize(15);
    text("Current Salary: $" + nf(salary, 0, 2) + "/day", 129, 187);
    text("Next Paycheck: Day " + (day+1), 129, 209);
    text("Total Job Earnings: $" + nf(totalJobEarnings, 0, 2), 129, 231);
    if (day < 25) text("Next Job: Cafe Manager on Day 25", 129, 253);
    else          text("Next Job: Cafe Manager is available!", 129, 253);
    textAlign(CENTER);
    text("Make coffee", 580, 223);
    text("and earn $ for", 580, 238);
    text(alligator.petName + "!", 580, 253);

  } else {
    image(manager, 466, 230, barista.width/3, barista.height/3);
    textSize(25);
    text("Cafe Manager", 580, 193);
    textAlign(CORNER);
    text("Job Info:", 129, 165);
    textSize(15);
    text("Current Salary: $" + nf(salary, 0, 2) + "/day", 129, 187);
    text("Next Paycheck: Day " + (day+1), 129, 209);
    text("Total Job Earnings: $" + nf(totalJobEarnings, 0, 2), 129, 231);
    text("Next Job: N/A", 129, 253);
    textAlign(CENTER);
    text("Make coffee", 580, 223);
    text("and earn $ for", 580, 238);
```

```
    text(alligator.petName + "!", 580, 253);
  }

  textAlign(CENTER, CENTER);
  textSize(28);
  text("Choose a section:", width/2, 345);

  stroke(255);
  strokeWeight(2);
  fill(80, 220);
  rect(330, 455, 250, 90);
  rect(770, 455, 320, 90);

  fill(255);
  textSize(26);
  text("JOB FINDER", 330, 448);
  text("TASKS & UPGRADES", 770, 448);

  textSize(14);
  text("Apply for jobs and view unlocks", 330, 482);
  text("Help around town and buy upgrades", 770, 482);

  noFill();
  rectMode(CORNERS);
  stroke(169);
  rect(933, 132, 976, 171.5f);
  textAlign(LEFT, CENTER);
  textSize(40);
  fill(255);
  text("X", 942, 151.75f);

  rectMode(CORNER);
  stroke(0);
}


// ========================
// Job Finder Sub-Panel
// ========================
void earnJobFinder() {
  rectMode(CORNERS);
  stroke(169);
  strokeWeight(5);
  fill(80, 220);
  rect(180, 145, 920, 518);
  line(180, 205, 920, 205);

  textAlign(CENTER, CENTER);
  fill(255);
  textSize(35);
  text("JOB FINDER:", 550, 175);

  noFill();
  rect(863, 155, 906, 194.5f);
  textAlign(LEFT, CENTER);
  textSize(40);
  fill(255);
  text("X", 872, 174.75f);

  imageMode(CENTER);
  textAlign(CENTER, CENTER);
  textSize(22);
  text("Jobs:", 550, 235);

  image(cashier, 300, 287, cashier.width/4, cashier.height/4);
```

```
  image(barista, 550, 287, barista.width/4, barista.height/4);
  image(manager, 800, 287, manager.width/4, manager.height/4);

  textSize(17.5f);
  text("Cashier", 300, 330);
  text("Barista", 550, 330);
  text("Cafe Manager", 800, 330);

  textSize(12);
  text("Work the register", 300, 347);
  text("and help customers", 300, 360);
  text("check out.", 300, 373);
  text("Base Salary:", 300, 397);
  text("$15/day", 300, 410);
  text("Status:", 300, 434);
  text("Unlocked", 300, 447);

  text("Prepare coffee", 550, 347);
  text("and drinks for", 550, 360);
  text("cafe customers.", 550, 373);
  text("Base Salary:", 550, 397);
  text("$35/day", 550, 410);
  text("Status:", 550, 434);
  text("Unlocks on day 10", 550, 447);

  text("Run the cafe,", 800, 347);
  text("manage staff,", 800, 360);
  text("ensure efficiency.", 800, 373);
  text("Base Salary:", 800, 397);
  text("$75/day", 800, 410);
  text("Status:", 800, 434);
  text("Unlocks on day 25", 800, 447);

  rectMode(CENTER);
  stroke(255);
  strokeWeight(2);
  textSize(17);

  if (job.equals("unemployed")) fill(0, 255, 0, 80);
  else fill(80, 220);
  rect(300, 487, 80, 20);
  fill(255);
  text("APPLY", 300, 487);

  if (job.equals("unemployed") && day >= 10) fill(0, 255, 0, 80);
  else {
    fill(80, 220);
    if (day < 10) image(lock, 550, 287, lock.width/12, lock.height/12);
  }
  rect(550, 487, 80, 20);
  fill(255);
  text("APPLY", 550, 487);

  if (job.equals("unemployed") && day >= 25) fill(0, 255, 0, 80);
  else {
    fill(80, 220);
    if (day < 25) image(lock, 800, 287, lock.width/12, lock.height/12);
  }
  rect(800, 487, 80, 20);
  fill(255);
  text("APPLY", 800, 487);

  if (job.equals("unemployed")) redarrow(216, 482, "right");

  rectMode(CORNER);
```

```
  stroke(0);
}


// ========================
// Tasks & Upgrades Sub-Panel
// ========================
void earnTasksUpgrades() {
  rectMode(CORNERS);
  stroke(169);
  strokeWeight(5);
  fill(80, 220);
  rect(210, 145, 890, 470);
  line(210, 205, 890, 205);
  line(550, 205, 550, 470);

  textAlign(CENTER, CENTER);
  fill(255);
  textSize(35);
  text("TASKS & UPGRADES:", 550, 175);

  noFill();
  rect(833, 155, 876, 194.5f);
  textAlign(LEFT, CENTER);
  textSize(40);
  fill(255);
  text("X", 842, 174.75f);

  imageMode(CENTER);
  textAlign(CENTER, CENTER);
  textSize(25);
  text("Help Around Town:", 380, 240);
  text("Upgrades:", 720, 240);

  image(town, 380, 320, town.width/6, town.height/6);

  textSize(12);
  text("Run community errands for", 380, 362);
  text("quick cash with the risk", 380, 375);
  text("of your alligator's stats", 380, 388);
  text("changing due to your absence.", 380, 401);

  rectMode(CENTER);
  fill(255, 127, 0);
  stroke(255);
  strokeWeight(2);
  rect(380, 430, 80, 20);
  fill(255);
  textSize(17);
  text("HELP", 380, 430);

  pushMatrix();
  translate(485, 277);
  rotate(HALF_PI);
  image(topalligator1, 0, -topalligator1.width/12, topalligator1.width/12, topalligator1.height/12);
  popMatrix();

  image(cash, 720, 277, cash.width/13, cash.height/13);
  image(house, 825, 272, house.width/17, house.height/17);

  textSize(14);
  text("$ per Point", 615, 307);
  text("Salary", 720, 307);
  text("$ Per Town Task", 825, 307);

```

```
  textSize(10);
  text("Current:", 615, 320);
  text("Current:", 720, 320);
  text("Current:", 825, 320);
  text("$" + nf(moneyPerMinigamePoint, 0, 2) + "/pt", 615, 333);
  text("$" + nf(salary, 0, 2) + "/day", 720, 333);
  text("$" + nf(taskRewardAmount, 0, 2) + "/task", 825, 333);

  text("Upgrade:", 615, 359);
  if (moneyPerMinigamePoint == 0) text("$0.10/pt", 615, 372);
  else text("$" + nf(moneyPerMinigamePoint*1.2f, 0, 2) + "/pt", 615, 372);

  text("Upgrade:", 720, 359);
  if (job.equals("cashier")) {
    if (salary >= 32 || salary * 1.12f >= 32) text("MAXED", 720, 372);
    else text("$" + nf(salary * 1.12f, 0, 2) + "/day", 720, 372);
  } else if (job.equals("barista")) {
    if (salary >= 70 || salary * 1.12f >= 70) text("MAXED", 720, 372);
    else text("$" + nf(salary * 1.12f, 0, 2) + "/day", 720, 372);
  } else {
    text("$" + nf(salary * 1.12f, 0, 2) + "/day", 720, 372);
  }

  text("BUY:", 615, 402);
  text("BUY:", 720, 402);
  text("BUY:", 825, 402);

  text("Upgrade:", 825, 359);
  text("$" + nf(taskRewardAmount*1.12f, 0, 2) + "/task", 825, 372);

  textSize(13);
  if (money >= pointUpgradeCost) fill(0, 255, 0, 80);
  else fill(80, 220);
  rect(615, 420, 80, 20);
  fill(255);
  text("$" + nf(pointUpgradeCost, 0, 2), 615, 420);

  if (!job.equals("unemployed") && money >= salaryUpgradeCost &&
      !(job.equals("cashier") && (salary >= 32 || salary * 1.12f >= 32)) &&
      !(job.equals("barista") && (salary >= 70 || salary * 1.12f >= 70))) {
    fill(0, 255, 0, 80);
  } else {
    fill(80, 220);
  }
  rect(720, 420, 80, 20);
  fill(255);
  if (job.equals("cashier") && (salary >= 32 || salary * 1.12f >= 32)) text("MAXED", 720, 420);
  else if (job.equals("barista") && (salary >= 70 || salary * 1.12f >= 70)) text("MAXED", 720, 420);
  else if (job.equals("unemployed")) text("N/A", 720, 420);
  else text("$" + nf(salaryUpgradeCost, 0, 2), 720, 420);

  if (money >= taskUpgradeCost) fill(0, 255, 0, 80);
  else fill(80, 220);
  rect(825, 420, 80, 20);
  fill(255);
  text("$" + nf(taskUpgradeCost, 0, 2), 825, 420);

  if (hasShownJobPopup && !isHelpTaskPending && isTasksPanelOpen && !hasUsedHelpTask) {
    redarrow(261, 429, "right");
  }

  rectMode(CORNER);
  stroke(0);
}
```

§ 12

G_Services.pde

# Services Panel

Vet, walker, cleaner services and prescription course management

```
// ========================
// G_Services.pde
// Services panel: vet, walker, cleaner, treatment popups, and prescription logic.
// ========================


// ========================
// Services State
// ========================
boolean isServicesOpen = false;
boolean hasOpenedServices = false;
boolean isVetOpen = false;
boolean isShowingTreatmentPopup = false;
boolean hasShownTreatmentPopup = false;
boolean isShowingVetFailedPopup = false;
boolean hasUsedLowQualityVet = false;
boolean isFirstLowQualityVetAttempt = true;
boolean hasNeverBoughtHighQualityCare = true;
boolean hasCleanerVisited = false;

String vetTreatmentMessage = "";
String currentSicknessName = "";
boolean isPetSick = false;

int prescribedMedicineIndex = -1;
int prescriptionDaysRequired = 0;
int prescriptionDaysCompleted = 0;
int lastDoseTakenDay = -1;

String[] sicknessNames = {
  "infection", "cold", "flu", "vitamin deficiency", "malnutrition",
  "electrolyte deficiency", "severe fatigue", "depression", "anxiety",
  "injury", "calcium deficiency", "food poisoning"
};

int timesUsedVetCare = 0;
int lowQualityCareCount = 0;
int highQualityCareCount = 0;
int cleanersHiredCount = 0;
int walkersHiredCount = 0;
float totalHealthRestored = 0;
float totalHappinessRestored = 0;

PImage vet, walker, cleaner;


// ========================
// Services Panel
// ========================
void services() {
  rectMode(CORNERS);
  stroke(169);
  strokeWeight(5);
  fill(80, 220);
  rect(110, 122.5f, 990, 572.5f);
```

```
  textAlign(CENTER, CENTER);
  fill(255);
  textSize(35);
  text("SERVICES:", width/2, height*0.235f);
  textSize(25);
  imageMode(CENTER);
  text("Vet:", 256.67f, 200);
  text("Walker:", 550, 200);
  text("Cleaner:", 843.33f, 200);

  rectMode(CENTER);
  fill(0, 255, 0, 80);
  rect(256.67f, 520, 160, 40);
  if (money < 10) fill(169, 80);
  rect(550, 520, 160, 40);
  rect(843.33f, 520, 160, 40);
  fill(255);
  text("VISIT", 256.7f, 520);
  text("HIRE - $10", 550, 520);
  text("HIRE - $10", 843.33f, 520);
  image(vet, 250, 330, vet.width/1.55f, vet.height/1.55f);
  textSize(15);
  text("A veterinarian can prescribe", 256.7f, 418);
  text("your pet medicine and/or", 256.7f, 433);
  text("give advice to you on", 256.7f, 448);
  text("pet care, getting rid of", 256.7f, 463);
  text("injury, sickness, or infection", 256.7f, 478);

  image(walker, 530, 330, walker.width/3.5f, walker.height/3.5f);
  text("A walker will take", 604, 418);
  text("your pet on a walk", 580, 433);
  text("in order to stabilize", 575, 448);
  text("energy while increasing", 575, 463);
  text("health and happiness.", 575, 478);

  image(cleaner, 830, 310, cleaner.width/3.8f, cleaner.height/3.8f);
  text("A cleaner will tidy", 843.33f, 418);
  text("your pet's habitat,", 843.33f, 433);
  text("reducing the daily", 843.33f, 448);
  text("risk of your pet", 843.33f, 463);
  text("getting sick.", 843.33f, 478);

  noFill();
  rectMode(CORNERS);
  rect(933, 132, 976, 171.5f);
  textAlign(LEFT, CENTER);
  textSize(40);
  fill(255);
  text("X", 942, 151.75f);
  rectMode(CORNER);
  stroke(0);
  strokeWeight(2);
  if (!hasUsedLowQualityVet) redarrow(122, 520, "right");
}


// ========================
// Vet Sub-Panel
// ========================
void vet() {
  imageMode(CENTER);
  rectMode(CORNERS);
  stroke(169);
  strokeWeight(5);
  fill(80, 220);
```

```
  rect(330, 222.5f, 770, 472.5f);
  textAlign(CENTER, CENTER);
  fill(255);
  textSize(35);
  text("THE VET:", width/2, height*0.235f+100);
  textSize(25);
  text("3 Star Vet:", 440, 294);
  image(vet, 440, 347, vet.width/3.75f, vet.height/3.75f);
  text("5 Star Vet:", 660, 294);
  image(vet, 660, 347, vet.width/3.75f, vet.height/3.75f);
  textSize(15);
  text("Treat " + alligator.petName + "'s " + currentSicknessName, 440, 389);
  text("for $5.", 440, 402);
  text("Treat " + alligator.petName + "'s " + currentSicknessName, 660, 389);
  text("for $20.", 660, 402);

  if (money >= 5) fill(0, 255, 0, 80);
  else fill(169, 80);
  rectMode(CENTER);
  rect(440, 435, 120, 30);
  if (money < 20) fill(169, 80);
  rect(660, 435, 120, 30);
  fill(255);
  textSize(20);
  text("TREAT", 440, 435);
  text("TREAT", 660, 435);

  noFill();
  rectMode(CORNERS);
  rect(713, 232, 756, 271.5f);
  textAlign(LEFT, CENTER);
  textSize(40);
  fill(255);
  text("X", 722, 251.75f);
  rectMode(CORNER);
  stroke(0);
  strokeWeight(2);
  if (!hasUsedLowQualityVet) redarrow(330, 435, "right");
}


// ========================
// Treatment Popup
// ========================
void treatmentpopup() {
  imageMode(CENTER);
  image(popupbackground, width/2, height*0.42f, popupbackground.width*0.6f, popupbackground.height*0.6f);
  fill(0);
  textFont(times50);
  textSize(20);
  drawWrappedTextInBox(vetTreatmentMessage, 338, 271, 761, 400, 6);
}


// ========================
// Prescription Management
// ========================
void clearPrescriptionCourse() {
  for (int i = 0; i < medicineIsPrescribed.length; i++) {
    medicineIsPrescribed[i] = false;
  }
  prescribedMedicineIndex = -1;
  prescriptionDaysRequired = 0;
  prescriptionDaysCompleted = 0;
  lastDoseTakenDay = -1;
```

```
}

void startPrescriptionCourse(int medIndex) {
  int preservedProgress = (prescribedMedicineIndex == medIndex && prescriptionDaysCompleted > 0)
    ? min(prescriptionDaysCompleted, medicineDefaultQuantities[medIndex] - 1)
    : 0;

  clearPrescriptionCourse();

  prescribedMedicineIndex = medIndex;
  prescriptionDaysRequired = medicineDefaultQuantities[medIndex];
  prescriptionDaysCompleted = preservedProgress;
  lastDoseTakenDay = -1;

  medicineIsPrescribed[medIndex] = true;
}
```

§ 13

G_Rest.pde

# Rest System

Gradient rest bar, bouncing timing marker, rest attempt tracking

```
// ========================
// G_Rest.pde
// Rest panel rendering: gradient bar, bouncing marker, and rest button.
// ========================


// ========================
// Rest State
// ========================
boolean isRestOpen = false;
boolean hasUsedRest = false;
boolean hasAlligatorRestedOnce = false;
boolean isShowingRestPopup = false;
boolean hasShownRestPopup = false;

int restAttemptsRemaining = 2;
int restAttempts = 0;
int timesRestedSuccessfully = 0;
float totalEnergyRestoredFromResting = 0;

float restBarCenterX = 439;
float restBarCenterY = 490;
float restBarWidth = 220;
float restBarHeight = 30;
float restBarLeft   = restBarCenterX - restBarWidth  / 2;
float restBarTop    = restBarCenterY - restBarHeight / 2;
float restBarBottom = restBarCenterY + restBarHeight / 2;

int COLOR_RED    = color(255, 0, 0);
int COLOR_ORANGE = color(255, 165, 0);
int COLOR_YELLOW = color(255, 255, 0);
int COLOR_GREEN  = color(0, 255, 0);

float restMarkerProgress  = 0;
float restMarkerDirection = 1;
float restMarkerX         = restBarLeft;


// ========================
// Rest Popup
// ========================
void restpopup() {
  imageMode(CENTER);
  image(popupbackground, width/2, height*0.42f, popupbackground.width*0.6f, popupbackground.height*0.6f);
  fill(0);
  textFont(times50);
  textSize(20);
  drawWrappedTextInBox("Seems like " + alligator.petName + " is tired. Click rest after closing this window to attempt to stabilize his energy.", 338, 271, 761, 400, 6);
}


// ========================
// Rest Panel
// ========================
```

```
55  void rest() {
56    rectMode(CENTER);
57    stroke(169);
58    strokeWeight(5);
59    fill(80, 220);
60    rect(439, 490, 320, 90);
61
62    int pixelSize = 5;
63    rectMode(CORNER);
64    noStroke();
65
66    for (int px = 0; px < restBarWidth; px += pixelSize) {
67      float position = px / restBarWidth;
68      int gradientColor;
69
70      if (position < 0.25f) {
71        gradientColor = lerpColor(COLOR_RED, COLOR_ORANGE, map(position, 0, 0.25f, 0, 1));
72      } else if (position < 0.4f) {
73        gradientColor = lerpColor(COLOR_ORANGE, COLOR_YELLOW, map(position, 0.25f, 0.4f, 0, 1));
74      } else if (position < 0.5f) {
75        gradientColor = lerpColor(COLOR_YELLOW, COLOR_GREEN, map(position, 0.4f, 0.5f, 0, 1));
76      } else if (position < 0.6f) {
77        gradientColor = lerpColor(COLOR_GREEN, COLOR_YELLOW, map(position, 0.5f, 0.6f, 0, 1));
78      } else if (position < 0.75f) {
79        gradientColor = lerpColor(COLOR_YELLOW, COLOR_ORANGE, map(position, 0.6f, 0.75f, 0, 1));
80      } else {
81        gradientColor = lerpColor(COLOR_ORANGE, COLOR_RED, map(position, 0.75f, 1, 0, 1));
82      }
83
84      fill(gradientColor);
85      rect(restBarLeft + px, restBarTop, pixelSize, restBarHeight);
86    }
87
88    float edgeSlowdown = abs(restMarkerProgress - 0.5f) * 2.0f;
89    float markerSpeed = map(edgeSlowdown, 0, 1, 0.03f, 0.008f);
90
91    restMarkerProgress += markerSpeed * restMarkerDirection;
92
93    if (restMarkerProgress >= 1) { restMarkerProgress = 1; restMarkerDirection = -1; }
94    if (restMarkerProgress <= 0) { restMarkerProgress = 0; restMarkerDirection = 1; }
95
96    restMarkerX = lerp(restBarLeft, restBarLeft + restBarWidth, restMarkerProgress);
97
98    textSize(17);
99    rectMode(CENTER);
100   if (restAttemptsRemaining > 0) fill(0, 255, 0, 80);
101   else fill(80, 220);
102   stroke(255);
103   strokeWeight(2);
104   rect(439, 520, 80, 20);
105   textAlign(CENTER, CENTER);
106   fill(255);
107   text("REST", 439, 520);
108   stroke(0);
109   strokeWeight(1);
110   fill(255);
111   rectMode(CORNERS);
112   rect(restMarkerX, restBarTop - 6, restMarkerX + 2, restBarBottom + 6);
113   textSize(12);
114   textAlign(CENTER);
115   text("Poor", 309, 486);
116   text("Rest", 309, 500);
117   text("Poor", 569, 486);
118   text("Rest", 569, 500);
119   text("ATTEMPTS LEFT TODAY: " + restAttemptsRemaining, 439, 462);
```

```
  noFill();
  stroke(255);
  strokeWeight(1);
  rect(567, 454, 583, 470);
  fill(255);
  textAlign(CENTER, CENTER);
  textSize(14);
  text("X", 575, 462);
  rectMode(CORNER);
  stroke(0);
  strokeWeight(2);
}
```

§ 14

G_Achievements.pde

# Achievement System

30 tiered achievements with progress tracking, priority sorting, and reward collection

```
// ========================
// G_Achievements.pde
// Achievement data initialization, progress tracking, sorting, and panel rendering.
// ========================


// ========================
// Achievement State
// ========================
boolean isAchievementsOpen = false;
boolean hasOpenedAchievements = false;
boolean hasClosedAchievements = false;

String[] achievementNames        = new String[30];
String[] achievementDescriptions = new String[30];
int[]    achievementRewards      = new int[30];
float[]  achievementGoals        = new float[30];
float[]  achievementProgress     = new float[30];
int[]    achievementTiers        = new int[30];
boolean[] isAchievementCollectable = new boolean[30];
float[]  achievementDisplayPriority = new float[30];
int[]    achievementDrawOrder    = new int[30];
int[]    achievementType         = new int[30];

float achievementScrollOffset = 0;
float achievementScrollContentHeight = 0;
float achievementViewportX = 320;
float achievementViewportY = 190;
float achievementViewportWidth = 450;
float achievementViewportHeight = 370;
float achievementItemLineHeight = 110;
float achievementScrollbarX = 770;
float achievementScrollbarY = 190;
float achievementScrollbarWidth = 12;
float achievementScrollbarHeight = 370;
boolean isDraggingAchievementScrollbar = false;
float achievementScrollThumbOffsetY = 0;

// Counters used by achievements
// Note: salaryUpgradeCount, taskUpgradeCount, helpTaskCount live in G_Earn.pde
//       timesRestedSuccessfully, restAttempts, totalEnergyRestoredFromResting live in G_Rest.pde
//       timesUsedVetCare, lowQualityCareCount, highQualityCareCount,
//       cleanersHiredCount, walkersHiredCount, totalHealthRestored,
//       totalHappinessRestored live in G_Services.pde
//       bankTransactionsLoggedCount lives in G_Bank.pde
int timesFedPet = 0;
int timesBoughtMedicine = 0;
int playPointUpgradeCount = 0;
int currentDay = 1;
int inventoryFullCount = 0;
int medicineGivenCount = 0;
int itemsBoughtCount = 0;
int inventoryItemsUsedCount = 0;
float totalMoneyEarned = 0;
float totalMoneySpent = 0;
```

```
float highestMoneyBalance = 0;
float moneyEarnedFromMinigames = 0;
float currentPlaySessionMoneyEarned = 0;


// ========================
// Achievement Initialization
// ========================
void initAchievements() {
  for (int i = 0; i < 30; i++) {
    achievementTiers[i] = 1;
    isAchievementCollectable[i] = false;
    achievementDrawOrder[i] = i;
  }
  refreshAchievementData();
}

void refreshAchievementData() {
  setHopAchievement(0, "Swamp Hop High Score", 20, 2);
  setSnackAchievement(1, "Snack Snatch High Score", 15, 2);
  setFetchAchievement(2, "Fetch Frenzy High Score", 10, 2);

  setMoneyEarnedAchievement(3, "Total Money Earned", 160, 3, totalMoneyEarned);
  setMoneySpentAchievement(4, "Total Money Spent", 55, 3, totalMoneySpent);
  setHighestMoneyAchievement(5, "Highest Money Balance", 125, 3, highestMoneyBalance);

  setCountAchievement(6,  "Times Pet Fed",                    "Feed your pet",                  5,  6, timesFedPet);
  setCountAchievement(7,  "Times Medicine Given",             "Give medicine",                  3,  6, medicineGivenCount);
  setCountAchievement(8,  "Total Vet Visits",                 "Visit the vet",                  3,  8, timesUsedVetCare);
  setCountAchievement(9,  "Perfect Rests Completed",          "Complete perfect rests",         3,  6, timesRestedSuccessfully);
  setCountAchievement(10, "Salary Upgrades Purchased",        "Buy salary upgrades",            1,  6, salaryUpgradeCount);
  setCountAchievement(11, "Task Income Upgrades Purchased",   "Buy task income upgrades",       1,  7, taskUpgradeCount);
  setCountAchievement(12, "Play Point Upgrades Purchased",    "Buy play point upgrades",        5,  7, playPointUpgradeCount);
  setCountAchievement(13, "Help Tasks Completed",             "Complete help tasks",            1,  5, helpTaskCount);

  setMoneyEarnedAchievement(14, "Money Earned From Salary", 30, 8, moneyEarnedFromSalary);
  setMoneyEarnedAchievement(15, "Money Earned From Tasks",  20, 8, moneyEarnedFromTasks);

  setCountAchievement(16, "Items Purchased From Store",  "Buy store items",       3,  7, itemsBoughtCount);
  setCountAchievement(17, "Medicine Purchased",          "Buy medicine",          2,  7, timesBoughtMedicine);
  setCountAchievement(18, "Inventory Items Used",        "Use inventory items",   3,  7, inventoryItemsUsedCount);

  setMoneyEarnedAchievement(19, "Money Earned From Minigames", 25, 7, moneyEarnedFromMinigames);

  setCountAchievement(20, "Rest Attempts",  "Attempt rests",   1, 5, restAttempts);
  setAmountAchievement(21, "Total Energy Restored From Resting", "Restore energy from resting", 70, 7, totalEnergyRestoredFromResting);
  setAmountAchievement(22, "Total Health Restored",    "Restore health",     20, 7, totalHealthRestored);
  setAmountAchievement(23, "Total Happiness Restored", "Restore happiness",  20, 7, totalHappinessRestored);

  setDayAchievement(24, "Days Survived", 5, 10, currentDay);
  setCountAchievement(25, "Cleaners Hired",           "Hire a cleaner",         1,  10, cleanersHiredCount);
  setCountAchievement(26, "Low Quality Care Purchased",  "Buy low quality care",   2,  7, lowQualityCareCount);
```

```
  setCountAchievement(27, "High Quality Care Purchased", "Buy high quality care",  1,  10, highQualityCareCount);
  setCountAchievement(28, "Bank Transactions Logged",  "Do a transaction (buy or earn)", 10, 15, bankTransactionsLoggedCount);
  setCountAchievement(29, "Walkers Hired", "Hire a walker", 1, 10, walkersHiredCount);

  updateAchievementOrder();
}


// ========================
// Achievement Setters
// ========================
void setHopAchievement(int i, String baseName, float baseGoal, int baseReward) {
  int tier = max(1, achievementTiers[i]);
  achievementTiers[i] = tier;
  achievementType[i] = 0;
  achievementNames[i] = baseName + " - Tier " + tier;
  achievementGoals[i] = max(1, round(baseGoal * pow(1.2f, tier - 1)));
  achievementRewards[i] = max(1, round(baseReward * pow(1.18f, tier - 1)));
  achievementDescriptions[i] = "Highest score reached in Swamp Hop";
  achievementProgress[i] = swampHopBestScore;
  isAchievementCollectable[i] = achievementProgress[i] >= achievementGoals[i];
}

void setSnackAchievement(int i, String baseName, float baseGoal, int baseReward) {
  int tier = max(1, achievementTiers[i]);
  achievementTiers[i] = tier;
  achievementType[i] = 1;
  achievementNames[i] = baseName + " - Tier " + tier;
  achievementGoals[i] = max(1, round(baseGoal * pow(1.2f, tier - 1)));
  achievementRewards[i] = max(1, round(baseReward * pow(1.18f, tier - 1)));
  achievementDescriptions[i] = "Highest score reached in Snack Snatch";
  achievementProgress[i] = snatchBestScore;
  isAchievementCollectable[i] = achievementProgress[i] >= achievementGoals[i];
}

void setFetchAchievement(int i, String baseName, float baseGoal, int baseReward) {
  int tier = max(1, achievementTiers[i]);
  achievementTiers[i] = tier;
  achievementType[i] = 2;
  achievementNames[i] = baseName + " - Tier " + tier;
  achievementGoals[i] = max(1, round(baseGoal * pow(1.2f, tier - 1)));
  achievementRewards[i] = max(1, round(baseReward * pow(1.18f, tier - 1)));
  achievementDescriptions[i] = "Highest score reached in Fetch Frenzy";
  achievementProgress[i] = fetchBestScore;
  isAchievementCollectable[i] = achievementProgress[i] >= achievementGoals[i];
}

void setMoneyEarnedAchievement(int i, String baseName, float baseGoal, int baseReward, float currentValue) {
  int tier = max(1, achievementTiers[i]);
  achievementTiers[i] = tier;
  achievementType[i] = 3;
  achievementNames[i] = baseName + " - Tier " + tier;
  achievementGoals[i] = max(1, round(baseGoal * pow(1.3f, tier - 1)));
  achievementRewards[i] = max(1, round(baseReward * pow(1.2f, tier - 1)));
  achievementDescriptions[i] = baseName;
  achievementProgress[i] = currentValue;
  isAchievementCollectable[i] = achievementProgress[i] >= achievementGoals[i];
}

void setMoneySpentAchievement(int i, String baseName, float baseGoal, int baseReward, float currentValue) {
  int tier = max(1, achievementTiers[i]);
  achievementTiers[i] = tier;
  achievementType[i] = 4;
```

```
171 ■   achievementNames[i] = baseName + " - Tier " + tier;
172 ■   achievementGoals[i] = max(1, round(baseGoal * pow(1.3f, tier - 1)));
173 ■   achievementRewards[i] = max(1, round(baseReward * pow(1.2f, tier - 1)));
174 ■   achievementDescriptions[i] = baseName;
175 ■   achievementProgress[i] = currentValue;
176 ■   isAchievementCollectable[i] = achievementProgress[i] >= achievementGoals[i];
177 ■ }
178 ■
179 ■ void setHighestMoneyAchievement(int i, String baseName, float baseGoal, int baseReward, float currentValue) {
180 ■   int tier = max(1, achievementTiers[i]);
181 ■   achievementTiers[i] = tier;
182 ■   achievementType[i] = 5;
183 ■   achievementNames[i] = baseName + " - Tier " + tier;
184 ■   achievementGoals[i] = max(1, round(baseGoal * pow(1.25f, tier - 1)));
185 ■   achievementRewards[i] = max(1, round(baseReward * pow(1.2f, tier - 1)));
186 ■   achievementDescriptions[i] = baseName;
187 ■   achievementProgress[i] = currentValue;
188 ■   isAchievementCollectable[i] = achievementProgress[i] >= achievementGoals[i];
189 ■ }
190 ■
191 ■ void setCountAchievement(int i, String baseName, String actionText, float baseGoal, int baseReward, float
currentValue) {
192 ■   int tier = max(1, achievementTiers[i]);
193 ■   achievementTiers[i] = tier;
194 ■   achievementType[i] = 6;
195 ■   achievementNames[i] = baseName + " - Tier " + tier;
196 ■   achievementGoals[i] = max(1, round(baseGoal * pow(1.5f, tier - 1)));
197 ■   achievementRewards[i] = max(1, round(baseReward * pow(1.18f, tier - 1)));
198 ■   if (PApplet.parseInt(achievementGoals[i]) == 1)
199 ■     achievementDescriptions[i] = actionText + " 1 time";
200 ■   else
201 ■     achievementDescriptions[i] = actionText + " " + PApplet.parseInt(achievementGoals[i]) + " times";
202 ■   achievementProgress[i] = currentValue;
203 ■   isAchievementCollectable[i] = achievementProgress[i] >= achievementGoals[i];
204 ■ }
205 ■
206 ■ void setAmountAchievement(int i, String baseName, String actionText, float baseGoal, int baseReward, float
currentValue) {
207 ■   int tier = max(1, achievementTiers[i]);
208 ■   achievementTiers[i] = tier;
209 ■   achievementType[i] = 7;
210 ■   achievementNames[i] = baseName + " - Tier " + tier;
211 ■   achievementGoals[i] = max(1, round(baseGoal * pow(1.35f, tier - 1)));
212 ■   achievementRewards[i] = max(1, round(baseReward * pow(1.18f, tier - 1)));
213 ■   achievementDescriptions[i] = actionText + " by " + PApplet.parseInt(achievementGoals[i]) + " total points";
214 ■   achievementProgress[i] = currentValue;
215 ■   isAchievementCollectable[i] = achievementProgress[i] >= achievementGoals[i];
216 ■ }
217 ■
218 ■ void setDayAchievement(int i, String baseName, float baseGoal, int baseReward, float currentValue) {
219 ■   int tier = max(1, achievementTiers[i]);
220 ■   achievementTiers[i] = tier;
221 ■   achievementType[i] = 8;
222 ■   achievementNames[i] = baseName + " - Tier " + tier;
223 ■   achievementGoals[i] = max(1, round(baseGoal * pow(1.4f, tier - 1)));
224 ■   achievementRewards[i] = max(1, round(baseReward * pow(1.2f, tier - 1)));
225 ■   achievementDescriptions[i] = "Reach day " + PApplet.parseInt(achievementGoals[i]);
226 ■   achievementProgress[i] = currentValue;
227 ■   isAchievementCollectable[i] = achievementProgress[i] >= achievementGoals[i];
228 ■ }
229 ■
230 ■
231 ■ // ========================
232 ■ // Achievement Order / Progress Update
233 ■ // ========================
```

```
void updateAchievementProgress() {
  refreshAchievementData();
}

void updateAchievementOrder() {
  for (int i = 0; i < 30; i++) {
    float ratio = (achievementGoals[i] > 0) ? achievementProgress[i] / achievementGoals[i] : 0;
    ratio = constrain(ratio, 0, 1);
    achievementDisplayPriority[i] = ratio;
    if (isAchievementCollectable[i]) achievementDisplayPriority[i] += 1000;
    achievementDrawOrder[i] = i;
  }
  for (int a = 0; a < 29; a++) {
    for (int b = a + 1; b < 30; b++) {
      int ia = achievementDrawOrder[a];
      int ib = achievementDrawOrder[b];
      if (achievementDisplayPriority[ib] > achievementDisplayPriority[ia]) {
        int temp = achievementDrawOrder[a];
        achievementDrawOrder[a] = achievementDrawOrder[b];
        achievementDrawOrder[b] = temp;
      }
    }
  }
}


// ========================
// Achievements Panel
// ========================
void achievements() {
  updateAchievementProgress();

  rectMode(CORNERS);
  stroke(169);
  strokeWeight(5);
  fill(80, 220);
  rect(310, 122.5f, 790, 572.5f);
  line(310, 182, 790, 182);

  textAlign(CENTER);
  fill(255);
  textSize(35);
  text("ACHIEVEMENTS:", width/2, height*0.235f);

  achievementScrollContentHeight = 20 + 30 * achievementItemLineHeight;
  float maxScroll = max(0, achievementScrollContentHeight - achievementViewportHeight);
  achievementScrollOffset = constrain(achievementScrollOffset, 0, maxScroll);

  pushMatrix();
  clip((int)achievementViewportX, (int)achievementViewportY, (int)achievementViewportWidth,
(int)achievementViewportHeight);

  rectMode(CORNER);
  textAlign(LEFT, TOP);

  for (int row = 0; row < 30; row++) {
    int i = achievementDrawOrder[row];
    float y = achievementViewportY + 20 + row * achievementItemLineHeight - achievementScrollOffset;
    float boxX = achievementViewportX + 10;
    float boxW = achievementViewportWidth - 30;
    float boxH = 96;

    float progressRatio = (achievementGoals[i] > 0) ? achievementProgress[i] / achievementGoals[i] : 0;
    progressRatio = constrain(progressRatio, 0, 1);

```

```
    fill(120, 180);
    stroke(220);
    strokeWeight(2);
    rect(boxX, y, boxW, boxH, 8);

    fill(255);
    textAlign(LEFT, TOP);
    textSize(16);
    text(achievementNames[i], boxX + 12, y + 7);

    textAlign(RIGHT, TOP);
    textSize(15);
    fill(255, 230, 120);
    text("+$" + achievementRewards[i], boxX + boxW - 12, y + 8);

    textAlign(LEFT, TOP);
    textSize(11);
    fill(215);
    text(achievementDescriptions[i], boxX + 12, y + 28, boxW - 24, 24);

    float barX = boxX + 12;
    float barY = y + 58;
    float barW = boxW - 116;
    float barH = 12;

    fill(55, 160);
    stroke(180);
    strokeWeight(1.5f);
    rect(barX, barY, barW, barH, 4);

    noStroke();
    fill(100, 220, 140);
    rect(barX, barY, barW * progressRatio, barH, 4);

    fill(235);
    textSize(10);
    textAlign(RIGHT, CENTER);
    text(PApplet.parseInt(min(achievementProgress[i], achievementGoals[i])) + "/" +
PApplet.parseInt(achievementGoals[i]),
         boxX + boxW - 12, barY + 6);

    rectMode(CENTER);
    stroke(169);
    strokeWeight(2);
    if (isAchievementCollectable[i]) fill(0, 255, 0, 80);
    else fill(80, 220);
    rect(boxX + boxW - 52, y + 38, 80, 20);
    fill(255);
    textAlign(CENTER, CENTER);
    textSize(12);
    text("COLLECT", boxX + boxW - 52, y + 38);
    rectMode(CORNER);
  }

  noClip();
  popMatrix();

  noFill();
  stroke(200);
  strokeWeight(2);
  rectMode(CORNER);
  rect(achievementViewportX, achievementViewportY, achievementViewportWidth, achievementViewportHeight);

  fill(120);
  stroke(80);
```

```
  rect(achievementScrollbarX, achievementScrollbarY, achievementScrollbarWidth, achievementScrollbarHeight);

  if (achievementScrollContentHeight > achievementViewportHeight) {
    float thumbH = max(40, (achievementViewportHeight / achievementScrollContentHeight) * achievementScrollbarHeight);
    float thumbY = map(achievementScrollOffset, 0, maxScroll, achievementScrollbarY, achievementScrollbarY + achievementScrollbarHeight - thumbH);
    fill(220);
    stroke(100);
    rect(achievementScrollbarX, thumbY, achievementScrollbarWidth, thumbH);
  } else {
    fill(220);
    stroke(100);
    rect(achievementScrollbarX, achievementScrollbarY, achievementScrollbarWidth, achievementScrollbarHeight);
  }

  noFill();
  stroke(169);
  rectMode(CORNERS);
  rect(733, 132, 776, 171.5f);
  textAlign(LEFT, CENTER);
  textSize(40);
  fill(255);
  text("X", 742, 151.75f);

  strokeWeight(2);
  stroke(0);
  rectMode(CORNER);
}
```

§ 15

H_Play_Screen.pde

# Minigames

Swamp Hop (side-scroller), Snack Snatch (food catcher), Fetch Frenzy (top-down)

```
// ========================
// H_Play_Screen.pde
// Three minigames: Swamp Hop, Snack Snatch, and Fetch Frenzy.
// Each minigame awards money based on moneyPerMinigamePoint (upgradeable in the Earn panel).
// All three share earnMinigameMoney(), tickMinigameStats(), and drawMinigameModal().
// ========================

Fade minigameFade = new Fade(255);  // starts black on minigame entry; fades to clear as game begins

// ========================
// Minigame helper functions
// ========================

// Adds money earned in a minigame to all relevant counters

// ========================
// Additional Global State (moved to top for clarity)
// ========================
boolean isOnChoiceScreen = false;   // true while the player is on the minigame selection screen
boolean isEnterSwampHop = false;    // set true when player picks Swamp Hop
PImage minigamechoice;            // background image for the minigame selection screen
PImage alligatorf1;
PImage alligatorf2;
PImage alligatorf3;
PImage alligatorf4;
PImage log;
PImage rock;
PImage vine;
PImage mud;
int walkAnimFrameDelay = 6;    // frames to hold each sprite before advancing
float x = 224;          // Alligator's fixed horizontal position on screen
float y = 500;          // Vertical position; changes during jumps
float hopGravity = 1.2;    // Acceleration applied to hopVelocityY each frame
float hopJumpStrength = 20; // Upward impulse applied when SPACE is pressed
boolean isSwampHopOnGround = true;           // Prevents double-jumping
boolean isSwampHopRetry        = false;        // Set true when player clicks RETRY
boolean isExitingMinigame = false;                // Set true when player clicks EXIT to return to main screen
boolean isSwampHopFirstPlay = true; // True on first entry — shows "WELCOME!" modal instead of "YOU LOST!"
boolean isEnterFetchFrenzy = false;    // Set true when player picks Fetch Frenzy
int swampHopGraceFrames = 0;            // Prevents immediate collision detection on isSwampHopRetry
int swampHopSpawnDelay = 0;       // Delays first obstacle spawn after a isSwampHopRetry
boolean isEnterSnackSnatch = false;    // Set true when player picks Snack Snatch
ArrayList<Obstacle> swampHopObstacles = new ArrayList<Obstacle>(); // Active obstacle list
float obstacleSpawnGap = 900;    // Minimum pixel gap between consecutive swampHopObstacles
float nextObstacleX = 0;       // X position at which the next obstacle will be spawned
float hopGatorHitboxWidth = 0.70;   // 70% of sprite width
float hopGatorHitboxHeight = 0.45;   // 45% of sprite height
float vineHitboxWidth = 0.45;
float vineHitboxHeight = 0.85;
float vineHitboxOffsetY = -10;
float rockHitboxWidth = 0.75;
float rockHitboxHeight = 0.55;
float rockHitboxOffsetY = 0;
float mudHitboxWidth = 0.75;
```

```
float mudHitboxHeight = 0.55;
float mudHitboxOffsetY = 0;
float logHitboxWidth = 1.0;
float logHitboxHeight = 0.7;
float logHitboxOffsetY = 0;
float swampHopScoreTimer = 0;    // Counts frames; every 60 frames = 1 point + moneyPerMinigamePoint earned
int snatchScore = 0;               // Current run score for Snack Snatch
float swampHopBgScrollX = 0;
float swampHopScrollSpeed = 16;
float swampHopBgWidth;
float swampHopBgHeight;
PImage bluegill;
PImage bass;
PImage perch;
PImage goldfish;
PImage crab;
PImage lambchop;
PImage porkchop;
PImage broccoli;
PImage carrot;
PImage tomato;
PImage pepper;
ArrayList<FallingFood> fallingFoods = new ArrayList<FallingFood>();
boolean isSnatchFirstPlay = true;
boolean isSnatchLost = true;
int snatchBestScore = 0;
float snatchGatorWidthMult = 0.600; // fraction of sprite width used for collision
float snatchGatorHeightMult = 0.240; // fraction of sprite height used for collision
float snatchGatorOffsetX = -4.0;     // horizontal center offset in pixels
float bluegillW=0.520, bluegillH=0.380, bluegillOX=-2, bluegillOY=-4;
float bassW=0.560, bassH=0.400, bassOX=4, bassOY=-8;
float perchW=0.580, perchH=0.360, perchOX=0, perchOY=-2;
float goldfishW=0.520, goldfishH=0.380, goldfishOX=-2, goldfishOY=-4;
float crabW=0.600, crabH=0.360, crabOX=0, crabOY=-4;
float lambchopW=0.420, lambchopH=0.340, lambchopOX=-8, lambchopOY=8;
float porkchopW=0.500, porkchopH=0.420, porkchopOX=0, porkchopOY=0;
float broccoliW=0.540, broccoliH=0.620, broccoliOX=2, broccoliOY=-4;
float carrotW=0.380, carrotH=0.540, carrotOX=-6, carrotOY=8;
float tomatoW=0.380, tomatoH=0.480, tomatoOX=2, tomatoOY=-2;
float pepperW=0.340, pepperH=0.540, pepperOX=2, pepperOY=-4;
PImage fetchfrenzybackground;
PImage ball;
float ballX = 548, ballY = 618;
float ballVelocityX = 0, ballVelocityY = 0;
float ballZ = 0;
float ballVerticalVelocity = 0;
boolean isBallMoving = false;
float ballGravity = 1.0;
float ballBounceFactor = 0.55;
float ballAirDrag = 0.995;
float ballGroundFriction = 0.92;
float ballStopSpeedThreshold = 0.15;
float ballStopBounceThreshold = 0.6;
PImage topalligator1, topalligator2;
int fetchWalkFrameTimer = 0;
float fetchPlayerX = 0;
float fetchPlayerY = 0;
String fetchFacingDirection = "RIGHT";
boolean ballHitsPlayer = false;
boolean wasBallHittingPlayer = false;
boolean hasBallBeenLaunched = false;
int fetchScore=0;
int fetchBestScore=0;
float fetchTimer=3;
RectHB ballHB  = new RectHB(0, -0.50, 36.00, 37.00);
```

```
RectHB gatorHB = new RectHB(0, -42.00, 95.00, 144.00);
boolean isBallWaitingToLaunch = false;
int ballLaunchCountdown = 0;
final int LAUNCH_DELAY = 30;
boolean isBallOnField = true;
boolean isFetchLost = true;
boolean isFetchFirstPlay = true;
void earnMinigameMoney(float amount) {
  money += amount;
  currentPlaySessionMoneyEarned += amount;
  totalMoneyEarned += amount;
  moneyEarnedFromMinigames += amount;
}

// Drains energy, boosts happiness each frame during active gameplay
void tickMinigameStats() {
  alligator.energy    -= 0.01;
  alligator.happiness += 0.01;
  totalHappinessRestored += 0.01;
}

// Draws the lose/welcome modal shared by all three minigames.
// title/sub1/sub2/sub3 are the header and instruction lines (empty string = skip).
// leftBtn / rightBtn are the button labels.
void drawMinigameModal(String title, String sub1, String sub2, String sub3,
                       String leftBtn, String rightBtn) {
  stroke(255);
  strokeWeight(5);
  fill(80, 150);
  rectMode(CENTER);
  rect(width/2, height*0.4, 300, 400);
  fill(255);
  textAlign(CENTER, CENTER);
  textFont(arcade);
  text(title, width/2, 131);
  textSize(sub2.isEmpty() ? 30 : 18);
  if (!sub1.isEmpty()) text(sub1, width/2, 160);
  if (!sub2.isEmpty()) text(sub2, width/2, 180);
  if (!sub3.isEmpty()) text(sub3, width/2, 200);
  textSize(30);
  applyAlligatorTint();
  image(alligator.energeticalligator, width/2, 300,
        alligator.energeticalligator.width/4, alligator.energeticalligator.height/4);
  noTint();
  text(leftBtn,  width/2 - 70, 436);
  text(rightBtn, width/2 + 70, 436);
  noFill();
  rect(width/2 - 70, 436, 105, 50);
  rect(width/2 + 70, 436, 105, 50);
}

// Draws the stat bars overlay shown during all three minigames
void drawMinigameStats() {
  strokeWeight(1);
  stroke(0);
  rectMode(CORNER);
  textAlign(LEFT, CENTER);
  textFont(arcade);
  textSize(20);
  fill(255);
  text("Energy:", width * 0.015, height * 0.2);
  energybar.setValue(alligator.energy);
  energybar.drawenergyscale();
  fill(255);
  text("Happiness:", width * 0.015, height * 0.15);
```

```
  happinessbar.setValue(alligator.happiness);
  happinessbar.drawpositive();
}

// ========================
// Minigame Choice Screen State
// ========================

// ========================
// Swamp Hop — Assets
// ========================
PImage swamphopbackground;  // Scrolling background image (drawn twice for seamless loop)

// Four-frame walk cycle sprites for the side-scrolling alligator character
// Obstacle sprites (randomly spawned, each with unique hitbox tuning)

// ========================
// Swamp Hop — Animation State
// ========================
int walkAnimFrameIndex = 1;    // current walk-cycle frame (1–4)
int walkAnimFrameTimer = 0;  // counts up each draw() call; resets at walkAnimFrameDelay


// ========================
// Swamp Hop — Physics
// ========================
float hopVelocityY = 0;    // Current downward velocity (increases each frame by hopGravity)
float hopGroundY;          // Calculated ground threshold (set each frame based on sprite height)


// ========================
// Swamp Hop — Game State
// ========================
boolean isSwampHopLost = true;         // True = showing lose/welcome modal (not actively playing)

// ========================
// Swamp Hop — Obstacle Spawning
// ========================

// ========================
// Swamp Hop — Hitbox Multipliers
// Each obstacle type has its own width/height/Y-offset tuning so collision feels fair.
// Values are fractions of the sprite's actual drawn size (1.0 = full size).
// ========================

// Alligator hitbox (slightly inset so near-misses feel forgiving)
float hopGatorHitboxOffsetY = -8;     // shift upward from sprite center (pixels)

// Vine hitbox (tall but narrow — hangs from above)
// Rock hitbox (wide and low — sits on ground)
// Mud hitbox (same as rock — ground obstacle)
// Log hitbox (full width, slightly reduced height)
// ========================
// Swamp Hop — Scoring
// ========================
int swampHopScore = 0;                  // Current run score (increments once per second survived)
int swampHopBestScore = 0;              // All-time best score across all runs (persisted to save)
float moneyPerMinigamePoint = 0;              // Dollars earned per minigame point (upgradeable in Earn panel)

// ========================
// Snack Snatch — Scoring
// ========================
// ========================
// Fetch Frenzy — Player Position
```

```
// ========================
float snatchPlayerX = 550;            // Horizontal position of the alligator in Fetch Frenzy (top-down)

void resetSwampHop() {
  isSwampHopLost = false;
  isSwampHopRetry = false;
  swampHopScore = 0;
  swampHopScoreTimer = 0;

  x = 224;
  y = 500;
  hopVelocityY = 0;
  isSwampHopOnGround = true;

  swampHopScrollSpeed = 16;
  swampHopBgScrollX = 0;

  swampHopObstacles.clear();
  nextObstacleX = width + 600;

  swampHopGraceFrames = 45;
  swampHopSpawnDelay = 30;
}

void play() {
  isOnChoiceScreen=true;
  fill(255,0,0);
  rect(0,0,width,height);

  imageMode(CENTER);
  image(minigamechoice, width/2, height*0.52,
        (minigamechoice.width/1.4)*1.01, minigamechoice.height/1.4);

  if (isEnterSwampHop) {

    isOnChoiceScreen = false;
    swamphop();
    minigameFade.stepIn(6);  // fade from black to clear as game starts

  } else if (isEnterSnackSnatch) {

    isOnChoiceScreen = false;
    snacksnatch();
    minigameFade.stepIn(6);

  } else if (isEnterFetchFrenzy) {

    isOnChoiceScreen = false;
    fetchfrenzy();
    minigameFade.stepIn(6);

  } else {

    isOnChoiceScreen = true;
    minigameFade.stepIn(3);  // choice screen fades in slightly slower

  }

  // Draw the fade overlay on top of everything
  rectMode(CORNER);
  minigameFade.draw();
}


void swamphop() {
```

```
  swampHopBgWidth = swamphopbackground.width / 1.38;
  swampHopBgHeight = swamphopbackground.height / 1.38;

  imageMode(CORNER);
  image(swamphopbackground, swampHopBgScrollX, 0, swampHopBgWidth, swampHopBgHeight);
  image(swamphopbackground, swampHopBgScrollX + swampHopBgWidth, 0, swampHopBgWidth, swampHopBgHeight);


  strokeWeight(1);
  stroke(0);
  rectMode(CORNER);
  textAlign(LEFT,CENTER);
  textFont(arcade);
  fill(255);
  text("Score: " + swampHopScore, width*0.015, 54);
  drawMinigameStats();

  if (isSwampHopLost) {

    swampHopObstacles.clear();
    nextObstacleX = width + 600;
    imageMode(CENTER);

    applyAlligatorTint();
    image(alligatorf1, x, y, alligatorf1.width/1.5, alligatorf1.height/1.5);
    noTint();
    if (swampHopScore > swampHopBestScore) swampHopBestScore = swampHopScore;
    if (!isSwampHopFirstPlay) {
      drawMinigameModal("YOU LOST!", "High Score: " + swampHopBestScore, "", "", "RETRY", "EXIT");
    } else {
      drawMinigameModal("WELCOME!", "Click space to hop!", "", "", "PLAY", "EXIT");
    }

    strokeWeight(1);
    stroke(0);
    rectMode(CORNER);
    return;
  }
  stroke(0);

  swampHopScoreTimer++;
  if (swampHopScoreTimer % 60 == 0) {
    swampHopScore++;
    earnMinigameMoney(moneyPerMinigamePoint);
  }
  if (isSwampHopOnGround && !isSwampHopLost) {
    swampHopScrollSpeed = 16;
  } else if (!isSwampHopLost) {
    swampHopScrollSpeed = 22;
  }

  imageMode(CORNER);
  if (!isSwampHopLost) tickMinigameStats();

  swampHopBgScrollX -= swampHopScrollSpeed;
  if (swampHopBgScrollX <= -swampHopBgWidth) swampHopBgScrollX = 0;

  float halfH = (alligatorf1.height/1.5) / 2.0;
  hopGroundY = 500 + halfH;

  hopVelocityY += hopGravity;
  y += hopVelocityY;

  if (y + halfH >= hopGroundY) {
```

```
    y = hopGroundY - halfH;
    hopVelocityY = 0;
    isSwampHopOnGround = true;
  }


  imageMode(CENTER);

  walkAnimFrameTimer++;
  if (walkAnimFrameTimer >= walkAnimFrameDelay) {
    walkAnimFrameTimer = 0;
    walkAnimFrameIndex++;
    if (walkAnimFrameIndex > 4) {
      walkAnimFrameIndex = 1;
    }
  }

  applyAlligatorTint();
  if (walkAnimFrameIndex == 1) {
    image(alligatorf1, x, y, alligatorf1.width/1.5, alligatorf1.height/1.5);
  }
  else if (walkAnimFrameIndex == 2) {
    image(alligatorf2, x, y, alligatorf2.width/1.5, alligatorf2.height/1.5);
  }
  else if (walkAnimFrameIndex == 3) {
    image(alligatorf3, x, y, alligatorf3.width/1.5, alligatorf3.height/1.5);
  }
  else if (walkAnimFrameIndex == 4) {
    image(alligatorf4, x, y, alligatorf4.width/1.5, alligatorf4.height/1.5);
  }
  noTint();
  updateAndDrawObstacles();

}


class Obstacle {
  PImage img;
  float ox, oy;
  float w, h;

  Obstacle(PImage img, float ox, float oy, float w, float h) {
    this.img = img;
    this.ox = ox;
    this.oy = oy;
    this.w = w;
    this.h = h;
  }

  void update(float speed) {
    ox -= speed;
  }

  void drawObstacle() {
    image(img, ox, oy, w, h);
  }

  boolean offscreen() {
    return ox + w/2 < 0;
  }
}

void initObstaclesIfNeeded() {
  if (swampHopObstacles.size() == 0) {
    nextObstacleX = width + 200;
```

```
    for (int i = 0; i < 4; i++) spawnObstacle();
  }
}

void spawnObstacle() {
  int t = int(random(4));
  PImage img = log;
  if (t == 1) img = rock;
  if (t == 2) img = mud;
  if (t == 3) img = vine;

  float scale = 5.0;
  if (img == vine) {
    scale = 3.0;
  }
  float ow = img.width / scale;
  float oh = img.height / scale;

  float oy;
  if (img == vine) {
    oy = oh/2;
  } else {
    oy = hopGroundY - oh/2;
  }

  swampHopObstacles.add(new Obstacle(img, nextObstacleX, oy, ow, oh));
  nextObstacleX += obstacleSpawnGap;
}

void updateAndDrawObstacles() {
  if (isSwampHopLost) return;

  if (swampHopGraceFrames > 0) swampHopGraceFrames--;

  if (swampHopSpawnDelay > 0) {
    swampHopSpawnDelay--;
    return;
  }

  initObstaclesIfNeeded();

  float gatorW = alligatorf1.width/1.5;
  float gatorH = alligatorf1.height/1.5;

  float gHitW = gatorW * 0.760;
  float gHitH = gatorH * 0.300;
  float gCenterX = x - 24.0;
  float gCenterY = y + 12.0;

  for (int i = swampHopObstacles.size()-1; i >= 0; i--) {
    Obstacle o = swampHopObstacles.get(i);
    o.update(swampHopScrollSpeed);
    o.drawObstacle();

    float oHitW = o.w;
    float oHitH = o.h;
    float oCenterX = o.ox;
    float oCenterY = o.oy;

    if (o.img == vine) {
      oHitW = o.w * 0.160;
      oHitH = o.h * 0.680;
      oCenterX += 3.0;
      oCenterY += 15.0;
    }
```

```
    else if (o.img == rock) {
      oHitW = o.w * 0.500;
      oHitH = o.h * 0.220;
      oCenterX += 6.0;
      oCenterY += 3.0;
    }
    else if (o.img == mud) {
      oHitW = o.w * 0.560;
      oHitH = o.h * 0.200;
      oCenterX += 6.0;
    }
    else if (o.img == log) {
      oHitW = o.w * 0.8;
      oHitH = o.h * 0.240;
      oCenterX += 6.0;
      oCenterY -= 6.0;
    }

    boolean collisionX = abs(gCenterX - oCenterX) < (gHitW/2 + oHitW/2);
    boolean collisionY = abs(gCenterY - oCenterY) < (gHitH/2 + oHitH/2);

    if (collisionX && collisionY && swampHopGraceFrames == 0) {
      isSwampHopLost = true;
    }

    if (o.offscreen()) {
      swampHopObstacles.remove(i);
    }
  }

  while (swampHopObstacles.size() < 6) {
    spawnObstacle();
  }
}

void snacksnatch() {
  isEnterSnackSnatch=true;
  if (!isSnatchLost) {
  if (isMoveLeft) {
    snatchPlayerX -= 10;
  }

  if (isMoveRight) {
    snatchPlayerX += 10;
  }
  }
  if (snatchPlayerX < 140) snatchPlayerX = 140;
  if (snatchPlayerX > 960) snatchPlayerX = 960;

  if (!isSnatchLost) tickMinigameStats();

  imageMode(CORNER);
  image(mainscreen, 0, 0, width, height);

  imageMode(CENTER);
  applyAlligatorTint();
  image(alligator.energeticalligator, snatchPlayerX, 572,
        alligator.energeticalligator.width/4, alligator.energeticalligator.height/4);
  noTint();

  updateSnackSnatchFalling();

  strokeWeight(1);
  stroke(0);
  rectMode(CORNER);
```

```
  textAlign(LEFT, CENTER);
  textFont(arcade);
  fill(255);
  text("Score: " + snatchScore, width*0.015, 54);
  drawMinigameStats();

  if (isSnatchLost) {
    if (snatchScore > snatchBestScore) snatchBestScore = snatchScore;
    if (!isSnatchFirstPlay) {
      drawMinigameModal("YOU LOST!", "High Score: " + snatchBestScore, "", "", "RETRY", "EXIT");
    } else {
      drawMinigameModal("WELCOME", "Catch food, dodge veggies!", "Use A/D or the right and", "left arrow
keys to move.", "PLAY", "EXIT");
    }
    stroke(0);
  }
}

void updateSnackSnatchFalling() {
  if (isSnatchLost) return;

  if (fallingFoods.size() == 0) {
    fallingFoods.add(new FallingFood(randomFood()));
  }

  for (int i = fallingFoods.size()-1; i >= 0; i--) {
    FallingFood f = fallingFoods.get(i);
    f.update();
    float gW = alligator.energeticalligator.width/4.0;
    float gH = alligator.energeticalligator.height/4.0;

    float gCenterX = snatchPlayerX + snatchGatorOffsetX;
    float gCenterY = 572 + snatchGatorOffsetY;

    float gHitW = gW * snatchGatorWidthMult;
    float gHitH = gH * snatchGatorHeightMult;

    float wMult=0.5, hMult=0.5, ox=0, oy=0;

    if (f.img == bluegill) { wMult=bluegillW; hMult=bluegillH; ox=bluegillOX; oy=bluegillOY; }
    else if (f.img == bass) { wMult=bassW; hMult=bassH; ox=bassOX; oy=bassOY; }
    else if (f.img == perch) { wMult=perchW; hMult=perchH; ox=perchOX; oy=perchOY; }
    else if (f.img == goldfish) { wMult=goldfishW; hMult=goldfishH; ox=goldfishOX; oy=goldfishOY; }
    else if (f.img == crab) { wMult=crabW; hMult=crabH; ox=crabOX; oy=crabOY; }
    else if (f.img == lambchop) { wMult=lambchopW; hMult=lambchopH; ox=lambchopOX; oy=lambchopOY; }
    else if (f.img == porkchop) { wMult=porkchopW; hMult=porkchopH; ox=porkchopOX; oy=porkchopOY; }
    else if (f.img == broccoli) { wMult=broccoliW; hMult=broccoliH; ox=broccoliOX; oy=broccoliOY; }
    else if (f.img == carrot) { wMult=carrotW; hMult=carrotH; ox=carrotOX; oy=carrotOY; }
    else if (f.img == tomato) { wMult=tomatoW; hMult=tomatoH; ox=tomatoOX; oy=tomatoOY; }
    else if (f.img == pepper) { wMult=pepperW; hMult=pepperH; ox=pepperOX; oy=pepperOY; }

    float fHitW = (f.img.width/6.0) * wMult;
    float fHitH = (f.img.height/6.0) * hMult;

    float fCenterX = f.x + ox;
    float fCenterY = f.y + oy;

    boolean hit = abs(gCenterX - fCenterX) < (gHitW/2 + fHitW/2) &&
                  abs(gCenterY - fCenterY) < (gHitH/2 + fHitH/2);

    if (hit) {

      boolean isVeggie = (f.img == broccoli ||
                          f.img == carrot ||
                          f.img == tomato ||
```

```
                          f.img == pepper);

      if (isVeggie) {
        isSnatchLost = true;
      } else {
        snatchScore++;
        earnMinigameMoney(moneyPerMinigamePoint);
      }

      fallingFoods.remove(i);
      continue;
    }
    f.drawFood();

    if (!f.spawnedNext && f.y > height * 0.7 && fallingFoods.size() < 2) {
      fallingFoods.add(new FallingFood(randomFood()));
      f.spawnedNext = true;
    }

    if (f.y > height + 80) {

      boolean isVeggie = (f.img == broccoli ||
                          f.img == carrot ||
                          f.img == tomato ||
                          f.img == pepper);

      if (!isVeggie) {
        isSnatchLost = true;
      }

      fallingFoods.remove(i);
      continue;
    }
  }
}

PImage randomFood() {
  PImage[] weightedFood = {
    bluegill, bluegill,
    bass, bass,
    perch, perch,
    goldfish, goldfish,
    crab, crab,
    lambchop, lambchop,
    porkchop, porkchop,
    broccoli,
    carrot,
    tomato,
    pepper
  };
  return weightedFood[int(random(weightedFood.length))];
}

class FallingFood {
  PImage img;
  float x, y;
  float fallSpeed;
  float angle, rotationSpeed;
  boolean spawnedNext = false;

  FallingFood(PImage img) {
    this.img = img;
    x = random(50, width - 50);
    y = -60;
    fallSpeed = random(4, 8);
```

```
    angle = random(TWO_PI);
    rotationSpeed = random(-0.1, 0.1);
    if (abs(rotationSpeed) < 0.03) rotationSpeed = (rotationSpeed < 0 ? -0.05 : 0.05);
  }

  void update() {
    y += fallSpeed;
    angle += rotationSpeed;
  }

  void drawFood() {
    pushMatrix();
    translate(x, y);
    rotate(angle);
    image(img, 0, 0, img.width/6.0, img.height/6.0);
    popMatrix();
  }
}

// =========================
// Snack Snatch Hitbox Tuning
// Each variable pair (W/H) scales the hitbox relative to the sprite size.
// Each pair (OX/OY) offsets the hitbox center from the sprite center.
// Values were hand-tuned to match each sprite's visual footprint.
// =========================

// Alligator catcher hitbox (applied to the player character)
float snatchGatorOffsetY = -20.0;    // vertical center offset in pixels (shifted up toward mouth)

// Per-food hitbox tuning (W/H = size fraction, OX/OY = center offset)
// Vegetables (touching these causes a loss)
void resetSnackSnatch() {
  snatchScore = 0;
  isSnatchLost = false;
  isSnatchFirstPlay = false;
  fallingFoods.clear();
  fallingFoods.add(new FallingFood(randomFood()));
  snatchPlayerX=width/2;
}

class RectHB {
  float offX, offY, w, h;
  RectHB(float offX, float offY, float w, float h) {
    this.offX = offX;
    this.offY = offY;
    this.w = w;
    this.h = h;
  }
}

void initFetchFrenzyPositions() {
  fetchPlayerX = width/2;
  fetchPlayerY = height/2;
}

RectHB rotatedGatorHB(RectHB hb, String dir) {
  float ox = hb.offX, oy = hb.offY;
  float w = hb.w, h = hb.h;

  if (dir.equals("UP")) {
    return new RectHB(ox, oy, w, h);
  } else if (dir.equals("RIGHT")) {
    return new RectHB(-oy, ox, h, w);
  } else if (dir.equals("DOWN")) {
    return new RectHB(-ox, -oy, w, h);
```

```
  } else if (dir.equals("LEFT")) {
    return new RectHB(oy, -ox, h, w);
  }
  return new RectHB(ox, oy, w, h);
}

void fetchfrenzy() {
  imageMode(CENTER);
  image(fetchfrenzybackground, width/2, height/2, width, height);

  if (isBallOnField) updateBall();
  if (isBallOnField && !isFetchLost) fetchTimer -= 1.0/60.0;
  if (!hasBallBeenLaunched && !isFetchLost) {
    launchBall();
    hasBallBeenLaunched = true;
  }

  if (!isFetchLost) tickMinigameStats();

  walkanimation();

  RectHB gatorHBRot = rotatedGatorHB(gatorHB, fetchFacingDirection);

  wasBallHittingPlayer = ballHitsPlayer;
  ballHitsPlayer = hitRectRectCenter(ballX, ballY - ballZ, ballHB, fetchPlayerX, fetchPlayerY, gatorHBRot);

if (ballHitsPlayer && !wasBallHittingPlayer && isBallOnField && !isBallWaitingToLaunch) {
  isBallOnField = false;
  fetchScore++;
  earnMinigameMoney(moneyPerMinigamePoint);
  isBallWaitingToLaunch = true;
  ballLaunchCountdown = LAUNCH_DELAY;
}

if (isBallWaitingToLaunch) {
  ballLaunchCountdown--;

  if (ballLaunchCountdown <= 0) {
    launchBall();
    fetchTimer=3;
    isBallOnField = true;
    isBallWaitingToLaunch = false;
  }
}

  wasBallHittingPlayer = ballHitsPlayer;

  strokeWeight(1);
  stroke(0);
  rectMode(CORNER);
  textAlign(LEFT,CENTER);
  textFont(arcade);
  fill(255);
  text("Score: " + fetchScore, width*0.015, 30);
  text("Time: " + nf(fetchTimer, 0, 2), width*0.015, 70);
  drawMinigameStats();

  if (fetchScore>fetchBestScore) fetchBestScore=fetchScore;
  if (fetchTimer<=0) {
    isFetchLost=true;
    fetchTimer=0;
  }
  if (isFetchLost) {
    if (!isFetchFirstPlay) {
      drawMinigameModal("YOU LOST!", "High Score: " + fetchBestScore, "", "", "RETRY", "EXIT");
```

```
    } else {
      drawMinigameModal("WELCOME", "Fetch the ball in the", "allotted time using WASD", "or the arrow keys!", "PLAY", "EXIT");
    }
    stroke(0);
  }
}

void walkanimation() {
  if (fetchFacingDirection.equals("DOWN")  && fetchPlayerY > 395 && fetchPlayerX > 456 && fetchPlayerX < 631) fetchPlayerY = 394;
  if (fetchFacingDirection.equals("RIGHT") && fetchPlayerY > 504 && fetchPlayerX > 371 && fetchPlayerX < 500) fetchPlayerX = 371;
  if (fetchFacingDirection.equals("LEFT")  && fetchPlayerY > 504 && fetchPlayerX > 600 && fetchPlayerX < 746) fetchPlayerX = 747;

  if (fetchPlayerY < 96) fetchPlayerY = 97;
  if (fetchPlayerX < 100) fetchPlayerX = 101;
  if (fetchPlayerX > 1000) fetchPlayerX = 999;
  if (fetchPlayerY > (height - 96)) fetchPlayerY = height - 97;

  if (!isFetchLost) {
  if (fetchFacingDirection.equals("LEFT")) {
    fetchPlayerX -= 5;
  } else if (fetchFacingDirection.equals("RIGHT")) {
    fetchPlayerX += 5;
  } else if (fetchFacingDirection.equals("UP")) {
    fetchPlayerY -= 5;
  } else if (fetchFacingDirection.equals("DOWN")) {
    fetchPlayerY += 5;
  }
  }

  pushMatrix();
  translate(fetchPlayerX, fetchPlayerY);

  if (fetchFacingDirection.equals("DOWN")) {
    rotate(radians(180));
  } else if (fetchFacingDirection.equals("RIGHT")) {
    rotate(radians(90));
  } else if (fetchFacingDirection.equals("LEFT")) {
    rotate(radians(270));
  }

  if (!isFetchLost) {
  applyAlligatorTint();
  fetchWalkFrameTimer++;
  if (fetchWalkFrameTimer < 7) {
    image(topalligator1, 0, 0, topalligator1.width/4, topalligator1.height/4);
  } else if (fetchWalkFrameTimer < 14) {
    image(topalligator2, 0, 0, topalligator2.width/4, topalligator2.height/4);
  } else {
    fetchWalkFrameTimer = 0;
    image(topalligator1, 0, 0, topalligator1.width/4, topalligator1.height/4);
  }
  noTint();
  }

  popMatrix();
}

void launchBall() {
  ballX = 548;
  ballY = 618;

```

```
  float speed = 11;
  float baseAngle = -HALF_PI;
  float spread = radians(180);
  float angle = baseAngle + random(-spread, spread);

  ballVelocityX = cos(angle) * speed;
  ballVelocityY = sin(angle) * speed;

  ballZ = 0;
  ballVerticalVelocity = 22;

  isBallMoving = true;
}

void updateBall() {
  float ballW = ball.width / 10.0;
  float ballH = ball.height / 10.0;
  float halfW = ballW * 0.5;
  float halfH = ballH * 0.5;

  if (isBallMoving) {
    ballVerticalVelocity -= ballGravity;
    ballZ  += ballVerticalVelocity;

    ballX += ballVelocityX;
    ballY += ballVelocityY;

    ballVelocityX *= ballAirDrag;
    ballVelocityY *= ballAirDrag;

    if (ballZ <= 0) {
      ballZ = 0;

      if (abs(ballVerticalVelocity) > ballStopBounceThreshold) ballVerticalVelocity = -ballVerticalVelocity
* ballBounceFactor;
      else ballVerticalVelocity = 0;

      ballVelocityX *= ballGroundFriction;
      ballVelocityY *= ballGroundFriction;

      if (ballVerticalVelocity == 0 && abs(ballVelocityX) < ballStopSpeedThreshold && abs(ballVelocityY) <
ballStopSpeedThreshold) {
        ballVelocityX = 0;
        ballVelocityY = 0;
        isBallMoving = false;
      }
    }

    if (ballX - halfW < 0)        { ballX = halfW;        ballVelocityX *= -0.8; }
    if (ballX + halfW > width)    { ballX = width-halfW;  ballVelocityX *= -0.8; }
    if (ballY - halfH < 0)        { ballY = halfH;        ballVelocityY *= -0.8; }
    if (ballY + halfH > height)   { ballY = height-halfH; ballVelocityY *= -0.8; }
  }

  float maxHeight = 70.0;
  float zN = constrain(ballZ / maxHeight, 0, 1);

  float baseShadow = ballW * 0.45;
  float shadowGrow = ballW * 0.15;
  float shadowSize = baseShadow + shadowGrow * zN;

  float shadowAlpha = 120 - (60 * zN);

  noStroke();
  fill(0, shadowAlpha);
```

```
  ellipse(ballX, ballY, shadowSize, shadowSize);

  image(ball, ballX, ballY - ballZ, ballW, ballH);
}

boolean hitRectRectCenter(float aCx, float aCy, RectHB a, float bCx, float bCy, RectHB b) {
  float aX = aCx + a.offX;
  float aY = aCy + a.offY;
  float bX = bCx + b.offX;
  float bY = bCy + b.offY;

  float aHalfW = a.w * 0.5;
  float aHalfH = a.h * 0.5;
  float bHalfW = b.w * 0.5;
  float bHalfH = b.h * 0.5;

  return (abs(aX - bX) < (aHalfW + bHalfW) &&
          abs(aY - bY) < (aHalfH + bHalfH));
}

void resetFetchFrenzy() {
  fetchTimer=3;
  fetchScore=0;
  fetchPlayerX = width/2;
  fetchPlayerY = height*0.25;
  isFetchLost = false;
  hasBallBeenLaunched = false;
}
```

§ 16

I_Alligator_Class.pde

# Pet Class

Pet class, eat() method with 40+ foods, mood sprites, stat clamping, skin tint system

```
// ========================
// Pet Class
// Represents the player's virtual alligator pet.
// Stores all per-pet state: identity, mood sprites, and the five core stats.
// The eat() method is the primary way stats change from player-initiated feeding.
// ========================

class Pet {

  // --- Identity ---
  String petName;              // Display name chosen by the player on the naming screen

  // --- Mood sprites (loaded in D_General_Functions.pde :: fileWork()) ---
  // Displayed based on which stat threshold is triggered each frame
  PImage neutralalligator;     // Default sprite — no critical stat triggered
  PImage hungryalligator;      // Shown when hunger > 80
  PImage energeticalligator;   // Shown when energy > 80 (too hyper)
  PImage sickalligator;        // Shown when isPetSick == true

  // --- Core Stats (all 0–100 unless noted) ---
  // health:    Overall well-being. Drops from junk food, unprescribed medicine, or illness.
  // happiness: Mood level. Rises from treats and play; falls from illness/bad food.
  // energy:    Activity level. Best in mid-range (40–70). Too high triggers energetic mood.
  // sickrisk:  Probability of falling ill next day. Clamped min 20 (baseline risk always exists).
  // hunger:    How full the pet is. Rises each day; high hunger triggers the hungry mood.
  float health = 100;
  float happiness = 100;
  float energy = 50;
  float sickrisk = 10;
  float hunger = 95;


  // Constructor: create a new pet with a given name
  Pet(String _petName) {
    petName = _petName;
  }

void neutralmood() {
  applyAlligatorTint();
  image(neutralalligator, width * 0.33, height * 0.45, 267 * 1.4, 187 * 1.4);
  noTint();
}

void hungrymood()   { drawMoodSprite(hungryalligator); }
void energeticmood() { drawMoodSprite(energeticalligator); }
void sickmood()     { drawMoodSprite(sickalligator); }

void drawMoodSprite(PImage img) {
  applyAlligatorTint();
  imageMode(CENTER);
  image(img, 550, 452, img.width/4.2, img.height/4.2);
  imageMode(CORNER);
  noTint();
}

```

```
// ========================
// eat(String item)
// Central stat-update method called whenever the player uses an inventory item.
// Logic flow:
//   1. Steak (premium meat from adoption center) — biggest hunger/health boost
//   2. Medicines — check medicineIsPrescribed[] first; prescribed = heals, unprescribed = harms
//   3. Snacks / Meat — each has unique stat tradeoffs (junk hurts health, fish/meat helps)
// All stats are clamped at the end to prevent overflow or underflow.
// ========================
void eat(String item) {

// --- Premium Food ---
// Steak is the player's starting item; best all-around nutrition.
if (item.equals("Steak")) {
    hunger-=70;
    energy+=40;
    health+=20;
    happiness+=20;
} else {

  // --- Medicine ---
  // Each medicine maps to an index in medicineItemList[].
  // If the vet prescribed it (prescribedMedicineIndex == i), it heals the pet.
  // If given without a prescription, it harms health and raises sickness risk —
  // teaching the player that self-medicating without a vet is dangerous.
  boolean foundMedicine = false;
  for (int i = 0; i < medicineItemList.length; i++) {
    if (item.equals(medicineItemList[i])) {
      if (prescribedMedicineIndex == i) {
        // Prescribed: improves all stats
        health += 20;
        energy += 10;
        happiness += 10;
        if (i == 0) hasGivenFirstMedicine = true;  // tracks first medicine use for tutorial
      } else {
        // Unprescribed: penalizes health and raises future illness risk
        health -= 35;
        energy -= 10;
        happiness -= 10;
        sickrisk += 25;
      }
      foundMedicine = true;
      break;
    }
  }

  if (!foundMedicine) {
  // --- Snacks (junk food) ---
  // Generally reduce hunger and boost happiness/energy in the short term,
  // but damage health — reflecting real-world junk food consequences.
  if (item.equals("Nachos")) {
    hunger -= 40;
    health -= 15;
    happiness += 5;
    energy += 30;

  } else if (item.equals("Cheesepuffs")) {
    hunger -= 25;
    health -= 5;
    happiness += 20;
    energy += 10;

  } else if (item.equals("Chips")) {
    hunger -= 30;
    health -= 10;
```

```
        happiness += 10;
        energy += 10;

      } else if (item.equals("Chocolate Bar")) {
        hunger -= 7;
        health -= 5;
        happiness += 20;
        energy += 5;

      } else if (item.equals("Cookies")) {
        hunger -= 30;
        health -= 5;
        happiness += 10;
        energy += 5;

      } else if (item.equals("Crackers")) {
        hunger -= 5;
        energy += 20;

      } else if (item.equals("Energy Drink")) {
        hunger += 20;
        health -= 35;
        happiness -= 10;
        energy += 70;

      } else if (item.equals("Granola Bar")) {
        hunger -= 15;
        health -= 5;
        happiness += 5;
        energy += 20;

      } else if (item.equals("Popcorn")) {
        hunger -= 10;
        health -= 10;
        happiness += 30;
        energy += 10;

      } else if (item.equals("Pretzels")) {
        hunger -= 10;
        health -= 5;
        happiness += 5;
        energy += 15;

      } else if (item.equals("Soda")) {
        hunger += 30;
        health -= 30;
        happiness += 5;
        energy += 40;

      } else if (item.equals("Trail Mix")) {
        hunger -= 10;
        health -= 5;
        happiness += 5;
        energy += 20;

      // --- Meat / Fish (premium foods) ---
      // Natural diet for alligators; improve health and hunger the most.
      // Better nutritional value than snacks — player learns to invest in quality food.
      } else if (item.equals("Bluegill")) {
        hunger -= 55;
        health += 15;
        happiness += 10;
        energy += 20;

      } else if (item.equals("Bass")) {
```

```
186 ■       hunger -= 60;
187 ■       health += 20;
188 ■       happiness += 10;
189 ■       energy += 25;
190 ■
191 ■     } else if (item.equals("Perch")) {
192 ■       hunger -= 50;
193 ■       health += 20;
194 ■       happiness += 5;
195 ■       energy += 15;
196 ■
197 ■     } else if (item.equals("Goldfish")) {
198 ■       hunger -= 30;
199 ■       health += 5;
200 ■       happiness += 15;
201 ■       energy += 10;
202 ■
203 ■     } else if (item.equals("Crab")) {
204 ■       hunger -= 50;
205 ■       health += 25;
206 ■       happiness += 20;
207 ■       energy += 15;
208 ■
209 ■     } else if (item.equals("Lamb Chop")) {
210 ■       hunger -= 70;
211 ■       health += 10;
212 ■       happiness += 15;
213 ■       energy += 35;
214 ■
215 ■     } else if (item.equals("Pork Chop")) {
216 ■       hunger -= 65;
217 ■       health += 10;
218 ■       happiness += 10;
219 ■       energy += 30;
220 ■
221 ■     } else if (item.equals("Chicken")) {
222 ■       hunger -= 60;
223 ■       health += 20;
224 ■       happiness += 10;
225 ■       energy += 20;
226 ■
227 ■     } else if (item.equals("Catfish")) {
228 ■       hunger -= 60;
229 ■       health += 15;
230 ■       happiness += 10;
231 ■       energy += 25;
232 ■
233 ■     } else if (item.equals("Frog")) {
234 ■       hunger -= 45;
235 ■       health += 15;
236 ■       happiness += 25;
237 ■       energy += 20;
238 ■
239 ■     } else if (item.equals("Shrimp")) {
240 ■       hunger -= 35;
241 ■       health += 25;
242 ■       happiness += 15;
243 ■       energy += 10;
244 ■     }
245 ■     } // end !foundMedicine
246 ■   } // end else (not Steak)
247 ■
248 ■   // --- Stat Clamping ---
249 ■   // After any food effect, constrain every stat to its valid range.
250 ■   // sickrisk minimum is 20 (not 0) because a baseline illness risk always exists.
```

```
  hunger   = clampStat(hunger,   0,  100);
  health   = clampStat(health,   0,  100);
  energy   = clampStat(energy,   0,  100);
  happiness= clampStat(happiness,0,  100);
  sickrisk = clampStat(sickrisk, 20, 100);

  // Close the inventory panel after the player uses an item
  isInventoryVisible = false;
}
}

float clampStat(float val, float lo, float hi) {
  return constrain(val, lo, hi);
}

// ========================
// Tint helper — applies color tint based on selected alligator skin
// Accessible globally from all files
// ========================
void applyAlligatorTint() {
  if (selectedAlligatorSkin == 1) tint(0, 255, 0, 255);
  else if (selectedAlligatorSkin == 2) tint(70, 130, 255, 255);
}
```